
python™

PYTHON

Table of Content

- **Chapter 1(Introduction)** (4-47)
  - 1.1 Features
  - 1.2 Installation of python 3.7
  - 1.3 Variable and Data types
  - 1.4 Operators
  - 1.5 Conditional statements
  - 1.6 Looping
  - 1.7 Control Statements
  - 1.8 String Manipulation

- **Chapter 2 (Function)** (48-67)
  - 2.1 Function
  - 2.2 Object Oriented Programming
  - 2.3 Class and object
  - 2.4 Inheritance
  - 2.5 Lists-Introduction
  - 2.6 Tuple**-** Introduction

- **Chapter 3 (Dictionaries)** (68-81)
  - 3.1 Introduction
  - 3.2 working with dictionaries
  - 3.3 Set - Introduction
  - 3.4 File Handling
  - 3.5 Reading/Writing Text and Numbers to/from a File
  - 3.6 The python data analysis library and data frames
- **Chapter 4 (Python Regular Expression & Exception Handling)** (82-92)
  - 4.1 RE Objects
  - 4.2 Finding Pattern in text
  - 4.3 Python Flags
  - 4.4 Python Exception Handling

- **Chapter 5 (Machine Learning with Python)** (93-166)
  - 5.1 Python Libraries- NumPy
  - 5.2 Pandas
  - 5.3 Matplotlib
  - 5.4 Scipy
  - 5.5 Scikit
  - 5.6 **Algorithms** - Linear Regression

5.7 Logistic Regression
5.8 Clustering
5.9 Decision Tree
5.10 Support vector machines
5.11 Naive Bayes

CHAPTER-01

Introduction

Introduction:-Python is a powerful multi-purpose programming language created by Guido van Rossum in 1989 at CWI(Centrum Wiskunde & Informatica) in Netherland.

History:-It has simple easy-to-use syntax, making it the perfect language for someone trying to learn computer programming for the first time as there is no need of main method to print something First version of python release in feb 1991 as (labeled version 0.9.0)

Python Version(with new feature)	Released Date
Python 1.0	January 1994
Python 1.5	December 31, 1997
Python 1.6	September 5, 2000
Python 2.0	October 16, 2000
Python 2.1	April 17, 2001
Python 2.2	December 21, 2001
Python 2.3	July 29, 2003
Python 2.4	November 30, 2004
Python 2.5	September 19, 2006
Python 2.6	October 1, 2008
Python 2.7	July 3, 2010
Python 3.0	December 3, 2008
Python 3.1	June 27, 2009
Python 3.2	February 20, 2011

Python 3.3	September 29, 2012
Python 3.4	March 16, 2014
Python 3.5	September 13, 2015
Python 3.6	December 23, 2016
Python 3.7	June 27, 2018

Python 2 vs Python 3

Python 2.7	Python 3.6
Python 2.7 has two string types: Unicode strings and non-Unicode strings.	All strings are Unicode strings by default in Python 3.6.
The xrange( ) function is used to create iterable objects.	The xrange( ) function is replaced by the range( ) function.
Separate int and long types for non-floating-point numbers.	There is only one integer type, called int, which behaves like the long type.
The return type of division of integers is "int" meaning 5/4 will return 1 instead of 1.25.	The return type of integer division is "float" meaning 5/4 returns 1.25.
Print is a special statement rather than a function.	Print ( ) is a built in function in Python 3.6.

1.1 Features of Python

1) Presence of third-party modules

2) Extensive support libraries(NumPy for numerical calculations, Pandas for data analytics etc)
3) Open source and community development
4) Easy to learn
5) User-friendly data structures
6) High-level language
7) Dynamically typed language(No need to mention data type based on value assigned, it takes data type)
8) Object-oriented language
9) Portable and Interactive
10) Portable across Operating systems

Application of Python

- Web Applications
- Desktop GUI Applications
- Software Development
- Scientific and Numeric
- Business Applications
- Console Based Application
- Audio or Video based Applications
- 3D CAD Applications
- Enterprise Applications
- Applications for Images

Most Popular website using Python

1) Google(Components of Google spider and Search Engine)
2) Yahoo(Maps)
3) YouTube
4) Mozilla

1.2 Python Download and Installation Instructions

You may want to print these instructions before proceeding, so that you can refer to them while downloading and installing Python. Or, just keep this document in your browser. You should read each step completely before performing the action that it describes.

This document shows downloading and installing Python 3.7.4 on Windows 10 in Summer 2019. **You should download and install the latest version of Python.** The current latest (as of Summer 2019) is Python 3.7.4.

Remember that you must install Java, Python, and Eclipse as all 64-bit applications.

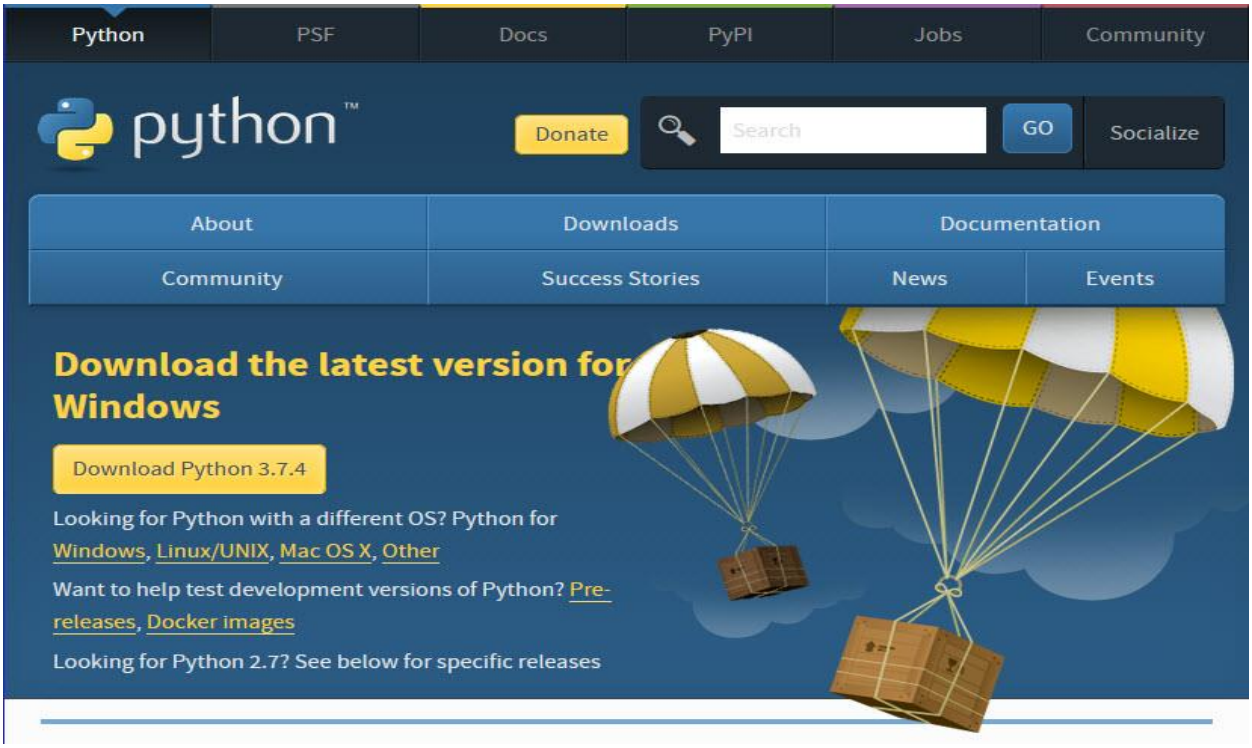
Python: Version 3.7.4

The Python download requires about 25 Mb of disk space; keep it on your machine, in case you need to re-install Python. When installed, Python requires about an additional 90 Mb of disk space.

Downloading

1. Click Python Download.

   The following page will appear in your browser.



2. Click the **Windows** link (two lines below the **Download Python 3.7.4** button). The following page will appear in your browser.



3. Click on the **Download <u>Windows x86-64 executable installer</u>** link under the top-left **Stable Releases**.

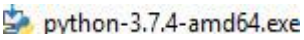
   The following pop-up window titled **Opening python-3.74-amd64.exe** will appear.



Click the **Save File** button.

The file named **python-3.7.4-amd64.exe** should start downloading into your standard download folder. This file is about 30 Mb so it might take a while to download fully if you are on a slow internet connection (it took me about 10 seconds over a cable modem).

The file should appear as

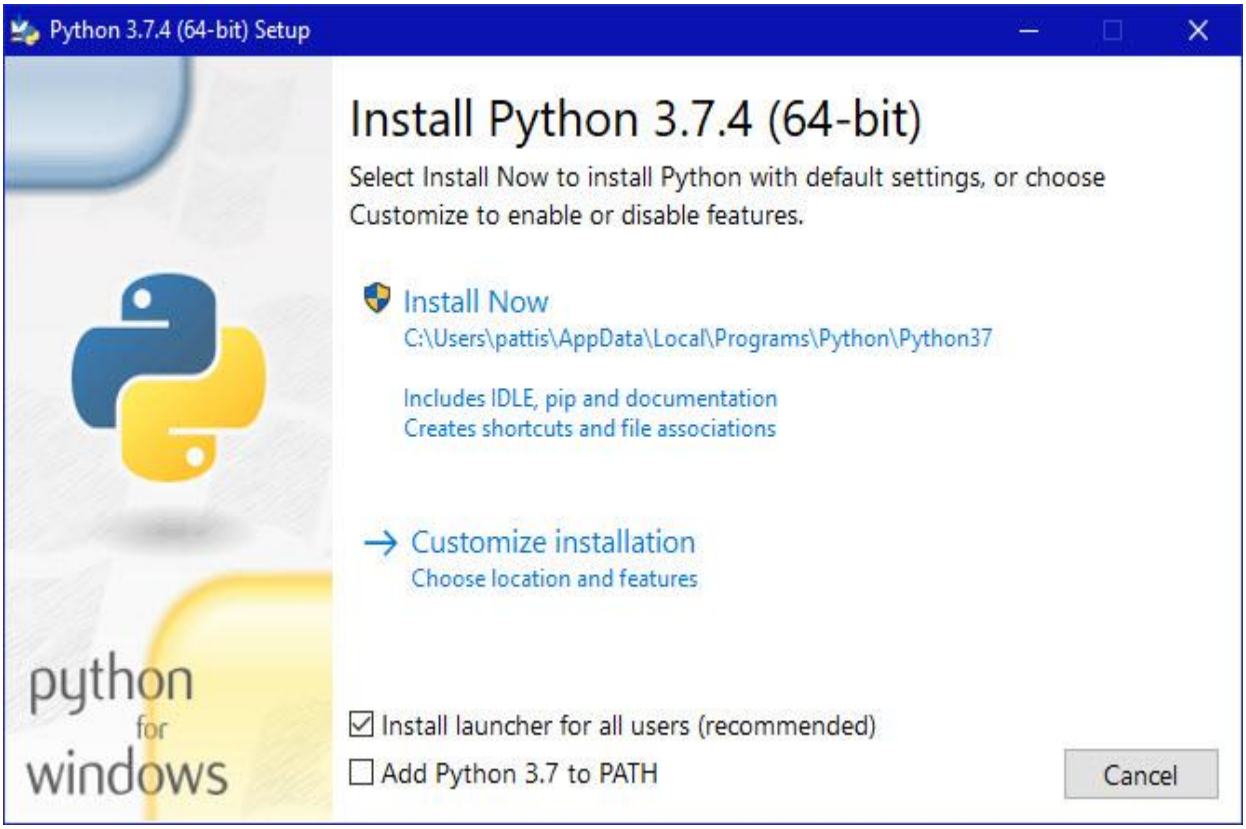


4. Move this file to a more permanent location, so that you can install Python (and reinstall it easily later, if necessary).
5. Feel free to explore this webpage further; if you want to just continue the installation, you can terminate the tab browsing this webpage.
6. Start the **Installing** instructions directly below.

Installing

1. Double-click the icon labeling the file **python-3.7.4-amd64.exe**.

   A **Python 3.7.4 (64-bit) Setup** pop-up window will appear.



Ensure that the **Install launcher for all users (recommended)** and the **Add Python 3.7 to PATH** checkboxes at the bottom are checked.

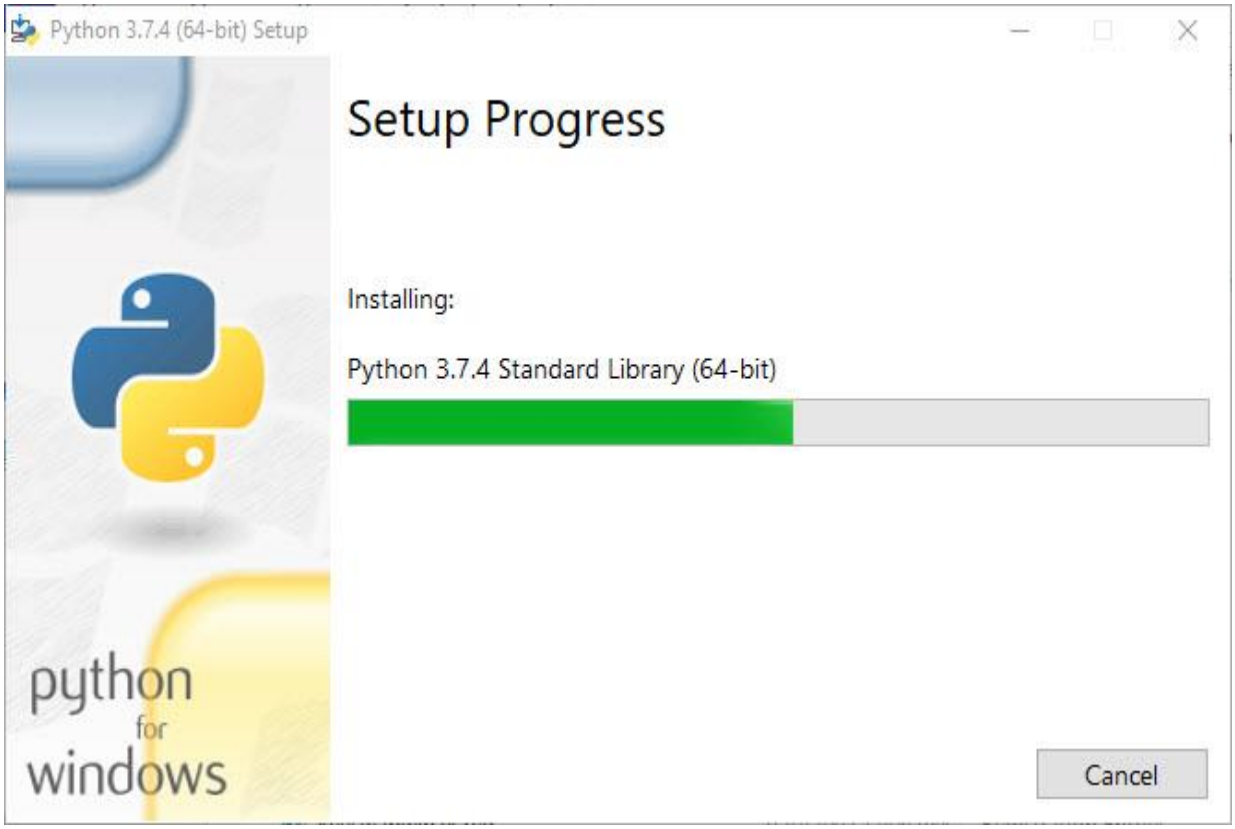
If the Python Installer finds an earlier version of Python installed on your computer, the **Install Now** message may instead appear as **Upgrade Now** (and the checkboxes will not appear).

2. Highlight the **Install Now** (or **Upgrade Now**) message, and then click it.

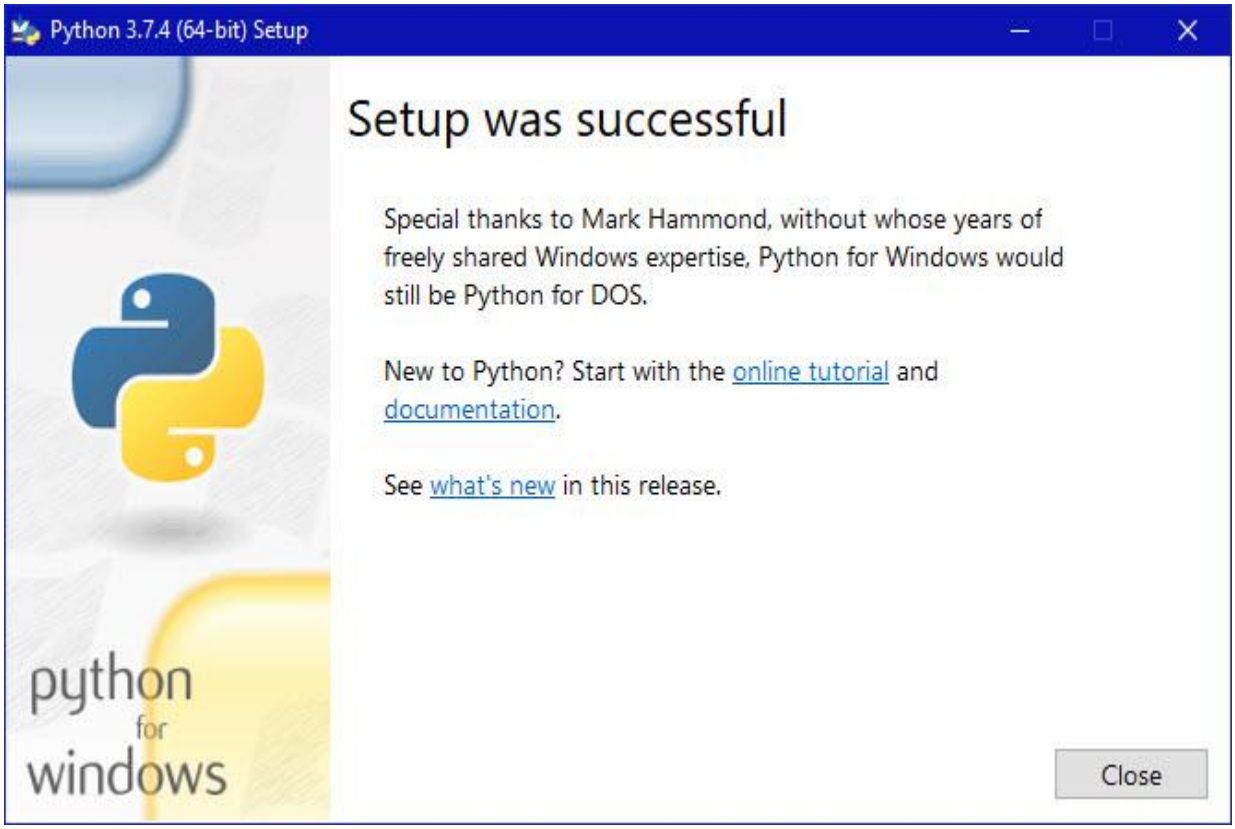
   When run, a **User Account Control** pop-up window may appear on your screen. I could not capture its image, but it asks, **Do you want to allow this app to make changes to your device**.

3. Click the **Yes** button.

   A new **Python 3.7.4 (64-bit) Setup** pop-up window will appear with a **Setup Progress** message and a progress bar.



During installation, it will show the various components it is installing and move the progress bar towards completion. Soon, a new **Python 3.7.4 (64-bit) Setup** pop-up window will appear with a **Setup was successfully** message.



4. Click the **Close** button.

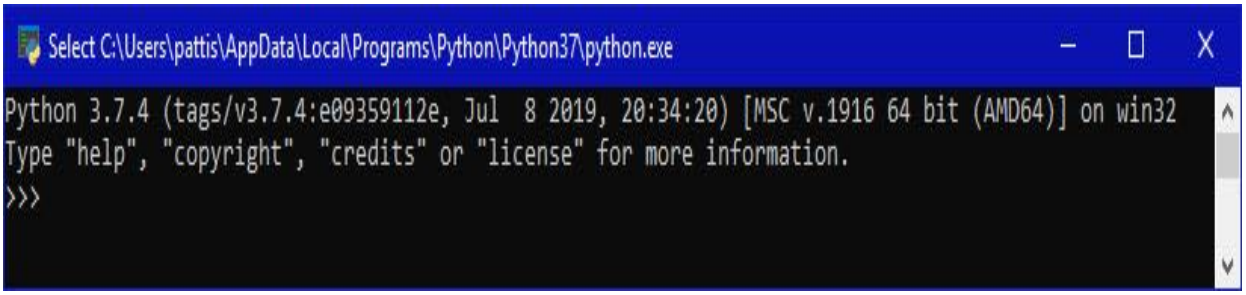
Python should now be installed.

Verifying

To try to verify installation,

1. Navigate to the directory **C:\Users\Pattis\AppData\Local\Programs\Python\Python37** (or to whatever directory Python was installed: see the pop-up window for Installing step 3).
2. Double-click the icon/file **python.exe**.

   The following pop-up window will appear.



A pop-up window with the title **C:\Users\Pattis\AppData\Local\Programs\Python\Python37\python.exe** appears, and inside the window; on the first line is the text **Python 3.7.4 ...** (notice that it should also say 64 bit). Inside the window, at the bottom left, is the prompt **>>>**: type **exit()** to this prompt and press **enter** to terminate Python.

You should keep the file **python-3.7.4.exe** somewhere on your computer in case you need to reinstall Python (not likely necessary).

You may now follow the instructions to download and install Java (you should have already installed Java, but if you haven't, it is OK to do so now, so long as you install both Python and Java before you install Eclipse), and then follows the instruction to download and install the Eclipse IDE. Note: you need to download/install Java even if you are using Eclipse only for Python)

Keyword and identifier

Keyword:- Python Keywords are special reserved words which take a special meaning to the compiler/interpreter. Each keyword has a special meaning and a specific operation. They represent the syntax and structure of a Python program. Since all of these are reserved and have special meaning already defined in the library, so we cannot use keywords for defining variables, classes or functions. All keywords in Python are case sensitive (keywords except true, false and none). So, you must be careful while using them in your code. We've just captured here a snapshot of the possible Python keywords. You can just write the keywords in command prompt and you get the list of various keywords stored in the library.

```
help> keywords

Here is a list of the Python keywords.  Enter any keyword to get more help.

False               def                 if                  raise
None                del                 import              return
True                elif                in                  try
and                 else                is                  while
as                  except              lambda              with
assert              finally             nonlocal            yield
break               for                 not
class               from                or
continue            global              pass
```

Despite this you can use Python's **keyword** module by importing it straight from the shell and run the below commands to view the supported keywords.it will represent the keywords in the form of list as you can see in the image

```
>>> import keyword
>>> keyword.kwlist
['False', 'None', 'True', 'and', 'as', 'assert', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
>>>
```

There are 33 keywords in Python 3.7. This number can vary slightly in the course of time.

Identifier: - An identifier is nothing but a name given to entities like class, functions, variables, etc. It helps to differentiate one entity from another. Python Identifiers are user-defined names represent a variable, function, class, module or any other object. If you assign some name to a programmable entity in Python, then it is nothing but technically called an identifier.

Some Rules for writing identifiers

- Identifiers can be a combination of letters in lowercase **(a to z)** or uppercase **(A to Z)** or digits **(0 to 9)** or an underscore _. For example Names like Class, class1,class_tea and Father_name, all are valid example.
- An identifier cannot start with a digit. For example 1Name is invalid.
- Keywords cannot be used as identifiers.it means we cannot used break, class, for etc for defining variable.
- In first rule you can see that I have used Class for defining then variable because it is not a reserve word 'class'. The reserve word stored in library is 'class' not 'Class'. So using Class, class1 is valid in case of variable

```
>>> class=1
SyntaxError: invalid syntax
>>> True=1
SyntaxError: can't assign to keyword
```

- We cannot use special symbols like **!**, @, **#**, **$**, **%** etc. in our identifier.

1.3 Python variable and comments

Variable

Variable is a name which is used to store the value in a memory location. Variable also known as identifier and used to hold value.

Since python is a type infer language, we don't need to specify the type of variable.it is smart enough to get variable type.

Variable names can be defined using the rules of defining identifier using group of both letters and digits, but they must begin with a letter or an underscore.

Assigning value to variable

With the help of python, we don't need to declare the variable first and then to use it. Python allows us to create variable at required time.

Whenever we assign any value to the variable that variable is declared automatically.

We used equal (=) operator to assign value to a variable.

Eg: These are different ways to assign value to a variable

Single Assignment

```
a=10
b='Mike'
c=20000.67
print(a)
print(b)
print(c)
Output:
10
Mike
20000.67
```

Multiple Assignment

We can assign a value to multiple variable in a single line or statement.

We can use multiple assignments in two ways :

1) either by assigning a single value to multiple variables in a single line/statement or
2) assigning multiple values to multiple variables.

Let's see given examples.

1. Assigning single value to multiple variables

Eg:

```
x=y=z=50
print x
print y
print z
Output
50
50
50
```

2.Assigning multiple values to multiple variables:

Eg:

```
a,b,c=5,10,15
print a
print b
print c
Output:
5
```

```
10
15
```

The values will be assigned in the order in which variables appears.

Python Comments

Comment in python is used in similar way as it is being used in other languages. Comments are basically used to make the code more readable or to know what is exactly going inside the code.

Writing comments is a good programming practice. They are non-executable part of the code, yet quite important in a program. They will be helpful in the case when number of programmers are working on same projects.They will not only help other programmers working on the same project but the testers can also refer them for clarity on white-box testing.

In python we can use single line comment and multiline comment as well

Single line comment

In case user wants to specify a single line comment, then comment must start with ?#?

Eg:

This is single line comment.

```
print "Hello world"
Output:
Hello world
```

Multi Line Comment:

```
Multi lined comment can be given inside triple quotes.
eg:
''' This
    Is
    Multiline comment'''
```

eg:

```
#single line comment
print "Hello world"
'''This is
multiline comment'''
Output:
Hello world
```

Python Data Types

Everything including variables, functions, modules in Python is an object. In python variable are nothing but a labels without any type. It is the value which gets associated with a type. Hence, the same variable, the label can refer the values of different Python data types.

Standard data types

A variable can store different types of values. For example, a name must be stored as a string whereas id must be stored as an integer.

Python provides various standard data types that define the storage method on each of them. The data types defined in Python are given below.

1. Numbers
2. String
3. List
4. Tuple
5. Dictionary
6. Boolean

In this section of the tutorial, we will give a brief introduction of the above data types. We will discuss each one of them in detail later in this tutorial.

Numbers

Number data type is used to stores numeric values. Python creates a Number object when a number is assigned to a variable. For example;

1. a = 3 , b = 5 #a and b are number objects

It supports 4 types of numeric data.

1. int (signed integers such as 6, 45 etc.)
2. long (long integers used for a higher range of values like 900090870L, -0x19876292L, etc.)
3. float (float is used to store floating point numbers like 1.5, 6.9867, 7.58 etc.)
4. complex (complex numbers like 2.43j, 3.0 + 5.7j, etc. [contains an ordered pair, i.e., x + iy where x and y denote the real and imaginary parts respectively])

String

A sequence of one or more characters represented within either single quotes('hello') or double quotes("hello") or triple quotes(''''hello''') is considered as String in Python. Any letter, a number or a symbol could be a part of the sting, only condition is that it should be in either of quotes.

There are various inbuilt functions and operators provided to handle the string.

In the case of string handling, the operator + is used to concatenate two strings as the operation "hello"+" python" returns "hello python".

The operator * is known as repetition operator as the operation "Python " *2 returns "Python Python ".

The following example illustrates the string handling in python.

```
str1 = 'hello world' #string str1
str2 = ' how are you' #string str2
print (str1[0:2]) #printing first two character using slice operator
print (str1[4]) #printing 4th character of the string
print (str1*2) #printing the string twice
print (str1 + str2) #printing the concatenation of str1 and str2
Output:
he
o
hello worldhello world
hello world how are you
```

List

Lists are like arrays in C despite the fact that list can contain data of different types. The items stored in the list are separated with a comma (,) and enclosed within the square brackets [].

We can use slice [:] operators to access the data of the list. The concatenation operator (+) and repetition operator (*) works with the list in the same way as they were working with the strings.

Consider the following example.

```
l = [1, "hi", "python", 2]

print (l[3:]);

print (l[0:2]);

print (l);

print (l + l);

print (l * 3);
```

Output:

```
[2]

[1, 'hi']

[1, 'hi', 'python', 2]

[1, 'hi', 'python', 2, 1, 'hi', 'python', 2]

[1, 'hi', 'python', 2, 1, 'hi', 'python', 2, 1, 'hi', 'python', 2]
```

Tuple

A tuple is similar to the list in many ways. Like lists, tuples also contain the collection of the items of different data types. The items of the tuple are separated with a comma (,) and enclosed in parentheses ().

A tuple is a read-only data structure as we can't modify the size and value of the items of a tuple.

Let's see a simple example of the tuple.

```
t = ("hi", "python", 2)

print (t[1:]);

print (t[0:1]);

print (t);
```

```
print (t + t);
print (t * 3);
print (type(t))
t[2] = "hi";
```

Output:

```
('python', 2)
('hi',)
('hi', 'python', 2)
('hi', 'python', 2, 'hi', 'python', 2)
('hi', 'python', 2, 'hi', 'python', 2, 'hi', 'python', 2)
<type 'tuple'>
Traceback (most recent call last):
  File "main.py", line 8, in <module>
    t[2] = "hi";
TypeError: 'tuple' object does not support item assignment
```

Dictionary

Dictionary is an ordered set of a key-value pair of items. It is like an associative array or a hash table where each key stores a specific value. Key can hold any primitive data type whereas value is an arbitrary Python object.

The items in the dictionary are separated with the comma and enclosed in the curly braces {}.

Consider the following example.

```
d = {1:'Jimmy', 2:'Alex', 3:'john', 4:'mike'};
print("1st name is "+d[1]);
print("2nd name is "+ d[4]);
print (d);
print (d.keys());
```

```
print (d.values());
```

Output:

```
1st name is Jimmy
2nd name is mike
{1: 'Jimmy', 2: 'Alex', 3: 'john', 4: 'mike'}
[1, 2, 3, 4]
['Jimmy', 'Alex', 'john', 'mike']
```

Booleans

A boolean is such a data type that almost every programming language has, and so is Python. Boolean in Python can have two values – **True or False**. These values are constants and can be used to assign or compare boolean values. Follow a simple example given below.

```
condition = False
if condition == True:
    print("You can continue with the prpgram.")
else:
    print("The program will end here.")
```

While making boolean conditions in Python, we can skip the explicit comparison in our code. And we'll still get the same behavior.

```
condition = False
if condition:
    print("You can continue with the prpgram.")
else:
    print("The program will end here.")
```

The above code will yield the same output as gave the previous one. It is because of the statement

```
if condition:
```

is equivalent to,

```
if condition == True:
```

Next, an expression in Python can also produce a boolean result.

1.4 Python Operators

Basic Operators In Python, operators are the special symbols that can manipulate the value of operands. For example, let's consider the expression 1 + 2 = 3. Here, 1 and 2 are called operands, which are the value on which operators operate and the symbol + is called an operator. Python language supports the following types of operators. • Arithmetic Operators • Comparison or Relational Operators • Assignment Operators • Bitwise Operators • Logical Operators • Membership Operators • Identity Operators Let's learn all operators through examples one by one.

Arithmetic Operators :Arithmetic operators are useful for performing mathematical operations on numbers such as addition, subtraction, multiplication, division, etc.

+ Addition x + y = 30

– Subtraction x – y = -10

* Multiplication x * y = 200

/ Division y / x = 2

% Modulus y % x = 0

** Exponent Exponentiation x**b =10

to the power 20

// Floor Division – Integer division rounded toward minus infinity 9//2 = 4 and 9.0//2.0 = 4.0,

Relational Operators

== The condition becomes True, if the values of two operands are equal.(x == y) is not true.

!= The condition becomes True, if the values of two operands are not equal. <> The condition becomes True, if values of two operands are not equal.

(x<>y) is true. This is similar to != operator.

> The condition becomes True, if the value of left operand is greater than the value of right operand.

(x>y) is not true .

< The condition becomes True, if the value of left operand is less than the value of right operand.

(x<y) is true.

>= The condition becomes True, if the value of left operand is greater than or equal to the value of right operand.

(x>= y) is not true.

<= The condition becomes True, if the value of left operand is less than or equal to the value of right operand.

Example

```
x = 10
y = 12

# Output: x > y is False
print('x > y  is',x>y)

# Output: x < y is True
print('x < y  is',x<y)

# Output: x == y is False
print('x == y is',x==y)

# Output: x != y is True
print('x != y is',x!=y)

# Output: x >= y is False
print('x >= y is',x>=y)

# Output: x <= y is True
print('x <= y is',x<=y)
```

Logical Operators

1. **Logical operators:** Logical operators perform **Logical AND**, **Logical OR** and **Logical NOT** operations.

OPERATOR	DESCRIPTION	SYNTAX
and	Logical AND: True if both the operands are true	x and y
or	Logical OR: True if either of the operands is true	x or y
not	Logical NOT: True if operand is false	not x

```
# Examples of Logical Operator
a = True
b = False

# Print a and b is False
print(a and b)

# Print a or b is True
print(a or b)

# Print not a is False
print(not a)
```

Output:

```
False
True
False
```

Bitwise operators: Bitwise operators acts on bits and performs bit by bit operation.

OPERATOR	DESCRIPTION	SYNTAX
&	Bitwise AND	x & y
|	Bitwise OR	x | y
~	Bitwise NOT	~x
^	Bitwise XOR	x ^ y
>>	Bitwise right shift	x>>
<<	Bitwise left shift	x<<

Examples of Bitwise operators

```
a = 10
b = 4

# Print bitwise AND operation
print(a & b)

# Print bitwise OR operation
print(a | b)

# Print bitwise NOT operation
print(~a)
```

```
# print bitwise XOR operation
print(a ^ b)

# print bitwise right shift operation
print(a >> 2)

# print bitwise left shift operation
print(a << 2)

Output:
0
14
-11
14
2
40
```

Assignment operators: Assignment operators are used to assign values to the variables.

OPERATOR	DESCRIPTION	SYNTAX
=	Assign value of right side of expression to left side operand	x = y + z
+=	Add AND: Add right side operand with left side operand and then assign to left operand	a+=b a=a+b
-=	Subtract AND: Subtract right operand from left operand and then assign to left operand	a-=b a=a-b
*=	Multiply AND: Multiply right operand with left operand and then assign to left operand	a*=b a=a*b
/=	Divide AND: Divide left operand with right operand and then assign to left operand	a/=b a=a/b
%=	Modulus AND: Takes modulus using left and right operands and assign result to left operand	a%=b a=a%b
//=	Divide(floor) AND: Divide left operand with right operand and then assign the value(floor) to left operand	a//=b a=a//b
**=	Exponent AND: Calculate exponent(raise power) value using operands and assign value to left operand	a**=b a=a**b
&=	Performs Bitwise AND on operands and assign value to left operand	a&=b a=a&b
|=	Performs Bitwise OR on operands and assign value to left operand	a|=b a=a|b

^=	Performs Bitwise xOR on operands and assign value to left operand	a^=b	a=a^b
>>=	Performs Bitwise right shift on operands and assign value to left operand	a>>=b	a=a>>b
<<=	Performs Bitwise left shift on operands and assign value to left operand	a <<= b	a= a << b

Identity operators-
is and **is not** are the identity operators both are used to check if two values are located on the same part of the memory. Two variables that are equal does not imply that they are identical.

is	True if the operands are identical
is not	True if the operands are not identical

```
a1 = 3
b1 = 3
a2 = 'helloworld'
b2 = 'helloworld'
a3 = [1,2,3]
b3 = [1,2,3]



print(a1 is not b1)



print(a2 is b2)

# Output is False, since lists are mutable.
print(a3 is b3)
```

Output:
False
True
False

Membership operators-
in and **not in** are the membership operators; used to test whether a value or variable is in a sequence.

in	True if value is found in the sequence
not in	True if value is not found in the sequence

```
# Examples of Membership operator
```

```
x = 'john in john'
y = {3:'a',4:'b'}



print('j' in x)


print('John' not in x)


print('john' not in x)


print(3 in y)


print('b' in y)
```

Output:

1. True
2. True
3. False
4. True
5. False

Type Conversion in Python

Type conversion in python is used to convert one datatype to another datatype.
1. int(a,base) : This function converts any data type to integer. 'Base' specifies the base in which string is if data type is string.

float() : This function is used to convert any data type to a floating point number

```
# Python code to demonstrate Type conversion
# using int(), float()


# initializing string
```

```
s = "10010"

# printing string converting to int base 2
c = int(s,2)
print ("After converting to integer base 2 : ", end="")
print (c)

# printing string converting to float
e = float(s)
print ("After converting to float : ", end="")
print (e)

Output:
After converting to integer base 2 : 18
After converting to float : 10010.0
```

- **ord() :** This function is used to convert a **character to integer.**
- **hex() :** This function is to convert **integer to hexadecimal string**.
- **oct() :** This function is to convert **integer to octal string**.

```
# Python code to demonstrate Type conversion
# using  ord(), hex(), oct()

# initializing integer
s = '4'

# printing character converting to integer
c = ord(s)
print ("After converting character to integer : ",end="")
print (c)
```

```
# printing integer converting to hexadecimal string
c = hex(56)
print ("After converting 56 to hexadecimal string : ",end="")
print (c)

# printing integer converting to octal string
c = oct(56)
print ("After converting 56 to octal string : ",end="")
print (c)

Output:
After converting character to integer : 52
After converting 56 to hexadecimal string : 0x38
After converting 56 to octal string : 0o70
```

- **tuple() :** This function is used to **convert to a tuple**.
- **set() :** This function returns the **type after converting to set**.
- **list() :** This function is used to convert **any data type to a list type**.

```
# Python code to demonstrate Type conversion
# using  tuple(), set(), list()

# initializing string
s = 'karan'

# printing string converting to tuple
c = tuple(s)
print ("After converting string to tuple : ",end="")
print (c)
```

```
# printing string converting to set
c = set(s)
print ("After converting string to set : ",end="")
print (c)

# printing string converting to list
c = list(s)
print ("After converting string to list : ",end="")
print (c)
```

Output:

```
After converting string to tuple : ('k', 'a', 'r', 'a', 'n')
After converting string to set : {'a', 'k', 'r', 'n'}
After converting string to list : ['k', 'a', 'r', 'a', 'n']
```

- **dict() :** This function is used to **convert a tuple of order (key,value) into a dictionary**.
- **str() :** Used to **convert integer into a string.**
- **complex(real,imag) :** : This function **converts real numbers to complex(real,imag) number.**

```
# Python code to demonstrate Type conversion
# using  dict(), complex(), str()

# initializing integers
a = 1
b = 2

# initializing tuple
tup = (('a', 1) ,('f', 2), ('g', 3))

# printing integer converting to complex number
c = complex(1,2)
```

```
print ("After converting integer to complex number : ",end="")
print (c)

# printing integer converting to string
c = str(a)
print ("After converting integer to string : ",end="")
print (c)

# printing tuple converting to expression dictionary
c = dict(tup)
print ("After converting tuple to dictionary : ",end="")
print (c)
```

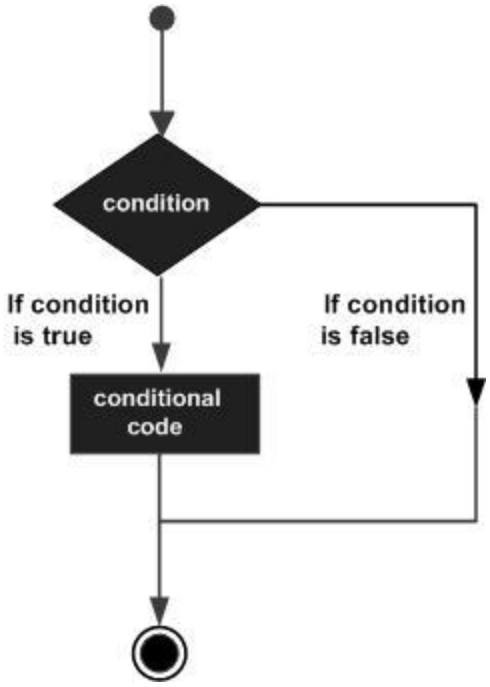
Output:

```
After converting integer to complex number : (1+2j)
After converting integer to string : 1
After converting tuple to dictionary : {'a': 1, 'f': 2, 'g': 3}
```

1.5 Conditional statement

The if, elseif ...else and switch statements are used to take decision based on the different condition.

You can use conditional statements in your code to make your decisions. PYTHON supports following three decision making statements −



The If...Else Statement

If you want to execute some code if a condition is true and another code if a condition is false, use the if....else statement.

Syntax

```
if condition:
   code to be executed if condition is true;
else:
   code to be executed if condition is false;
```

Example

The following example will output "Have a nice weekend!" if the current day is Friday, Otherwise, it will output "Have a nice day!":

```
d = “Friday”
if d == " Friday ":
   print("Have a nice weekend!");

else:
   print("Have a nice day");
```

It will produce the following result −

```
Have a nice day!
```

The ElIf Statement

If you want to execute some code if one of the several conditions are true use the else if statement

Syntax

```
if condition:
  code to be executed if condition is true;
elif condition:
  code to be executed if condition is true;
else:
  code to be executed if condition is false;
```

The following example will output Grade of Student according to %

```
Per=93
if per>=90:
    Print(“A”)
elif per>=80:
    Print(“B”)
elif per>=60:
    Print(“C”)
```

```
elif per>=50:
    Print("D")
```

It will produce the following result −

```
A
```

The Nested if Statement

If you want to execute if within if

Syntax

```
if condition:
   if condition:
      code to be executed if condition is true;
   else:
     code to be executed if condition is false;
else:
   code to be executed if condition is true;
```

The following example will output to check the no b/w 1 to 10

```
a=int (input("Enter the value which you want to check="))
if a>=0:
    if a<=10:
        print("value is b/w 1 to 10")
    else:
        print("Value is larger then 10")
else:
    print("Value is -ve")

Output
Enter the value which you want to check=20
Value is larger then 10
```

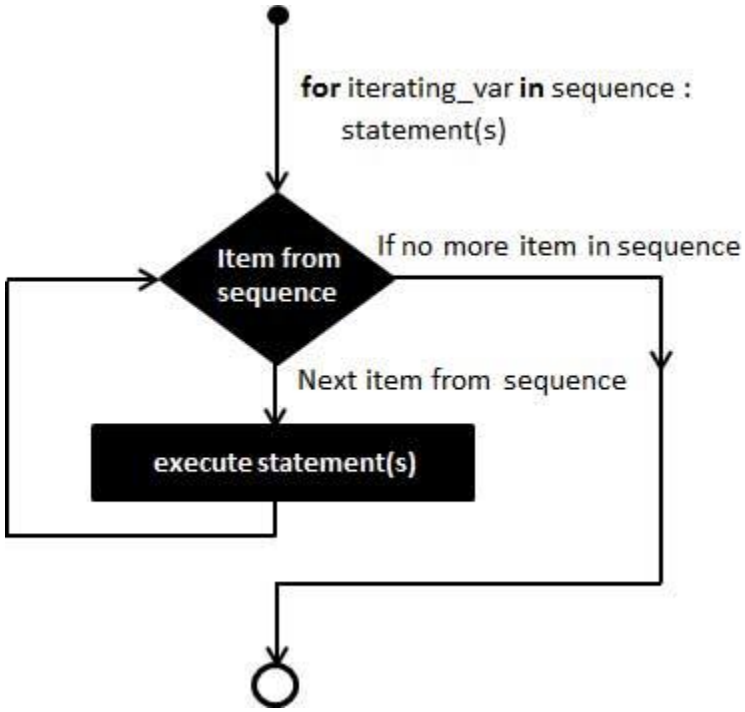
1.6 Loop types

Loops in Python are used to execute the same block of code a specified number of times. Python support three loop types.

- **for − loops through a block of code a specified number of times.**
- **while − loops through a block of code if and as long as a specified condition is true.**

The for loop statement

The for statement is used when you know how many times you want to execute a statement or a block of statements.



Syntax

```
for variable in range(initialization; condition; step-value)
  code to be executed;
```

The initializer is used to set the start value for the counter of the number of loop iterations. A variable may be declared here for this purpose and it is traditional to name it $i.

Example

The following example makes five iterations and print 1 to 5

```
for i in range(1, 6): #By default step value is 1
```

```
        print(i)
```

Output

```
1
2
3
4
5
```

```
for i in range(5, 0,-1):
        print(i)
```

Output

```
5
4
3
2
1
```

Nested For Loop

```
for i in range(1, 5):
    for j in range(i):
        print(i, end=' ')
    print()
```

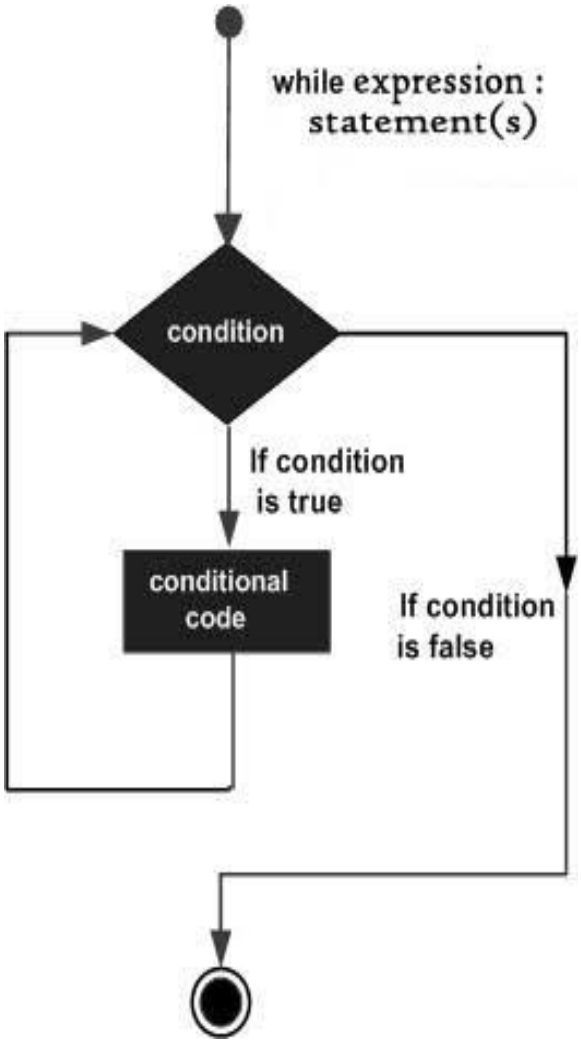
Output

```
1
2 2
3 3 3
4 4 4 4
```

The while loop statement

The while statement will execute a block of code if and as long as a test expression is true.

If the test expression is true then the code block will be executed. After the code has executed the test expression will again be evaluated and the loop will continue until the test expression is found to be false.



Syntax

```
while condition:
  code to be executed;
```

Example

This example decrements a variable value on each iteration of the loop and the counter increments until it reaches 10 when the evaluation is false and the loop ends.

```
i = 0
While i < 10 :
  Print(i)
  i++;
```

This will produce the following result −

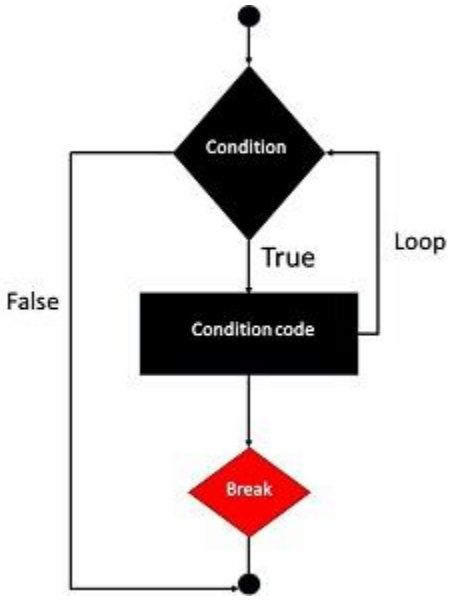
```
1 to 9
```

1.7 Control statements

The break statement

The PYTHON break keyword is used to terminate the execution of a loop prematurely.

The break statement is situated inside the statement block. It gives you full control and whenever you want to exit from the loop you can come out. After coming out of a loop immediate statement to the loop will be executed.



Example

In the following example condition test becomes true when the counter value reaches 3 and loop terminates.

```
for i in range(1, 6):
        if i==3:
         break;
          print(i)
```

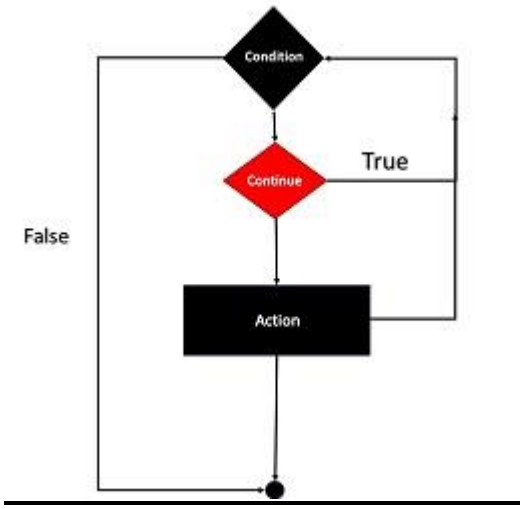
Output

```
1
2
```

The continue statement

The PYTHON continue keyword is used to halt the current iteration of a loop but it does not terminate the loop.

Just like the break statement the continue statement is situated inside the statement block containing the code that the loop executes, preceded by a conditional test. For the pass encountering continue statement, rest of the loop code is skipped and next pass starts.



Example

In the following example loop prints the value of array but for which condition becomes true it just skip the code and next value is printed.

```
for i in range(1, 6):
        if i==3:
         continue;
          print(i)
```

Output

```
1
```

```
2
4
5
```

Pass Statement

We use pass statement to write empty loops. Pass is also used for empty control statement, function and classes.

```
for letter in 'tech-crate':
   pass
print ('Last Letter :', letter )

Output
Last Letter : e
```

1.8 Manipulating Python Strings

If you have been exposed to another programming language before, you might have learned that you need to *declare* or *type* variables before you can store anything in them. This is not necessary when working with strings in Python. We can create a string simply by putting content wrapped with quotation marks into it with an equal sign (=):

To manipulate strings, we can use some of Pythons built-in methods.

Creation

```
word = "Hello World"

>>> print word
Hello World
```

Accessing

```
Use [ ] to access characters in a string

word = "Hello World"
letter=word[0]

>>> print letter
```

```
H
```

Length

```
word = "Hello World"

>>> len(word)
11
```

Finding

```
word = "Hello World"

>>> print word.count('l')        # count how many times l is in the string
3

>>> print word.find("H")        # find the word H in the string
0

>>> print word.index("World")         # find the letters World in the string
6
```

Count

```
s =  "Count, the number     of spaces"

>>> print s.count(' ')
8
```

Slicing

```
Use [ # : # ] to get set of letter

Keep in mind that python, as many other languages, starts to count from 0!!
```

```
word = "Hello World"

print word[0]          #get one char of the word
print word[0:1]        #get one char of the word (same as above)
print word[0:3]        #get the first three char
print word[:3]         #get the first three char
print word[-3:]        #get the last three char
print word[3:]         #get all but the three first char
print word[:-3]        #get all but the three last character

word = "Hello World"

word[start:end]          # items start through end-1
word[start:]           # items start through the rest of the list
word[:end]             # items from the beginning through end-1
word[:]               # a copy of the whole list
```

Split Strings

```
word = "Hello World"

>>> word.split(' ')  # Split on whitespace
['Hello', 'World']
```

Startswith / Endswith

```
word = "hello world"

>>> word.startswith("H")
True

>>> word.endswith("d")
True

>>> word.endswith("w")
False
```

Repeat Strings

```
print "."* 10    # prints ten dots

>>> print "." * 10
..........
```

Replacing

```
word = "Hello World"

>>> word.replace("Hello", "Goodbye")
'Goodbye World'
```

Changing Upper and Lower Case Strings

```
string = "Hello World"

>>> print string.upper()
HELLO WORLD

>>> print string.lower()
hello world

>>> print string.title()
Hello World

>>> print string.capitalize()
Hello world

>>> print string.swapcase()
hELLO wORLD
```

Reversing

```
string = "Hello World"

>>> print ' '.join(reversed(string))
d l r o W   o l l e H
```

Strip

Python strings have the strip(), lstrip(), rstrip() methods for removing
any character from both ends of a string.

If the characters to be removed are not specified then white-space will be removed

word = "Hello World"

Strip off newline characters from end of the string

```
>>> print word.strip('
')
Hello World

strip()     #removes from both ends
lstrip()    #removes leading characters (Left-strip)
rstrip()    #removes trailing characters (Right-strip)

>>> word = "    xyz    "

>>> print word
   xyz

>>> print word.strip()
xyz

>>> print word.lstrip()
xyz

>>> print word.rstrip()
   xyz
```

Concatenation

To concatenate strings in Python use the "+" operator.

"Hello " + "World" # = "Hello World"
"Hello " + "World" + "!"# = "Hello World!"

Join

```
>>> print ":".join(word)  # #add a : between every char
H:e:l:l:o: :W:o:r:l:d

>>> print " ".join(word)  # add a whitespace between every char
H e l l o   W o r l d
```

Testing

A string in Python can be tested for truth value.

The return type will be in Boolean value (True or False)

```
word = "Hello World"

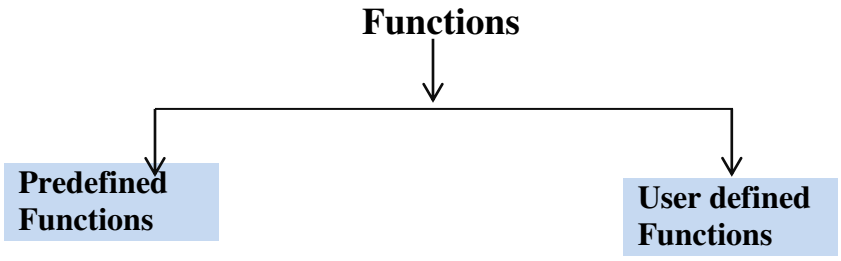
word.isalnum()        #check if all char are alphanumeric
word.isalpha()        #check if all char in the string are alphabetic
word.isdigit()        #test if string contains digits
word.istitle()        #test if string contains title words
word.isupper()        #test if string contains upper case
word.islower()        #test if string contains lower case
word.isspace()        #test if string contains spaces
word.endswith('d')    #test if string endswith a d
word.startswith('H')  #test if string startswith H
```

CHAPTER-02

Function

2.1 FUNCTION

Function:- Python allows us to divide a complex program into the basic building blocks known as function. The function contains the set of programming statements enclosed by {}. A function can be called multiple times to provide reusability and modularity to the python program.



Predefined function e.g math.pow, math.pi

User-defined function has four type

Arguments	Return
1. without	Without
2. with	Without
3. with	With
4. without	With

1. **Type 1 e.g**

```
def show():
  a=10
  b=20
  c=a+b
  print(c)
show()
```

2. **Type 2 e.g**

```
def show(a,b):
  c=a+b
```

```
  print(c)
show(10,20)
```

3. **Type 3 e.g**

```
def show(a,b):
  c=a+b
  return
show(10,40)
```

4. **Type 1 e.g**

```
def show():
  a=10
  b=20
  c=a+b
  return c
show()
```

Recursion:- Recursion is the process of calling itself. In other words when a process called itself till the particular condition true it is known as recursion

```
. def sum():
  a=int(input('enter the value of a'))
  b=int(input('enter the value of b'))
  c=a+b
  print(c)
  choice=int(input("Do you wnat to repeat this program"))
  if choice==1:
      sum()
   else:
      print("Wrong choice")
sum()
```

Anonymous :- Anonymous function is a function that is defined without a name.
While normal functions are defined using the def keyword, in Python anonymous functions are defined using the lambda keyword

```
double = lambda x: x * 2

# Output: 10
print(double(5))
```

Lambda functions also used along with built-in functions like filter(), map() etc.

Example

```
list = [1, 5, 4, 6, 8, 11, 3, 12]

new_list = list(filter(lambda x: (x%2 == 0) ,list))

# Output: [4, 6, 8, 12]
print(list)

list = [1, 5, 4, 6, 8, 11, 3, 12]

list = list(map(lambda x: x * 2 , list))

# Output: [2, 10, 8, 12, 16, 22, 6, 24]
print(list)
```

Scope of Variables

The availability of a variable or identifier within the program during and after the execution is determined by the scope of a variable. There are two fundamental variable scopes in Python.
1. Global variables
2. Local variables

Example

```
# Global variable
a = 10
# Simple function to add two numbers
def sum_two_numbers(b):
return a + b
# Call the function and print result
print sum_two_numbers(10)
----- output -----
20
```

Default Argument

You can define a default value for an argument of function, which means the function will assume or use the default value in case any value is not provided in the function call for that argument

```
def sum_two_numbers(a,b = 10):
return a + b
```

```
# Call the function and print result
print sum_two_numbers(10)
20
print sum_two_numbers(10, 5)
15
```

Variable Length Arguments

There are situations when you do not know the exact number of arguments while defining the function and would want the ability to process all the arguments dynamically.

```
Def sample_function(*args):
for a in args:
print a
# Call the function
Sample_function(1,2,3)
1
2
3
```

The **kwargs will give you the ability to handle named or keyword arguments keyword that you have not defined in advance.

```
Def sample_function(**kwargs):
for a in kwargs:
print a, kwargs[a]
# Call the function
sample_function(name='John', age=27)
age 27
name 'John'
```

Module

Module A module is a logically organized multiple independent but related set of codes or functions or classes. The key principle behind module creating is it's easier to understand, use, and has efficient maintainability. You can import a module and the Python interpreter will search for the module in interest in the following sequences

Example code for importing modules

- Import all functions from a module
  - import module_name
  - from modname import*
- Import specific function from module
  - from module_name import function_name

Namespace

Python has an internal dictionary known as namespace that stores each variable or identifier name as the key and their corresponding value is the respective Python object. There are two types of namespace, local and global. The local namespace gets created during execution of a Python program to hold all the objects that are being created by the program. The local and global variable have the same name and the local variable shadows the global variable. Each class and function has its own local namespace. Python assumes that any variable assigned a value in a function is local. For global variables you need to explicitly specify them. Another key built-in function is the dir(), and running this will return a sorted list of strings containing the names of all the modules, variables, and functions that are defined in a module.

```
Import os
content = dir(os)
print (content)

---- output ---
['F_OK', 'O_APPEND', 'O_BINARY', 'O_CREAT', 'O_EXCL', 'O_NOINHERIT',
'O_RANDOM', 'O_RDONLY', 'O_RDWR', 'O_SEQUENTIAL', 'O_SHORT_LIVED',
'O_TEMPORARY', 'O_TEXT', 'O_TRUNC', 'O_WRONLY', 'P_DETACH', 'P_NOWAIT',
'P_NOWAITO', 'P_OVERLAY', 'P_WAIT', 'R_OK', 'SEEK_CUR', 'SEEK_END', 'SEEK_ SET',
'TMP_MAX', 'UserDict', 'W_OK', 'X_OK', '_Environ', '__all__', '__ builtins__', '__doc__',
'__file__', '__name__', '__package__', '_copy_reg', '_execvpe', '_exists', '_exit',
'_get_exports_list', '_make_stat_result', '_make_statvfs_result', '_pickle_stat_result',
'_pickle_statvfs_result', 'abort', 'access', 'altsep', 'chdir', 'chmod', 'close', 'closerange', 'curdir',
'defpath', 'devnull', 'dup', 'dup2', 'environ', 'errno', 'error', 'execl', 'execle', 'execlp', 'execlpe',
'execv', 'execve', 'execvp',

'execvpe', 'extsep', 'fdopen', 'fstat', 'fsync', 'getcwd', 'getcwdu', 'getenv', 'getpid', 'isatty', 'kill',
'linesep', 'listdir', 'lseek', 'lstat', 'makedirs', 'mkdir', 'name', 'open', 'pardir', 'path', 'pathsep', 'pipe',
'popen', 'popen2', 'popen3', 'popen4', 'putenv', 'read', 'remove', 'removedirs', 'rename', 'renames',
'rmdir', 'sep', 'spawnl', 'spawnle', 'spawnv', 'spawnve', 'startfile', 'stat', 'stat_float_times', 'stat_
result', 'statvfs_result', 'strerror', 'sys', 'system', 'tempnam', 'times', 'tmpfile', 'tmpnam', 'umask',
'unlink', 'unsetenv', 'urandom', 'utime', 'waitpid', 'walk', 'write']
```

2.2 Concept of OOPs

Object:-One of the popular approach to solve a programming problem is by creating objects. This is known as Object-Oriented Programming (OOP).

An object has two characteristics:

- Attributes(e.g a marker has colour blue)
- Behavior (e.g marker is use to write)

Class:- A class is a blueprint for the object. Its collection of similar object. E.g

```
class Car:

    # class attribute
    Type1 = "Four wheeler"

    # instance attribute
    def __init__(self, name, old):
        self.name = name
        self.old = old

# instantiate the Parrot class
Maruti = Car("Maruti", 14)
Tata = Car("Tata", 13)

# access the class attributes
print("Maruti is a {}".format(Maruti.__class__.Type1))
print("Tata is also a {}".format(Tata.__class__.Type1))

# access the instance attributes
print("{} is {} years old".format( Maruti.name, Maruti.old))
print("{} is {} years old".format( Tata.name, Tata.old))

Output
Maruti is a Four wheeler
Tata is also a Four wheeler
Maruti is 14 years old
Tata is 13 years old
```

2.3 Define Method in class

```
class Car2:

    # class attribute
    Type1 = "Four wheeler"
    def read(self):
        self.name="Maruti"

    def show(self):
        print(self.name)
        print(__class__.Type1)

obj=Car2()
obj.read()
```

```
obj.show()
```

2.4 Inheritance :- In inheritance, the child class inherit the properties and can access all the data members and functions defined in the parent class. A child class can also provide its specific implementation to the functions of the parent class.

1. **Single Inheritance**

```
class demo
    # class attribute
    Type1 = "Four wheeler"
    def read(self):
        self.name="Maruti"

    def show(self):
        print(self.name)
        print(__class__.Type1)
class Bike(Car2):

    # class attribute
    Type2 = "Two wheeler"
    def read1(self):
        self.name="HONDA"

    def show1(self):
        print(self.name)
        print(__class__.Type2)

obj=Bike()
obj.read()
obj.show()
obj.read1()
obj.show1()

#Output

Maruti
Four wheeler
HONDA
Two wheeler
```

2. **Multilevel Inheritance**

```
class Car1:
```

```
    # class attribute
    Type1 = "three wheeler"
    def read2(self):
        self.name="Auto"

    def show2(self):
        print(self.name)
        print(__class__.Type1)
class Car2(Car1):
    # class attribute
    Type1 = "Four wheeler"
    def read(self):
        self.name="Maruti"

    def show(self):
        print(self.name)
        print(__class__.Type1)
class Bike(Car2):

    # class attribute
    Type2 = "Two wheeler"
    def read1(self):
        self.name="HONDA"

    def show1(self):
        print(self.name)
        print(__class__.Type2)

obj=Bike()
obj.read()
obj.show()
obj.read1()
obj.show1()
obj.read2()
obj.show2()
Output
Maruti
Four wheeler
HONDA
Two wheeler
Auto
three wheeler
```

3. Multiple Inheritance

```
class Car1:
    # class attribute
    Type1 = "three wheeler"
    def read2(self):
        self.name="Auto"

    def show2(self):
        print(self.name)
        print(__class__.Type1)
class Car2:
    # class attribute
    Type1 = "Four wheeler"
    def read(self):
        self.name="Maruti"

    def show(self):
        print(self.name)
        print(__class__.Type1)
class Bike(Car2,Car1):

    # class attribute
    Type2 = "Two wheeler"
    def read1(self):
        self.name="HONDA"

    def show1(self):
        print(self.name)
        print(__class__.Type2)

obj=Bike()
obj.read()
obj.show()
obj.read1()
obj.show1()
obj.read2()
obj.show2()
Output
Maruti
Four wheeler
HONDA
Two wheeler
Auto
three wheeler
```

4. Hierarchical Inheritance

```
class Car1:
    # class attribute
    Type1 = "three wheeler"
    def read2(self):
        self.name="Auto"

    def show2(self):
        print(self.name)
        print(__class__.Type1)
class Car2(Car1):
    # class attribute
    Type1 = "Four wheeler"
    def read(self):
        self.name="Maruti"

    def show(self):
        print(self.name)
        print(__class__.Type1)
class Bike(Car1):

    # class attribute
    Type2 = "Two wheeler"
    def read1(self):
        self.name="HONDA"

    def show1(self):
        print(self.name)
        print(__class__.Type2)

obj=Bike()
obj1=Car2()
obj.read()
obj.show()
obj.read1()
obj.show1()
obj1.read()
obj1.show()
obj.read2()
obj.show2()
Output
Maruti
Four wheeler
```

```
HONDA
Two wheeler
Maruti
Four wheeler
Auto
three wheeler
```

Operator Overloading:- Python operators work for built-in classes. But same operator behaves differently with different types. For example, the + operator will, perform arithmetic addition on two numbers, merge two lists and concatenate two strings.

```
class ol:
    def __init__(self, a = 0, b = 0):
        self.a = a
        self.b = b
    def __str__(self):
        return "({0},{1})".format(self.a,self.b)

    def __add__(self,other):
        a = self.a + other.a
        b = self.b + other.b
        return ol(a,b)
obj=ol([1,3],[2,3])
obj1=ol([3,4],[5,6])
sobj=ol('yogesh ','Sonu ')
sobj1=ol('Mehra','Kumar')
iobj=ol(2,3)
iobj1=ol(2,3)

print(obj+obj1)
print(sobj+sobj1)
print(iobj+iobj1)
Output
([1, 3, 3, 4],[2, 3, 5, 6])
(yogesh Mehra,Sonu Kumar)
(4,6)
```

2.5 List

A **list** is a data structure in Python that is a mutable, or changeable, ordered sequence of elements. Each element or value that is inside of a list is called an item. Just as strings are defined as characters between quotes, lists are defined by having values between square brackets [ ].

Lists are great to use when you want to work with many related values. They enable you to keep data together that belongs together, condense your code, and perform the same methods and operations on multiple values at once.

When thinking about Python lists and other data structures that are types of collections, it is useful to consider all the different collections you have on your computer: your assortment of files, your song playlists, your browser bookmarks, your emails, the collection of videos you can access on a streaming service, and more.

To get started, let's create a list that contains items of the string data type:

```
Li = [“pen”, “Fan”, “Laptop”, “Ishan”]
print(sea_creatures)

OUTPUT:-

[“pen”, “Fan”, “Laptop”, “Ishan”]
```

As an ordered sequence of elements, each item in a list can be called individually, through indexing. Lists are a compound data type made up of smaller parts, and are very flexible because they can have values added, removed, and changed. When you need to store a lot of values or iterate over values, and you want to be able to readily modify those values, you'll likely want to work with list data types.

Indexing Lists

Each item in a list corresponds to an index number, which is an integer value, starting with the index number 0.

For the list Li, the index breakdown looks like this:

“pen”	“Fan”	“Laptop”	“Ishan”
0	1	2	3

Because each item in a Python list has a corresponding index number, we're able to access and manipulate lists in the same ways we can with other sequential data types.

```
sea_creatures = ['shark', 'cuttlefish', 'squid', 'mantis shrimp', 'anemone']

print(sea_creatures[1])

Output
```

```
cuttlefish



print(sea_creatures[18])

Output
IndexError: list index out of range
```

Adding new values to list

```
sea_creatures[0] = 'shark'
sea_creatures[1] = 'cuttlefish'
sea_creatures[2] = 'squid'
sea_creatures[3] = 'mantis shrimp'
sea_creatures[4] = 'anemone
```

In addition to positive index numbers, we can also access items from the list with a negative index number, by counting backwards from the end of the list, starting at -1. This is especially useful if we have a long list and we want to pinpoint an item towards the end of a list.

the negative index breakdown looks like this:

'shark’	'cuttlefish’	'squid’	'mantis shrimp’	'anemone’
-5	-4	-3	-2	-1

So, if we would like to print out the item 'squid' by using its negative index number, we can do so like this:

```
print(sea_creatures[-3])
Output
Squid
```

Modifying Items in Lists

We can use indexing to change items within the list, by setting an index number equal to a different value. This gives us greater control over lists as we are able to modify and update the items that they contain.

If we want to change the string value of the item at index 1 from 'cuttlefish' to 'octopus', we can do so like this:

```
sea_creatures[1] = 'octopus'
print(sea_creatures)
Output
['shark', 'octopus', 'squid', 'mantis shrimp', 'anemone']
sea_creatures[-3] = 'blobfish'
print(sea_creatures)
['shark', 'octopus', 'blobfish', 'mantis shrimp', 'anemone']
```

Slicing Lists

We can also call out a few items from the list. Let's say we would like to just print the middle items of sea_creatures, we can do so by creating a **slice**. With slices, we can call multiple values by creating a range of index numbers separated by a colon [x:y]:

```
print(sea_creatures[1:4])
Output
['octopus', 'blobfish', 'mantis shrimp']

print(sea_creatures[:3])
Output
['shark', 'octopus', 'blobfish']
```

List Append

List append will add the item at the end.

```
If you want to add at the beginning, you can use the insert function (see below)
list.insert(0, "Files")
list = ["Movies", "Music", "Pictures"]

list.append(x) will add an element to the end of the list
list.append("Files")

print list
```

```
['Movies', 'Music', 'Pictures', 'Files']
```

List Insert

```
The syntax is: list.insert(x, y)              #will add element y on the place before x
list = ["Movies", "Music", "Pictures"]

list.insert(2,"Documents")

print list
['Movies', 'Music', 'Documents', 'Pictures', 'Files']

You can insert a value anywhere in the list

list = ["Movies", "Music", "Pictures"]
list.insert(3, "Apps")
```

List Remove

```
To remove an element's first occurrence in a list, simply use list.remove

The syntax is: list.remove(x)
List = ['Movies', 'Music', 'Files', 'Documents', 'Pictures']

list.remove("Files")

print list
['Movies', 'Music', 'Documents', 'Pictures']

a = [1, 2, 3, 4]
a.remove(2)
print a
[1, 3, 4]
```

List Extend

```
The syntax is: list.extend(x) #will join the list with list x
list2 = ["Music2", "Movies2"]
list1.extend(list2)

print list1
['Movies', 'Music', 'Documents', 'Pictures', 'Music2', 'Movies2']
```

List Delete

```
Use del to remove item based on index position.
list = ["Matthew", "Mark", "Luke", "John"]
del list[1]

print list
>>>Matthew, Luke, John
```

List Keywords

```
The keyword "in" can be used to test if an item is in a list.
list = ["red", "orange", "green", "blue"]
if "red" in list:
    do_something()

#Keyword "not" can be combined with "in".

list = ["red", "orange", "green", "blue"]
if "purple" not in list:
    do_something()
```

List Reverse

```
The reverse method reverses the order of the entire list.
L1 = ["One", "two", "three", "four", "five"]

#To print the list as it is, simply do:
print L1

#To print a reverse list, do:
for i in L1[::-1]:
    print i
```

```
#OR

L = [0, 10, 20, 40]
L.reverse()

print L
[40, 20, 10, 0]
```

List Sorting

```
The easiest way to sort a List is with the sorted(list) function.

That takes a list and returns anew list with those elements in sorted order.

The original list is not changed.

The sorted() function can be customized though optional arguments.

The sorted() optional argument reverse=True, e.g. sorted(list, reverse=True),
makes it sort backwards.
#create a list with some numbers in it
numbers = [5, 1, 4, 3, 2, 6, 7, 9]

#prints the numbers sorted
print sorted(numbers)

#the original list of numbers are not changed
print numbers
my_string = ['aa', 'BB', 'zz', 'CC', 'dd', "EE"]

#if no argument is used, it will use the default (case sensitive)
print sorted(my_string)

#using the reverse argument, will print the list reversed
print sorted(strs, reverse=True)   ## ['zz', 'aa', 'CC', 'BB']

This will not return a value, it will modify the list
list.sort()
```

List Split

```
Split each element in a list.
mylist = ['one', 'two', 'three', 'four', 'five']
newlist = mylist.split(',')
```

```
print newlist
['one', ' two', ' three', ' four', 'five']
```

List Indexing

Each item in the list has an assigned index value starting from 0.

Accessing elements in a list is called indexing.

```
list        = ["first", "second", "third"]
list[0] == "first"
list[1] == "second"
list[2] == "third"
```

List Slicing

Accessing parts of segments is called slicing.

Lists can be accessed just like strings by using the [ ] operators.

The key point to remember is that the :end value represents the first value that is not in the selected slice.

So, the difference between end and start is the number of elements selected (if step is 1, the default).

Let's create a list with some values in it

```
colors = ['yellow', 'red', 'blue', 'green', 'black']

print colors[0]
>>> yellow

print colors [1:]
>>> red, blue, green, black


a[start:end]                    # items start through end-1
a[start:]                       # items start through the rest of the array
```

```
a[:end]                          # items from the beginning through end-1
a[:]                             # a copy of the whole array
There is also the step value, which can be used with any of the above
a[start:end:step]                # start through not past end, by step
The other feature is that start or end may be a negative number, which means it counts
from the end of the array instead of the beginning.
a[-1]                            # last item in the array
a[-2:]                           # last two items in the array
a[:-2]                           # everything except the last two items
```

List Loops

```
When using loops in programming, you sometimes need to store the results of the
loops.

One way to do that in Python is by using lists.

This short section will show how you can loop through a Python list and process
the list items.
#It can look something like this:
matching = []
for term in mylist:
    do something

#For example, you can add an if statement in the loop, and add the item to the (empty) list
if it's matching.
matching = []    #creates an empty list using empty square brackets []
for term in mylist:
    if test(term):
        matching.append(term)

#If you already have items in a list, you can easily loop through them like this:
items = [ 1, 2, 3, 4, 5 ]
for i in items:
    print i
```

List Methods

Calls to list methods have the list they operate on appear before the method name.

Any other values the method needs to do its job is provided in the normal way as an extra argument inside the round brackets.

```
s = ['h','e','l','l','o']   #create a list
s.append('d')                #append to end of list
len(s)                       #number of items in list
s.sort()                     #sorting the list
s.reverse()                  #reversing the list
s.extend(['w','o'])                  #grow list
s.insert(1,2)                #insert into list
s.remove('d')                #remove first item in list with value e
s.pop()                      #remove last item in the list
s.pop(1)                     #remove indexed value from list
s.count('o')           #search list and return number of instances found
s = range(0,10)              #create a list over range
s = range(0,10,2)            #same as above, with start index and increment
```

2.6 Tuple

A tuple is a sequence of immutable Python objects. Tuples are sequences, just like lists. The differences between tuples and lists are, the tuples cannot be changed unlike lists and tuples use parentheses, whereas lists use square brackets.

Creating a tuple is as simple as putting different comma-separated values. Optionally you can put these comma-separated values between parentheses also. For example −

```
tup1 = ('physics', 'chemistry', 1997, 2000);
tup2 = (1, 2, 3, 4, 5 );
tup3 = "a", "b", "c", "d";
```

Example

```
tup1 = ('physics', 'chemistry', 1997, 2000);
tup2 = (1, 2, 3, 4, 5, 6, 7 );
```

```
print "tup1[0]: ", tup1[0];
print "tup2[1:5]: ", tup2[1:5];

OUTPUT
tup1[0]:  physics
tup2[1:5]:  [2, 3, 4, 5]


x = ()                                  # empty tuple
x = (0,)                            # one item tuple
x = (0, 1, 2, "abc")          # four item tuple: indexed x[0]..x[3]
x = 0, 1, 2, "abc"            # parenthesis are optional
x = (0, 1, 2, 3, (1, 2))       # nested subtuples
y = x[0]                            # indexed item
y = x[4][0]                         # indexed subtuple
x = (0, 1) * 2                # repeat
x = (0, 1, 2) + (3, 4)              # concatenation
for item in x: print item       # iterate through tuple
b = 3 in x                                  # test tuple membership
```

There are only 2 tuple methods that tuple objects can call: **count** and **index.**

In simple terms, **count()** method searches the given element in a tuple and returns how many times the element has occurred in it.

The syntax of count() method is:

```
tuple.count(element)
```

In simple terms, **index()** method searches for the given element in a tuple and returns its position.

However, if the same element is present more than once, the first/smallest position is returned.

Note: Remember index in Python starts from 0 and not 1.

The syntax of index() method for Tuple is:

```
tuple.index(element)
```

CHAPTER-03

Dictionaries

3.1 Dictionary

Dictionary in Python is an unordered collection of data values, used to store data values like a map, which unlike other Data Types that hold only single value as an element, Dictionary holds **key:value** pair. Key value is provided in the dictionary to make it more optimized. Each key-value pair in a Dictionary is separated by a colon **:**, whereas each key is separated by a 'comma'.

A Dictionary in Python works similar to the Dictionary in a real world. Keys of a Dictionary must be unique and of *immutable* data type such as Strings, Integers and tuples, but the key-values can be repeated and be of any type.

3.2 Creating a Dictionary

Dictionary can be created by placing sequence of elements within curly **{}** braces, separated by 'comma'. Dictionary holds a pair of values, one being the Key and the other corresponding pair element being its **Key:value**.

Dictionary can also be created by the built-in function dict(). An empty dictionary can be created by just placing to curly braces{}.

Here is a small example using a dictionary:

```
tel = {'jack': 4098, 'sape': 4139}
tel['guido'] = 4127
tel

OUTPUT
{'jack': 4098, 'sape': 4139, 'guido': 4127}

tel['jack']

OUTPUT
4098
```

```
del tel['sape']
tel['irv'] = 4127
tel

OUTPUT
{'jack': 4098, 'guido': 4127, 'irv': 4127}


list(tel)

OUTPUT
['jack', 'guido', 'irv']

sorted(tel)

OUTPUT
['guido', 'irv', 'jack']

'guido' in tel

OUTPUT
True

'jack' not in tel

OUTPUT
False
```

3.3 Sets

Python Dictionary Methods	
Method	**Description**
clear()	Remove all items form the dictionary.
copy()	Return a shallow copy of the dictionary.
fromkeys(*seq*[, *v*])	Return a new dictionary with keys from *seq* and value equal to *v* (defaults to None).
get(*key*[,*d*])	Return the value of *key*. If *key* doesnot exit, return *d* (defaults to None).
items()	Return a new view of the dictionary's items (key, value).
keys()	Return a new view of the dictionary's keys.
pop(*key*[,*d*])	Remove the item with *key* and return its value or *d* if *key* is not found. If *d* is not provided and *key* is not found, raises KeyError.
popitem()	Remove and return an arbitary item (key, value). Raises KeyError if the dictionary is empty.
setdefault(*key*[,*d*])	If *key* is in the dictionary, return its value. If not, insert *key* with a value of *d* and return *d* (defaults to None).
update([*other*])	Update the dictionary with the key/value pairs from *other*, overwriting existing keys.
values()	Return a new view of the dictionary's values

A Set is an unordered collection data type that is iterable, mutable, and has no duplicate elements. Python's set class represents the mathematical notion of a set. The major advantage of using a set, as opposed to a list, is that it has a highly optimized method for checking whether a specific element is contained in the set. This is based on a data structure known as a hash table.

Methods for Sets

1. add(x) Method: Adds the item x to set if it is not already present in the set.

```
people = {"Jay", "Idrish", "Archil"}
people.add("Daxit")

This will add Daxit in people set.
```

2. union(s) Method: Returns a union of two set.Using the '|' operator between 2 sets is the same as writing set1.union(set2)

```
people = {"Jay", "Idrish", "Archil"}
vampires = {"Karan", "Arjun"}
population = people.union(vampires)

OR

population = people|vampires

Set population set will have components of both people and vampire
```

3. **intersect(s) Method:** Returns an intersection of two sets.The '&' operator comes can also be used in this case.

```
victims = people.intersection(vampires)

Set victims will contain the common element of people and vampire
```

4. difference(s) Method: Returns a set containing all the elements of invoking set but not of the second set. We can use '-' operator here.

```
safe = people.difference(vampires)

OR

safe = people – vampires

Set safe  will have all the elements that are in people but not vampire
```

5. **clear()** Method: Empties the whole set.

```
victims.clear()
```

Clears victim set

However there are two major pitfalls in Python sets:

1. The set doesn't maintain elements in any particular order.

2. Only instances of immutable types can be added to a Python set.

Operators for Sets

Sets and frozen sets support the following operators:

```
key in s       # containment check
key not in s   # non-containment check
s1 == s2       # s1 is equivalent to s2
s1 != s2       # s1 is not equivalent to s2
s1 <= s2    # s1is subset of s2 s1 < s2     # s1 is proper subset of s2 s1 >= s2    # s1is superset of
s2
s1 > s2     # s1 is proper superset of s2
s1 | s2        # the union of s1 and s2
s1 & s2 # the intersection of s1 and s2
s1 – s2        # the set of elements in s1 but not s2
s1 ^ s2        # the set of elements in precisely one of s1 or s2
```

3.4 File Handling and Exception

Like C, Python also supports file handling and allows users to handle files by reading and writing into files, along with many other file handling options, to operate on files. Since python use very easy and short way to implement codes, file handling in python also use the same way to implement the concept. We have two different types of files text files and binary files.

Use of file handling

Generally, we take input from console and write it back to console in order to interact with the user. But this is possible in case if we have limited amount of the data.

In case we want to display very large amount of data then it is not possible to store that data on console and since the memory is volatile, it become impossible to generate the data again and again.

In such cases we store files in local system which is volatile and can be accessed anytime with the use of file handling.

When we want to read from or write to a file we need to open it first. When we open a file after doing implementation on it(in case if there id any) it needs to be closed to release the resources hold by file.

Therefore, a file operation takes place in the following order.

1. Open a file
2. Read or write (perform operation)
3. Close the file

Mode	Description
R	It opens the file to read-only. The file pointer exists at the beginning. The file is by default open in this mode if no access mode is passed.
Rb	It opens the file to read only in binary format. The file pointer exists at the beginning of the file.
r+	It opens the file to read and write both. The file pointer exists at the beginning of the file.
rb+	It opens the file to read and write both in binary format. The file pointer exists at the beginning of the file.
W	It opens the file to write only. It overwrites the file if previously exists or creates a new one if no file exists with the same name. The file pointer exists at the beginning of the file.
Wb	It opens the file to write only in binary format. It overwrites the file if it exists previously or creates a new one if no file exists with the same name. The file pointer exists at the beginning of the file.

w+	It opens the file to write and read both. It is different from r+ in the sense that it overwrites the previous file if one exists whereas r+ doesn't overwrite the previously written file. It creates a new file if no file exists. The file pointer exists at the beginning of the file.
wb+	It opens the file to write and read both in binary format. The file pointer exists at the beginning of the file.
A	It opens the file in the append mode. The file pointer exists at the end of the previously written file if exists any. It creates a new file if no file exists with the same name.
Ab	It opens the file in the append mode in binary format. The pointer exists at the end of the previously written file. It creates a new file in binary format if no file exists with the same name.
a+	It opens a file to append and read both. The file pointer remains at the end of the file if a file exists. It creates a new file if no file exists with the same name.
ab+	It opens a file to append and read both in binary format. The file pointer remains at the end of the file.

Opening a file or creating a file

To create a file we use built-in function i.e. open function. Python use the open() function which accepts two arguments, file name and access mode in which the file is accessed. The function returns a file object which can be used to perform many operations like reading, writing, etc.

Syntax:

1. fileobject = open(<file-name>, <access-mode>, <buffering>)

The files can be accessed using different modes like read, write, or append. The following are the details about the access mode to open a file.

Let’s take an example to open a file named "file.txt" in read mode and printing its content on the console.
Example

```
#opens the file file.txt in read mode
a = open("file.txt","r")

if a:
   print("file is opened successfully")
Output:
<class '_io.TextIOWrapper'>
file is opened successfully
```

The close() method

Once we are done with all the operations on the file, we need to close it using the close() method.

We can perform any operation on the file externally in the file system is the file is opened in python, hence it is good practice to close the file once all the operations are done. Closing a file will free up the resources.

Syntax

1. fileobject.close()

```
Take an example.
# opens the file file.txt in read mode
a = open("file.txt","r")

if a:
   print("file is opened successfully")

#closes the opened file
a.close()
```

3.5 Reading/Writing text and number to/from a file

Reading the file

Python provides us the read() method to read a file. This method reads a string or data from the file in the both the format text as well as binary.

Syntax

1. fileobj.read(<count>)

Here, the count define the number of bytes to be read from the file starting from the beginning of the file. If the count is not specified in the synatx, then it may read the content of the file until the end.

Example

```
#open the file.txt in read mode. Raise an error if no such file exists.
a = open("file.txt","r");

#stores all the data of the file into the variable content
content = a.read(9);

# prints the type of the data stored in the file
print(type(content))

#prints the content of the file
print(content)

#closes the opened file
a.close()
Output:
<class 'str'>
Hi, I am
```

Read Lines of the file

Python enable us to read the file line by line with the help of a function readline(). The readline() method reads the lines of the file from the beginning, i.e., if we use the readline() method twqo or three times, then we can get the first two lines of the file or first three lines of the file respectively.

Example

#open the file.txt in read mode. causes error if no such file exists.

```
a = open("file.txt","r");

#stores all the data of the file into the variable content
content = a.readline();

# prints the type of the data stored in the file
print(type(content))

#prints the content of the file
print(content)

#closes the opened file
a.close()
Output:
```

```
<class 'str'>
Hi, I am the file and being used as
```

Looping through the file

This property is used to read the whole file by looping through the lines of the file. We can do this with the help of for loop.

Example

#open the file.txt in read mode. It causes an error if no such file exists.

```
  a = open("file.txt","r");
  #running a for loop
for i in a:
    print(i) # i contains each line of the file
Output:
Hi, I am the file and being used as
an example to read a
file in python.
```

Writing the file

If we want to write some text to a file, we need to open the file using the open method with one of the following access modes.

a: this mode is used to append the existing file. It creates new file if the file you are opening does not exist. The file pointer is at the end of the file.

w: It will overwrite the file if any file exists. The file pointer is at the beginning of the file in this mode.

Example 1

#open the file.txt in append mode. Creates a new file if no such file exists.

```
a = open("file.txt","a");
  #appending the content to the file
a.write("Python is the modern day language. It makes things so simple.")
  #closing the opened file
a.close();
```

We can see that the content of the file is modified.

File.txt:

Hi, I am the file **and** being used as
an example to read a
file **in** python.
Python **is** the modern day language. It makes things so simple.

Example 2

#open the file.txt in write mode.

```
a = open("file.txt","w");

#overwriting the content of the file
a.write("Python is the modern day language. It makes things so simple.")
  #closing the opened file
a.close();
```

We can check that all the previously written content of the file is overwritten with the new text we have passed in the syntax “a.write()”.

File.txt:

Python **is** the modern day language. It makes things so simple.

Creating a new file

If we want to create a new file, it can be created by using one of the following access modes with the function open().

x: it creates a new file with the specified name. It raises an error a file exists with the same name.

a: It creates a new file with the specified name if no such file exists. It appends the content to the file if the file already exists with the specified name.

w: It creates a new file with the specified name if no such file exists. It overwrites the existing file.

Example

#open the file.txt in read mode. causes error if no such file exists.

```
a= open("file2.txt","x");
```

```
print(a)

if a:
    print("File created successfully");
```
Output:
File created successfully

Python OS module

The OS module give us the functions that are involved in file processing operations like renaming, deleting, etc. To use this module we need to import it first and after that we can call rename or remove methods.

Let's look at some of the os module functions.

Renaming the file

The rename() method is used to rename the particular file to a new name. The syntax to use the rename() method is given here.

Example

```
import os;

#rename file1.txt to file2.txt
os.rename("file1.txt","file2.txt")
```

Removing the file

The remove() method is used to remove the specific file. The syntax to use the remove() method is given here.

Example

```
import os;

#deleting the file named file3.txt
```

```
os.remove("file3.txt")
```

Chapter 4

Python Regular Expression

4.1 RE objects

A regular expression (RE) in a programming language is a special text string used for describing a search pattern. It is extremely useful for extracting information from text such as code, files, log, spreadsheets, or even documents.

Regular expressions can contain both special and ordinary characters. Most ordinary characters such as 'A', 'a', or '0' are the simplest regular expressions. These characters simply match themselves.

Some characters such as '|' or '(' are special. Special characters either stand for classes of ordinary characters or affect how the regular expressions around them are interpreted.

Repetition qualifiers (*, +, ?, {m,n}, and so on) cannot be directly nested. This avoids ambiguity with the non-greedy modifier suffix '?', and with other modifiers in other implementations. To apply a second repetition to an inner repetition, parentheses may be used.

For instance, a regular expression could tell a program to search for specific text from the string and then to print out the result accordingly. Expression can include

- Text matching
- Repetition
- Branching
- Pattern-composition etc.

In Python, a regular expression is denoted as RE (REs, regexes or regex pattern) are imported through **re module**. Python supports regular expression through libraries. In Python regular expression supports various things like **Modifiers, Identifiers, and White space characters**

Identifiers	**Modifiers**	**White space characters**	**Escape required**
\d= any number (a digit)	\d represents a digit.Ex: \d{1,5} it will declare digit between 1,5 like 424,444,545 etc.	\n = new line	. + * ? [] $ ^ () {} | \
\D= anything but a number (a non-digit)	+ = matches 1 or more	\s= space	
\s = space (tab,space,newline etc.)	? = matches 0 or 1	\t =tab	

\S= anything but a space	* = 0 or more	\e = escape	
\w = letters (Match alphanumeric character, including "_")	$ match end of a string	\r = carriage return	
\W =anything but letters (Matches a non-alphanumeric character excluding "_")	^ match start of a string	\f= form feed	
. = anything but letters (periods)	| matches either or x/y	----------	
\b = any character except for new line	[] = range or "variance"	----------	
\.	{x} = this amount of preceding code	----------	

Regular Expression Syntax

RE

import re

- "re" module included with Python primarily used for string searching and manipulation
- Also used frequently for web page "Scraping" (extract large amount of data from websites)

We will begin the expression tutorial with this simple exercise by using the expressions (w+) and (^).

Example of w+ and ^ Expression

- **"^":** This expression matches the start of a string
- **"w+**": This expression matches the alphanumeric character in the string

Here we will see an example of how we can use w+ and ^ expression in our code. We cover re.findall function later in this tutorial but for a while we simply focus on \w+ and \^ expression.

```
import re
xx = "This is the Text123"
r1 = re.findall(r"^\w+",xx)
print(r1)

Output

['Text123']
```

Remember, if you remove +sign from the w+, the output will change, and it will only give the first character of the first letter, i.e., [T]

Example of \s expression in re.split function

- "s": This expression is used for creating a space in the string

To understand how this regular expression works in Python, we begin with a simple example of a split function. In the example, we have split each word using the "re.split" function and at the same time we have used expression \s that allows to parse each word in the string separately.

```
import re

print((re.split(r'\s','we are splitting the words')))
Output
['we', 'are', 'splitting', 'the', 'words']

print((re.split(r's','split the words')))
Output
[‘’ , ‘plit’,  ‘the’,  ‘word’, ‘’]
```

Similarly, there are series of other regular expressions in Python that you can use in various ways in Python like \d,\D,$,\.,\b, etc.

Using regular expression methods

The "re" package provides several methods to actually perform queries on an input string. The method we going to see are

- re.match()
- re.search()
- re.findall()

Note: Based on the regular expressions, Python offers two different primitive operations. The match method checks for a match only at the beginning of the string while search checks for a match anywhere in the string

Using re.match()

The match function is used to match the RE pattern to string with optional flags. In this method, the expression "w+" and "\W" will match the words starting with letter 'g' and thereafter, anything which is not started with 'g' is not identified. To check match for each element in the list or string, we run the forloop.

4.2 Finding Pattern in Text (re.search())

A regular expression is commonly used to search for a pattern in a text. This method takes a regular expression pattern and a string and searches for that pattern with the string.

In order to use search() function, you need to import re first and then execute the code. The search() function takes the "pattern" and "text" to scan from our main string and returns a match object when the pattern is found or else not match if the pattern is not found.

For example here we look for two literal strings "Software testing" "test123", in a text string "Software Testing is fun". For "software testing" we found the match hence it returns the output as "found a match", while for word "test123" we could not found in string hence it returns the output as "No match".

Using re.findall for text

Re.findall() module is used when you want to iterate over the lines of the file, it will return a list of all the matches in a single step. For example, here we have a list of e-mail addresses, and we want all the e-mail addresses to be fetched out from the list, we use the re.findall method. It will find all the e-mail addresses from the list.

```
import re

list = ["test tiger", "train telephone ", "Turn Python"]
for element in list:
   z = re.match("(t\w+)\W(t\w+)", element)
if z:
   print((z.groups()))

Output
('test', ' tiger')
('train' , 'telephone')



patterns = ['software testing', 'test123']
text = 'software testing is fun?'
for pattern in patterns:
   print('Looking for "%s" in "%s" ->' % (pattern, text), end=' ')
   if re.search(pattern, text):
      print('found a match!')
else:
   print('no match')

Output
```

```
Looking for 'software testing' in “software testing is fun?” -> found a match
Looking for 'test123' in “software testing is fun?” -> no match


// Email with Regular expression


abc = 'abc@gmail.com
            , xyz@gmail.com, pqr@yahoo.com,  123.com'

emails = re.findall(r'[\w\.-]+@[\w\.-]+', abc)
for email in emails:
   print(email)

Output
abc@gmail.com
xyz@gmail.com
 pqr@yahoo.com
```

4.3 Python Flags

Many Python Regex Methods and Regex functions take an optional argument called Flags. This flags can modify the meaning of the given Regex pattern. To understand these we will see one or two example of these Flags.

Various flags used in Python includes

Syntax for Regex Flags	What does this flag do
[re.M]	Make begin/end consider each line
[re.I]	It ignores case
[re.S]	Make [.]
[re.U]	Make { \w,\W,\b,\B} follows Unicode rules
[re.L]	Make {\w,\W,\b,\B} follow locale
[re.X]	Allow comment in Regex

Example of re.M or Multiline Flags

In multiline the pattern character [^] match the first character of the string and the beginning of each line (following immediately after the each newline). While expression small "w" is used to

mark the space with characters. When you run the code the first variable "k1" only prints out the character 'g' for word globe123, while when you add multiline flag, it fetches out first characters of all the elements in the string.

```
import re
xx = """globe123
python
anaconda"""
k1 = re.findall(r"^\w", xx)
k2 = re.findall(r"^\w", xx, re.MULTILINE)
print(k1)
print(k2)

Output

['g']

['g', 'p', 'a' ]
```

- We declared the variable xx for string " globe123…..python…..anaconda "
- Run the code without using flags multiline, it gives the output only 'g' from the lines
- Run the code with flag "multiline", when you print 'k2' it gives the output as 'g', 'p' and 'a'
- So, the difference we can see after and before adding multi-lines in above example.

4.4 Python exception handling

An exception can be defined as an unwanted condition in a program that result in the interruption in the flow of the program. Python supports many built-in exceptions which forces your program to output an error when something in it goes wrong.

Whenever an exception occurs, the program stop executing the further program.

Python provides us with the way to handle the Exception so that the other part of the code can be executed without any interruption.

We have some list of common exception that can occur in general programs

1. **ZeroDivisionError:** it Occurs when a number is divided by zero.
2. **NameError:** It occurs when a name is not found. It may be local or global.
3. **IndentationError:** it occurs when incorrect indentation is given.
4. **IOError:** It occurs in case when Input Output operation fails.
5. **EOFError:** It occurs when the end of the file is reached, and yet operations are being performed.

6. **ImportError:** it occurs when an import statement fails.
7. **OverflowError: it occurs** when a calculation exceeds maximum limit for a numeric type.

Problem without handling exceptions

As we know that the exception is an abnormal or unwanted condition that stop the execution of the program. If we don't handle exception in our program then it will display the output as shown in the given example

Example

```
a = int(input("Enter the value of a:"))
b = int(input("Enter the value of b:"))
c = a/b;
print("a/b = %d"%c)

#other part of the code:
print("Hi I am other part of the program")

Output:
Enter the value of a:10
Enter the value of b:0
Traceback (most recent call last):
  File "exception-test.py", line 3, in <module>
    c = a/b;
ZeroDivisionError: division by zero
```

Exception handling in python

Exception handling can be handled using try statement. If our program contains code that may throw an exception, we must place that code in the try block that must be followed with the except statement.

The except statement contains a block of code that will be executed if there is some exception in the try block.



Syntax

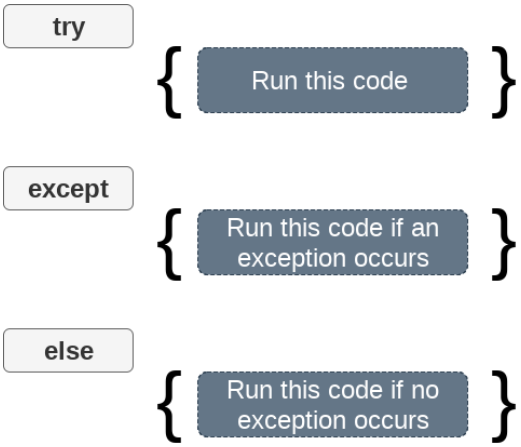
```
try:
   #block of code

except Exception1:
   #block of code
except Exception2:
   #block of code
  #other code
```

We can also use the else statement with the try-except statement. The else part will be executed when no exception occurs in the try block.

The syntax to use the else statement with the try-except statement is given below.

```
try:
   #block of code
except Exception1:
   #block of code
else:
   #this code executes if no except block is executed
```



Example

```
try:
    a = int(input("Enter the value of a:"))
    b = int(input("Enter the value of b:"))
    c = a/b;
    print("a/b = %d"%c)
except Exception:
    print("divide by zero exception raised")
else:
    print("Hi I am executing the else part since there is no exception")
```

Output:

```
Enter the value of a:10
Enter the value of b:2
a/b = 5
Hi I am executing the else block since there is no exception
```

Declaring multiple exceptions

We can Declare multiple exceptions in the cases where a try block throws multiple exceptions. Python provide this facility too.

Syntax

```
try:
    #block of code
except (<Exception 1>,<Exception 2>,<Exception 3>,...<Exception n>)
    #block of code
else:
    #block of code
Example
try:
```

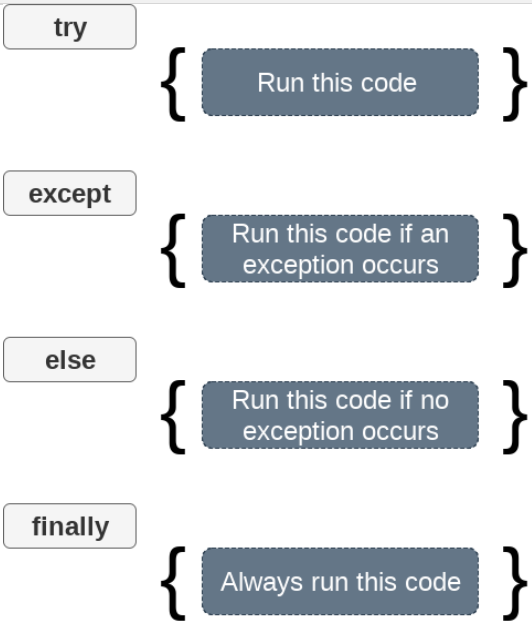
```
    a=10/0;
except ArithmeticError,StandardError:
    print "Arithmetic Exception"
else:
    print "Successfully Done"
```

Output:

Arithmetic Exception
The finally block
the finally block is used with the try block in which, we can place the important code which must be executed before the try statement throws an exception.

syntax

```
try:
    # block of code
    # this may throw an exception
finally:
    # block of code
    # this will always be executed
```



Example

```
try:
    fileptr = open("file.txt","r")
    try:
        fileptr.write("Hi I am good")
    finally:
```

```
        fileptr.close()
        print("file closed")
except:
    print("Error")
```

Output:

```
file closed
Error
```

Chapter 5

Machine Learning with Python

5.1 Python Libraries

NumPy

NumPy (or Numpy) is a Linear Algebra Library for Python, the reason it is so important for Data Science with Python is that almost all of the libraries in the PyData Ecosystem rely on NumPy as one of their main building blocks.

Numpy is also incredibly fast, as it has bindings to C libraries. For more info on why you would want to use Arrays instead of lists.

Installation Instructions

It is highly recommended you install Python using the Anaconda distribution to make sure all underlying dependencies (such as Linear Algebra libraries) all sync up with the use of a conda install. If you have Anaconda, install NumPy by going to your terminal or command prompt and typing:

For anaconda

```
conda install numpy
```

For Python

```
pip installl numpy
```

Numpy Arrays

NumPy arrays are the main way we will use Numpy throughout the course. Numpy arrays essentially come in two flavors: vectors and matrices. Vectors are strictly 1-d arrays and matrices are 2-d (but you should note a matrix can still have only one row or one column).

Let's begin our introduction by exploring how to create NumPy arrays

```
numpy.array(object, dtype = None, copy = True, order = None, subok = False, ndmin = 0)
```

The above constructor takes the following parameters −

Sr.No.	Parameter & Description
1	**object** Any object exposing the array interface method returns an array, or any (nested) sequence.
2	**dtype** Desired data type of array, optional
3	**copy** Optional. By default (true), the object is copied
4	**order** C (row major) or F (column major) or A (any) (default)
5	**Subok** By default, returned array forced to be a base class array. If true, sub-classes passed through
6	**Ndmin** Specifies minimum dimensions of resultant array

Take a look at the following examples to understand better.

Creating NumPy Arrays

From a Python List

We can create an array by directly converting a list or list of lists:

```
In [3]: import numpy as np
        my_list = [1,2,6]
        my_list

Out[3]: [1, 2, 6]
```

```
In [4]: b=np.array(my_list)
        b

Out[4]: array([1, 2, 6])
```

```
In [10]: my_matrix = [[2,1,9],[4,5,6],[7,8,9]]
         my_matrix
         type(my_matrix)

Out[10]: list
```

```
In [11]: np.array(my_matrix)

Out[11]: array([[2, 1, 9],
                [4, 5, 6],
                [7, 8, 9]])
```

Built-in Methods

There are lots of built-in ways to generate Arrays
Arrange
Return evenly spaced values within a given interval.

```
In [12]: np.arange(0,10,1)
         #range(0,10,2)

Out[12]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [34]: np.arange(0,11,2)

Out[34]: array([ 0,  2,  4,  6,  8, 10])
```

zeros and ones
Generate arrays of zeros or ones

```
In [21]: np.zeros((3,3))

Out[21]: array([[0., 0., 0.],
                [0., 0., 0.],
                [0., 0., 0.]])
```

```
In [36]: print(np.zeros((5,5)))

         [[0. 0. 0. 0. 0.]
          [0. 0. 0. 0. 0.]
          [0. 0. 0. 0. 0.]
          [0. 0. 0. 0. 0.]
          [0. 0. 0. 0. 0.]]
```

```
In [22]: np.ones(8)

Out[22]: array([1., 1., 1., 1., 1., 1., 1., 1.])
```

```
In [23]: np.ones((3,5))

Out[23]: array([[1., 1., 1., 1., 1.],
                [1., 1., 1., 1., 1.],
                [1., 1., 1., 1., 1.]])
```

Linspace

Return evenly spaced numbers over a specified interval.

```
In [27]: print(np.linspace(0,10,5))

         [ 0.   2.5  5.   7.5 10. ]
```

```
In [13]: np.linspace(0,10,50)

Out[13]: array([ 0.        ,  0.20408163,  0.40816327,  0.6122449 ,  0.81632653,
                 1.02040816,  1.2244898 ,  1.42857143,  1.63265306,  1.83673469,
                 2.04081633,  2.24489796,  2.44897959,  2.65306122,  2.85714286,
                 3.06122449,  3.26530612,  3.46938776,  3.67346939,  3.87755102,
                 4.08163265,  4.28571429,  4.48979592,  4.69387755,  4.89795918,
                 5.10204082,  5.30612245,  5.51020408,  5.71428571,  5.91836735,
                 6.12244898,  6.32653061,  6.53061224,  6.73469388,  6.93877551,
                 7.14285714,  7.34693878,  7.55102041,  7.75510204,  7.95918367,
                 8.16326531,  8.36734694,  8.57142857,  8.7755102 ,  8.97959184,
                 9.18367347,  9.3877551 ,  9.59183673,  9.79591837, 10.        ])
```

eye

Creates an identity matrix

```
In [34]: np.eye(4,5)

Out[34]: array([[1., 0., 0., 0., 0.],
                [0., 1., 0., 0., 0.],
                [0., 0., 1., 0., 0.],
                [0., 0., 0., 1., 0.]])
```

Random

Numpy also has lots of ways to create random number arrays:

rand

Create an array of the given shape and populate it with random samples from a uniform distribution over [0, 1)

```
In [15]: np.random.rand(4)

Out[15]: array([0.00494967, 0.62407491, 0.70515039, 0.91413351])

In [43]: np.random.rand(3,2)

Out[43]: array([[0.23953079, 0.22487074],
                [0.83692957, 0.74193123],
                [0.01136801, 0.76258278]])
```

randn

Return a sample (or samples) from the "standard normal" distribution. Unlike rand which is uniform:

```
In [16]: np.random.randn(5,8)

Out[16]: array([[ 0.90201802,  1.40083287, -1.4504507 ,  0.83759186,  1.87522434,
                 -0.88436015, -0.08371351,  1.29288105],
                [ 0.81634194,  0.17754021, -1.84038515,  0.80431344,  1.10808378,
                  1.13687101,  2.30099244, -0.08163165],
                [ 1.42667817, -0.12090487,  0.41394622,  0.86197397,  1.990281  ,
                 -0.33149476, -0.34210803,  0.55055678],
                [ 0.46934582, -1.97709978,  1.059288  , -1.03026784, -0.21497361,
                  0.19982773, -1.45805485, -0.07593413],
                [-1.6624872 , -1.62311585, -2.20209717,  1.40381115, -0.43872566,
                 -0.71147293,  0.14419507,  2.02676854]])

In [45]: np.random.randn(2,2)

Out[45]: array([[ 0.38404965,  1.44310501],
                [ 1.63635869, -0.01463879]])
```

randint

Return random integers from low (inclusive) to high (exclusive).

```
In [56]: np.random.randint(5,10)

Out[56]: 9

In [17]: np.random.randint(1,10,20)

Out[17]: array([4, 9, 8, 9, 9, 1, 6, 9, 2, 9, 8, 9, 1, 5, 4, 4, 3, 5, 5, 9]
```

Array Attributes and Methods

Let's discuss some useful attributes and methods or an array:

```
In [19]: arr = np.arange(25)
         ranarr = np.random.randint(0,50,10)

In [20]: arr

Out[20]: array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
                17, 18, 19, 20, 21, 22, 23, 24])

In [21]: ranarr

Out[21]: array([ 5, 41,  9, 44, 23, 18, 36, 18, 24, 33])
```

Reshape

Returns an array containing the same data with a new shape.

```
In [24]: arr.reshape(5,5)

Out[24]: array([[ 0,  1,  2,  3,  4],
                [ 5,  6,  7,  8,  9],
                [10, 11, 12, 13, 14],
                [15, 16, 17, 18, 19],
                [20, 21, 22, 23, 24]])
```

max,min,argmax,argmin

These are useful methods for finding max or min values. Or to find their index locations using argmin or argmax

```
In [74]: arr

Out[74]: array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
                17, 18, 19, 20, 21, 22, 23, 24])

In [25]: ranarr.max()

Out[25]: 44

In [26]: ranarr.argmax()

Out[26]: 3

In [91]: ranarr.min()

Out[91]: 7

In [92]: ranarr.argmin()

Out[92]: 5
```

NumPy Indexing and Selection

In this lecture we will discuss how to select elements or groups of elements from an array.

```
In [3]: import numpy as np

In [4]: #Creating sample array
        arr = np.arange(0,25)

In [10]: #Show
         a=arr.reshape(5,5)
         a

Out[10]: array([[ 0,  1,  2,  3,  4],
                [ 5,  6,  7,  8,  9],
                [10, 11, 12, 13, 14],
                [15, 16, 17, 18, 19],
                [20, 21, 22, 23, 24]])

In [29]: a[1:4:2,2:5:]

Out[29]: array([[ 7,  8,  9],
                [17, 18, 19]])
```

Bracket Indexing and Selection

The simplest way to pick one or some elements of an array looks very similar to python lists:

```
In [4]: #Get a value at an index
        arr[8]

Out[4]: 8

In [5]: #Get values in a range
        arr[1:5]

Out[5]: array([1, 2, 3, 4])

In [6]: #Get values in a range
        arr[0:5]

Out[6]: array([0, 1, 2, 3, 4])
```

Broadcasting

Numpy arrays differ from a normal Python list because of their ability to broadcast

```
In [7]: #Setting a value with index range (Broadcasting)
        arr[0:5]=100

        #Show
        arr

Out[7]: array([100, 100, 100, 100, 100,   5,   6,   7,   8,   9,  10])

In [8]: # Reset array, we'll see why I had to reset in  a moment
        arr = np.arange(0,11)

        #Show
        arr

Out[8]: array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10])

In [9]: #Important notes on Slices
        slice_of_arr = arr[0:6]

        #Show slice
        slice_of_arr

Out[9]: array([0, 1, 2, 3, 4, 5])
```

```
In [10]: #Change Slice
         slice_of_arr[:]=99

         #Show Slice again
         slice_of_arr

Out[10]: array([99, 99, 99, 99, 99, 99])
```

Now note the changes also occur in our original array!

```
In [11]: arr

Out[11]: array([99, 99, 99, 99, 99, 99,  6,  7,  8,  9, 10])
```

Data is not copied, it's a view of the original array! This avoids memory problems!

```
In [12]: #To get a copy, need to be explicit
         arr_copy = arr.copy()

         arr_copy

Out[12]: array([99, 99, 99, 99, 99, 99,  6,  7,  8,  9, 10])
```

Indexing a 2D array (matrices)

The general format is **arr_2d[row][col]** or **arr_2d[row,col]**. Recommend usually using the comma notation for clarity.

In [13]:
```
arr_2d = np.array(([5,10,15],[20,25,30],[35,40,45]))

#Show
arr_2d
```

Out[13]:
```
array([[ 5, 10, 15],
       [20, 25, 30],
       [35, 40, 45]])
```

In [14]:
```
#Indexing row
arr_2d[1]
```

Out[14]:
```
array([20, 25, 30])
```

In [15]:
```
# Format is arr_2d[row][col] or arr_2d[row,col]

# Getting individual element value
arr_2d[1:,1:]
```

Out[15]:
```
array([[25, 30],
       [40, 45]])
```

In [16]:
```
# Getting individual element value
arr_2d[1,0]
```

Out[16]:
```
20
```

In [17]:
```
# 2D array slicing

#Shape (2,2) from top right corner
arr_2d[:2,1:]
```

Out[17]:
```
array([[10, 15],
       [25, 30]])
```

In [18]:
```
#Shape bottom row
arr_2d[2]
```

Out[18]:
```
array([35, 40, 45])
```

In [19]:
```
#Shape bottom row
arr_2d[2,:]
```

Out[19]:
```
array([35, 40, 45])
```

Fancy Indexing

Fancy indexing allows you to select entire rows or columns out of order,to show this, let's quickly build out a numpy array

```
In [20]: #Set up matrix
         arr2d = np.zeros((10,10))

In [21]: #Length of array
         arr_length = arr2d.shape[1]

In [22]: #Set up array

         for i in range(arr_length):
             arr2d[i] = i

         arr2d

Out[22]: array([[0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
                [1., 1., 1., 1., 1., 1., 1., 1., 1., 1.],
                [2., 2., 2., 2., 2., 2., 2., 2., 2., 2.],
                [3., 3., 3., 3., 3., 3., 3., 3., 3., 3.],
                [4., 4., 4., 4., 4., 4., 4., 4., 4., 4.],
                [5., 5., 5., 5., 5., 5., 5., 5., 5., 5.],
                [6., 6., 6., 6., 6., 6., 6., 6., 6., 6.],
                [7., 7., 7., 7., 7., 7., 7., 7., 7., 7.],
                [8., 8., 8., 8., 8., 8., 8., 8., 8., 8.],
                [9., 9., 9., 9., 9., 9., 9., 9., 9., 9.]])
```

Fancy indexing allows the following

```
In [23]: arr2d[[2,4,6,8]]

Out[23]: array([[2., 2., 2., 2., 2., 2., 2., 2., 2., 2.],
                [4., 4., 4., 4., 4., 4., 4., 4., 4., 4.],
                [6., 6., 6., 6., 6., 6., 6., 6., 6., 6.],
                [8., 8., 8., 8., 8., 8., 8., 8., 8., 8.]])

In [24]: #Allows in any order
         arr2d[[6,4,2,7]]

Out[24]: array([[6., 6., 6., 6., 6., 6., 6., 6., 6., 6.],
                [4., 4., 4., 4., 4., 4., 4., 4., 4., 4.],
                [2., 2., 2., 2., 2., 2., 2., 2., 2., 2.],
                [7., 7., 7., 7., 7., 7., 7., 7., 7., 7.]])
```

Selection

Let's briefly go over how to use brackets for selection based off of comparison operators.

```
In [25]: arr = np.arange(1,11)
         arr

Out[25]: array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10])

In [26]: arr > 4

Out[26]: array([False, False, False, False,  True,  True,  True,  True,  True,
                True])

In [27]: bool_arr = arr>4

In [28]: bool_arr

Out[28]: array([False, False, False, False,  True,  True,  True,  True,  True,
                True])

In [29]: arr[bool_arr]

Out[29]: array([ 5,  6,  7,  8,  9, 10])

In [30]: arr[arr>2]

Out[30]: array([ 3,  4,  5,  6,  7,  8,  9, 10])

In [31]: x = 2
         arr[arr>x]

Out[31]: array([ 3,  4,  5,  6,  7,  8,  9, 10])
```

NumPy Operations

Arithmetic

You can easily perform array with array arithmetic, or scalar with array arithmetic. Let's see some example

```
In [1]: import numpy as np
        arr = np.arange(0,10)

In [2]: arr + arr
Out[2]: array([ 0,  2,  4,  6,  8, 10, 12, 14, 16, 18])

In [3]: arr * arr
Out[3]: array([ 0,  1,  4,  9, 16, 25, 36, 49, 64, 81])

In [4]: arr - arr
Out[4]: array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

Universal Array Functions

Numpy comes with many universal array functions, which are essentially just mathematical operations you can use to perform the operation across the array.

```
In [12]: #Taking Square Roots
         np.sqrt(arr)
Out[12]: array([ 0.        ,  1.        ,  1.41421356,  1.73205081,  2.        ,
                 2.23606798,  2.44948974,  2.64575131,  2.82842712,  3.        ])

In [13]: #Calcualting exponential (e^)
         np.exp(arr)
Out[13]: array([  1.00000000e+00,   2.71828183e+00,   7.38905610e+00,
                  2.00855369e+01,   5.45981500e+01,   1.48413159e+02,
                  4.03428793e+02,   1.09663316e+03,   2.98095799e+03,
                  8.10308393e+03])

In [14]: np.max(arr) #same as arr.max()
Out[14]: 9

In [15]: np.sin(arr)
Out[15]: array([ 0.        ,  0.84147098,  0.90929743,  0.14112001, -0.7568025 ,
                -0.95892427, -0.2794155 ,  0.6569866 ,  0.98935825,  0.41211849])
```

5.2 Pandas

Pandas is an open source, BSD-licensed library providing high-performance, easy-to-use data structures and data analysis tools for the Python programming language.

Using Pandas–the Python data analysislibrary

'Pandas' are used for data manipulation, analysis, and cleaning. Python 'pandas' are well suited for different kinds of data, such as:

- Tabular data with heterogeneously typed columns

- Ordered and unordered time series data
- Arbitrary matrix data with row and column labels
- Unlabeled data
- Any other form of observational or statistical data sets

How to installPandas

To install Python Pandas:

1. First, go to your command line or terminal.
2. Next, type *pip install pandas*.

   (If you have anaconda installed in your system, just type in *conda install pandas*.)

3. Once the installation is completed, go to your IDE (Jupyter, PyCharm, and so on) and simply import it by typing *import pandas as pd*.

Pandas deals with the following three data structures −

- Series
- DataFrame
- Panel

These data structures are built on top of Numpy array, which means they are fast.

Dimension & Description

The best way to think of these data structures is that the higher dimensional data structure is a container of its lower dimensional data structure. For example, DataFrame is a container of Series, Panel is a container of DataFrame.

Data Structure	Dimensions	Description
Series	1	1D labeled homogeneous array, size immutable.
Data Frames	2	General 2D labeled, size-mutable tabular structure with potentially heterogeneously typed columns.
Panel	3	General 3D labeled, size-mutable array.

Building and handling two or more dimensional arrays is a tedious task, burden is placed on the user to consider the orientation of the data set when writing functions. But using Pandas data structures, the mental effort of the user is reduced.

For example, with tabular data (DataFrame) it is more semantically helpful to think of the **index** (the rows) and the **columns** rather than axis 0 and axis 1.

Mutability

All Pandas data structures are value mutable (can be changed) and except Series all are size mutable. Series is size immutable.

Note − DataFrame is widely used and one of the most important data structures. Panel is used much less.

Series

Series is a one-dimensional array like structure with homogeneous data. For example, the following series is a collection of integers 10, 23, 56, …

10	23	56	17	52	61	73	90	26	72

Key Points

- Homogeneous data
- Size Immutable
- Values of Data Mutable

DataFrame

DataFrame is a two-dimensional array with heterogeneous data. For example,

Name	Age	Gender	Rating
Steve	32	Male	3.45
Lia	28	Female	4.6
Vin	45	Male	3.9
Katie	38	Female	2.78

The table represents the data of a sales team of an organization with their overall performance rating. The data is represented in rows and columns. Each column represents an attribute and each row represents a person.

Data Type of Columns

The data types of the four columns are as follows −

Column	Type
Name	String
Age	Integer

Gender	String
Rating	Float

Key Points

- Heterogeneous data
- Size Mutable
- Data Mutable

Panel

Panel is a three-dimensional data structure with heterogeneous data. It is hard to represent the panel in graphical representation. But a panel can be illustrated as a container of DataFrame.

Key Points

- Heterogeneous data
- Size Mutable
- Data Mutable

Series is a one-dimensional labeled array capable of holding data of any type (integer, string, float, python objects, etc.). The axis labels are collectively called index.

pandas.Series

A pandas Series can be created using the following constructor −

pandas.Series(data, index, dtype, copy)
The parameters of the constructor are as follows −

Sr.No	Parameter & Description
1	**data** data takes various forms like ndarray, list, constants
2	**index** Index values must be unique and hashable, same length as data. Default **np.arrange(n)** if no index is passed.
3	**dtype** dtype is for data type. If None, data type will be inferred
4	**copy** Copy data. Default False

A series can be created using various inputs like −

- Array
- Dict
- Scalar value or constant

Create an Empty Series

```
#import the pandas library and aliasing as pd
import pandas as pd
s = pd.Series()
print s
```

Output

```
Series([], dtype: float64)
```

Creating a Series

You can convert a list,numpy array, or dictionary to a Series:

```
In [3]: labels = ['p','q','r']
        my_list = [10,20,30]
        arr = np.array([10,20,30])
        d = {'a':10,'b':20,'c':30}
        arr
Out[3]: array([10, 20, 30])
```

Creating a Series with List

```
In [6]: pd.Series(my_list)

Out[6]: 0    10
        1    20
        2    30
        dtype: int64
```

```
In [10]: pd.Series(data=my_list,index=labels)

Out[10]: p    10
         q    20
         r    30
         dtype: int64
```

```
In [6]: pd.Series(my_list,labels)

Out[6]: a    10
        b    20
        c    30
        dtype: int64
```

Creating a Series with NumPy Arrays

```
In [7]: pd.Series(arr)

Out[7]: 0    10
        1    20
        2    30
        dtype: int64
```

```
In [11]: pd.Series(arr,labels)

Out[11]: p    10
         q    20
         r    30
         dtype: int64
```

Creating Series with Dictionary

```
In [12]: pd.Series(d)

Out[12]: a    10
         b    20
         c    30
         dtype: int64
```

Data in Series

A pandas Series can hold a variety of object types:

```
In [10]: pd.Series(data=labels)

Out[10]: 0    a
         1    b
         2    c
         dtype: object

In [11]: # Even functions (although unlikely that you will use this)
         pd.Series([sum,print,len])

Out[11]: 0      <built-in function sum>
         1    <built-in function print>
         2      <built-in function len>
         dtype: object
```

Using an Index

The key to using a Series is understanding its index. Pandas makes use of these index names or numbers by allowing for fast look ups of information (works like a hash table or dictionary).

Let's see some examples of how to grab information from a Series. Let us create two sereis, ser1 and ser2:

```
In [14]: ser1 = pd.Series([1,2,3,4],index = ['USA', 'Germany','USSR', 'Japan'])

In [15]: ser1

Out[15]: USA        1
         Germany    2
         USSR       3
         Japan      4
         dtype: int64

In [16]: ser2 = pd.Series([1,2,5,4],index = ['USA', 'Germany','Italy', 'Japan'])

In [20]: ser2
         type(ser2)

Out[20]: pandas.core.series.Series
```

```
In [16]: ser1['USA']

Out[16]: 1
```

Operations are then also done based off of index:

```
In [21]: ser1 + ser2

Out[21]: Germany    4.0
         Italy      NaN
         Japan      8.0
         USA        2.0
         USSR       NaN
         dtype: float64
```

DataFrames

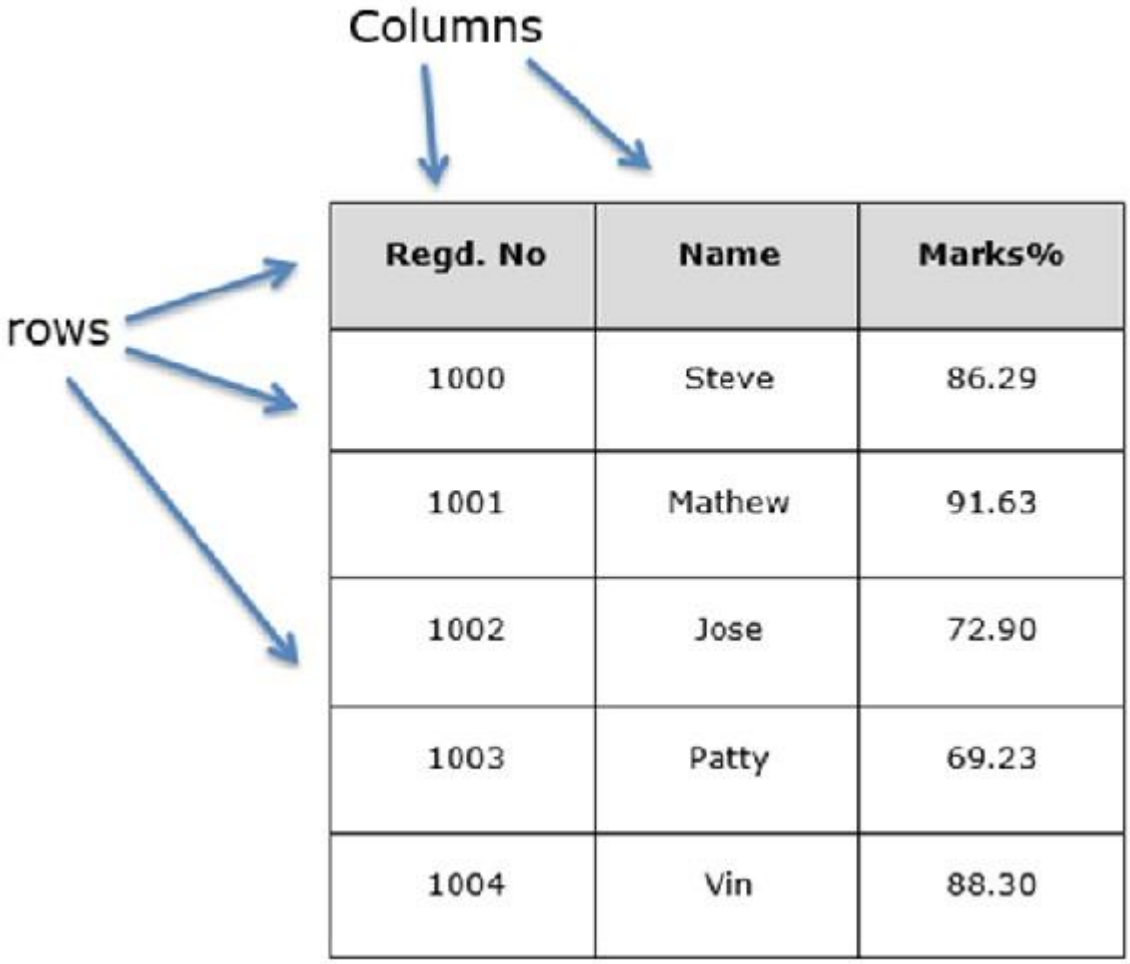
A Data frame is a two-dimensional data structure, i.e., data is aligned in a tabular fashion in rows and columns.

Features of DataFrame

- Potentially columns are of different types
- Size – Mutable
- Labeled axes (rows and columns)
- Can Perform Arithmetic operations on rows and columns

Structure

Let us assume that we are creating a data frame with student's data.



You can think of it as an SQL table or a spreadsheet data representation.

pandas.DataFrame

A pandas DataFrame can be created using the following constructor −

pandas.DataFrame(data, index, columns, dtype, copy)

The parameters of the constructor are as follows −

Sr.No	Parameter & Description
1	**data** data takes various forms like ndarray, series, map, lists, dict, constants and also another DataFrame.
2	**index**

	For the row labels, the Index to be used for the resulting frame is Optional Default np.arrange(n) if no index is passed.
3	**columns** For column labels, the optional default syntax is - np.arrange(n). This is only true if no index is passed.
4	**dtype** Data type of each column.
5	**copy** This command (or whatever it is) is used for copying of data, if the default is False.

Create DataFrame

A pandas DataFrame can be created using various inputs like −

- Lists
- dict
- Series
- Numpy ndarrays
- Another DataFrame

DataFrames are the workhorse of pandas and are directly inspired by the R programming language. We can think of a DataFrame as a bunch of Series objects put together to share the same index. Let's use pandas to explore this topic!

```
In [208]: import pandas as pd
          import numpy as np

In [209]:

In [210]: df = pd.DataFrame(randn(5,4),index='A B C D E'.split(),columns='W X Y Z'.split())

In [211]: df
```

Out[211]:

	W	X	Y	Z
A	1.624345	-0.611756	-0.528172	-1.072969
B	0.865408	-2.301539	1.744812	-0.761207
C	0.319039	-0.249370	1.462108	-2.060141
D	-0.322417	-0.384054	1.133769	-1.099891
E	-0.172428	-0.877858	0.042214	0.582815

Selection and Indexing

```
In [212]: df[['W']]
```

Out[212]:

	W
A	1.624345
B	0.865408
C	0.319039
D	-0.322417
E	-0.172428

Passing a List of Column names

```
In [213]: # Pass a list of column names
          df[['W','Z']]
```

Out[213]:

	W	Z
A	1.624345	-1.072969
B	0.865408	-0.761207
C	0.319039	-2.060141
D	-0.322417	-1.099891
E	-0.172428	0.582815

Creating new Column name

```
In [216]: df['new'] = df['W'] + df['Y']
```

```
In [217]: df
```

Out[217]:

	W	X	Y	Z	new
A	1.624345	-0.611756	-0.528172	-1.072969	1.096174
B	0.865408	-2.301539	1.744812	-0.761207	2.610219
C	0.319039	-0.249370	1.462108	-2.060141	1.781147
D	-0.322417	-0.384054	1.133769	-1.099891	0.811352
E	-0.172428	-0.877858	0.042214	0.582815	-0.130214

Removing Column

```
In [220]: df.drop('new',axis=1,inplace=True)

In [221]: df
```

Out[221]:

	W	X	Y	Z
A	1.624345	-0.611756	-0.528172	-1.072969
B	0.865408	-2.301539	1.744812	-0.761207
C	0.319039	-0.249370	1.462108	-2.060141
D	-0.322417	-0.384054	1.133769	-1.099891
E	-0.172428	-0.877858	0.042214	0.582815

Removing Rows

```
In [222]: df.drop('E',axis=0)
```

Out[222]:

	W	X	Y	Z
A	1.624345	-0.611756	-0.528172	-1.072969
B	0.865408	-2.301539	1.744812	-0.761207
C	0.319039	-0.249370	1.462108	-2.060141
D	-0.322417	-0.384054	1.133769	-1.099891

Selecting Rows

```
In [223]: df.loc[['A','B']]
```

Out[223]:

	W	X	Y	Z
A	1.624345	-0.611756	-0.528172	-1.072969
B	0.865408	-2.301539	1.744812	-0.761207

Or select based off of position instead of label

```
In [224]: df.iloc[[2]]
```

Out[224]:

	W	X	Y	Z
C	0.319039	-0.24937	1.462108	-2.060141

Selecting subset of rows and columns

```
In [225]: df.loc['B','Y']
Out[225]: 1.74481176421648

In [226]: df.loc[['A','B'],['W','Y']]
```

Out[226]:

	W	Y
A	1.624345	-0.528172
B	0.865408	1.744812

Conditional Selection

An important feature of pandas is conditional selection using bracket notation, very similar to numpy:

```
In [228]: df>0
```

Out[228]:

	W	X	Y	Z
A	True	False	False	False
B	True	False	True	False
C	True	False	True	False
D	False	False	True	False
E	False	False	True	True

```
In [229]: df[df>0]
```

Out[229]:

	W	X	Y	Z
A	1.624345	NaN	NaN	NaN
B	0.865408	NaN	1.744812	NaN
C	0.319039	NaN	1.462108	NaN
D	NaN	NaN	1.133769	NaN
E	NaN	NaN	0.042214	0.582815

```
In [231]: df[df['W']>0]
```

Out[231]:

	W	X	Y	Z
A	1.624345	-0.611756	-0.528172	-1.072969
B	0.865408	-2.301539	1.744812	-0.761207
C	0.319039	-0.249370	1.462108	-2.060141

```
In [233]: df[df['W']>0][['Y','X']]
```

Out[233]:

	Y	X
A	-0.528172	-0.611756
B	1.744812	-2.301539
C	1.462108	-0.249370

For two conditions you can use | and & with parenthesis

```
In [186]: df[(df['W']>0) & (df['Y'] > 1)]
```

Out[186]:

	W	X	Y	Z
B	0.865408	-2.301539	1.744812	-0.761207
C	0.319039	-0.249370	1.462108	-2.060141

Reset Index

```
In [234]: # Reset to default 0,1...n index
          df.reset_index()
```

Out[234]:

	index	W	X	Y	Z
0	A	1.624345	-0.611756	-0.528172	-1.072969
1	B	0.865408	-2.301539	1.744812	-0.761207
2	C	0.319039	-0.249370	1.462108	-2.060141
3	D	-0.322417	-0.384054	1.133769	-1.099891
4	E	-0.172428	-0.877858	0.042214	0.582815

```
In [236]: newind = 'CA NY WY OR CO'.split()

In [237]: df['States'] = newind

In [238]: df
```

```
In [194]: df.set_index('States',inplace=True)
```

```
In [195]: df
```

Out[195]:

States	W	X	Y	Z
CA	1.624345	-0.611756	-0.528172	-1.072969
NY	0.865408	-2.301539	1.744812	-0.761207
WY	0.319039	-0.249370	1.462108	-2.060141
OR	-0.322417	-0.384054	1.133769	-1.099891
CO	-0.172428	-0.877858	0.042214	0.582815

Multi-Index and Index Hierarchy

Let us go over how to work with Multi-Index, first we'll create a quick example of what a Multi-Indexed DataFrame would look like:

```
In [196]: # Index Levels
          outside = ['G1','G1','G1','G2','G2','G2']
          inside = [1,2,3,1,2,3]
          hier_index = list(zip(outside,inside))
          hier_index = pd.MultiIndex.from_tuples(hier_index)
```

```
In [197]: hier_index
```

```
Out[197]: MultiIndex(levels=[['G1', 'G2'], [1, 2, 3]],
                     labels=[[0, 0, 0, 1, 1, 1], [0, 1, 2, 0, 1, 2]])
```

```
In [240]: df = pd.DataFrame(np.random.randn(6,2),index=hier_index,columns=['A','B'])
          df
```

Out[240]:

Group	Num	A	B
G1	1	-1.100619	1.144724
	2	0.901591	0.502494
	3	0.900856	-0.683728
G2	1	-0.122890	-0.935769
	2	-0.267888	0.530355
	3	-0.691661	-0.396754

Now let's show how to index this! For index hierarchy we use df.loc[], if this was on the columns axis, you would just use normal bracket notation df[]. Calling one level of the index returns the sub-dataframe:

```
In [199]: df.loc['G1']
```

Out[199]:

	A	B
1	-1.100619	1.144724
2	0.901591	0.502494
3	0.900856	-0.683728

```
In [200]: df.loc['G1'].loc[1]
Out[200]: A   -1.100619
          B    1.144724
          Name: 1, dtype: float64

In [201]: df.index.names
Out[201]: FrozenList([None, None])
```

```
In [205]: df.xs(['G1',1])
Out[205]: A   -1.100619
          B    1.144724
          Name: (G1, 1), dtype: float64

In [206]: df.xs(1,level='Num')
```

Out[206]:

	A	B
Group		
G1	-1.100619	1.144724
G2	-0.122890	-0.935769

Missing Data

Let's show a few convenient methods to deal with Missing Data in pandas

```
In [9]: df = pd.DataFrame({'A':[1,2,np.nan],
                           'B':[5,np.nan,np.nan],
                           'C':[1,2,3]})

In [10]: df
```

Out[10]:

	A	B	C
0	1.0	5.0	1
1	2.0	NaN	2
2	NaN	NaN	3

Dropping NaN in columns and Rows value

```
In [12]: df.dropna()
```

Out[12]:

	A	B	C
0	1.0	5.0	1

```
In [13]: df.dropna(axis=1)
```

Out[13]:

	C
0	1
1	2
2	3

Groupby

The groupby method allows you to group rows of data together and call aggregate functions

```
In [31]: import pandas as pd
         # Create dataframe
         data = {'Company':['GOOG','GOOG','MSFT','MSFT','FB','FB'],
                 'Person':['Sam','Charlie','Amy','Vanessa','Carl','Sarah'],
                 'Sales':[200,120,340,124,243,350]}

In [32]: df = pd.DataFrame(data)

In [33]: df
```

Out[33]:

	Company	Person	Sales
0	GOOG	Sam	200
1	GOOG	Charlie	120
2	MSFT	Amy	340
3	MSFT	Vanessa	124
4	FB	Carl	243
5	FB	Sarah	350

Now you can use the .groupby() method to group rows together based off of a column name. For instance let's group based off of Company. This will create a DataFrameGroupBy object:

```
In [35]: by_comp = df.groupby("Company")
```

And then call aggregate methods off the object:

```
In [36]: by_comp.mean()
```

Out[36]:

Company	Sales
FB	296.5
GOOG	160.0
MSFT	232.0

Grouping and mean

```
In [37]: df.groupby('Company').mean()
```

Out[37]:

Company	Sales
FB	296.5
GOOG	160.0
MSFT	232.0

More examples of aggregate methods:

```
In [38]: by_comp.std()
```

Out[38]:

Company	Sales
FB	75.660426
GOOG	56.568542
MSFT	152.735065

```
In [39]: by_comp.min()
```

Out[39]:

Company	Person	Sales
FB	Carl	243
GOOG	Charlie	120
MSFT	Amy	124

```
In [40]: by_comp.max()
```

Out[40]:

Company	Person	Sales
FB	Sarah	350
GOOG	Sam	200
MSFT	Vanessa	340

Describing the dataFrame

Pandas describe() is used to view some basic statistical details like percentile, mean, std etc. of a data frame or a series of numeric values.

```
In [42]: by_comp.describe()
```

Out[42]:

Company		Sales
FB	count	2.000000
	mean	296.500000
	std	75.660426
	min	243.000000
	25%	269.750000
	50%	296.500000
	75%	323.250000
	max	350.000000
GOOG	count	2.000000
	mean	160.000000
	std	56.568542
	min	120.000000
	25%	140.000000
	50%	160.000000
	75%	180.000000
	max	200.000000
MSFT	count	2.000000
	mean	232.000000
	std	152.735065
	min	124.000000
	25%	178.000000
	50%	232.000000
	75%	286.000000
	max	340.000000

Transpose method in Dataframe

Pandas **DataFrame.transpose()** function transpose index and columns of the dataframe. It reflect the DataFrame over its main diagonal by writing rows as columns and vice-versa.

```
In [44]: by_comp.describe().transpose()['GOOG']
```

Out[44]:

	count	mean	std	min	25%	50%	75%	max
Sales	2.0	160.0	56.568542	120.0	140.0	160.0	180.0	200.0

5.3 Matplotlib

Matplotlib is the "grandfather" library of data visualization with Python. It was created by John Hunter. He created it to try to replicate MatLab's (another programming language) plotting capabilities in Python. So if you happen to be familiar with matlab, matplotlib will feel natural to you.

It is an excellent 2D and 3D graphics library for generating scientific figures.

Some of the major Pros of Matplotlib are:

- Generally easy to get started for simple plots
- Support for custom labels and texts
- Great control of every element in a figure
- High-quality output in many formats
- Very customizable in general

Matplotlib allows you to create reproducible figures programmatically. Let's learn how to use it! Before continuing this lecture, I encourage you just to explore the official Matplotlib web page: http://matplotlib.org/

Installation

You'll need to install matplotlib first with either:

conda install matplotlib

or pip install matplotlib

Importing

Import the matplotlib.pyplot module under the name plt (the tidy way):

```
In [1]: import matplotlib.pyplot as plt
```

You'll also need to use this line to see plots in the notebook:

```
In [50]: %matplotlib inline
```

Note: That line is only for jupyter notebooks, if you are using another editor, you'll use: **plt.show()** at the end of all your plotting commands to have the figure pop up in another window.

The data we want to plot:

```
In [2]: import numpy as np
        x = np.linspace(0, 5, 11)
        y = x ** 2

In [12]: x

Out[12]: array([0. , 0.5, 1. , 1.5, 2. , 2.5, 3. , 3.5, 4. , 4.5, 5. ])

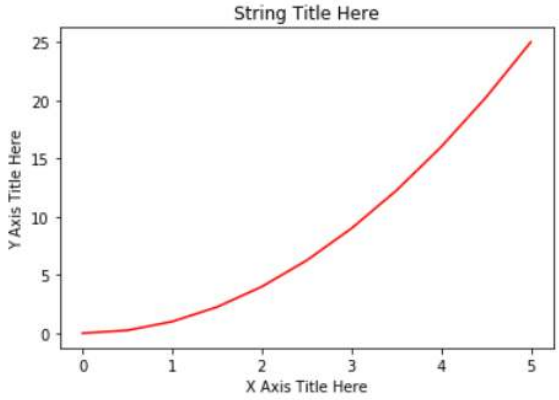
In [13]: y

Out[13]: array([ 0.  ,  0.25,  1.  ,  2.25,  4.  ,  6.25,  9.  , 12.25, 16.  ,
                20.25, 25.  ])
```

Basic Matplotlib Commands

We can create a very simple line plot using the following (I encourage you to pause and use Shift+Tab along the way to check out the document strings for the functions we are using).

```
In [3]: plt.plot(x, y, 'r') # 'r' is the color red
        plt.xlabel('X Axis Title Here')
        plt.ylabel('Y Axis Title Here')
        plt.title('String Title Here')
        plt.show()
```

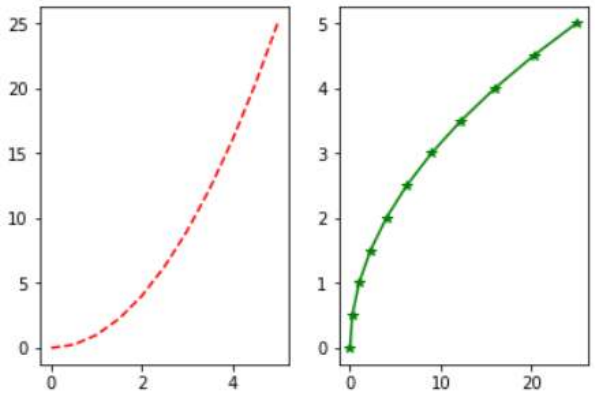


matplotlib.pyplot.plot(**args*, *scalex=True*, *scaley=True*, *data=None*, ***kwargs*)

Creating Multiplots on Same Canvas

```
In [4]: # plt.subplot(nrows, ncols, plot_number)
        plt.subplot(1,2,1)
        plt.plot(x, y, 'r--') # More on color options later
        plt.subplot(1,2,2)
        plt.plot(y, x, 'g*-')
         # More on color options later
```

```
Out[4]: [<matplotlib.lines.Line2D at 0x1174e16eda0>]
```



Matplotlib Object Oriented Method

Now that we've seen the basics, let's break it all down with a more formal introduction of Matplotlib's Object Oriented API. This means we will instantiate figure objects and then call methods or attributes from that object.

Introduction to the Object Oriented Method

The main idea in using the more formal Object Oriented method is to create figure objects and then just call methods or attributes off of that object. This approach is nicer when dealing with a canvas that has multiple plots on it.

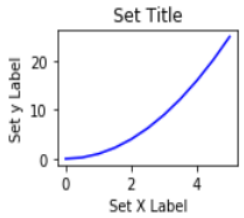
To begin we create a figure instance. Then we can add axes to that figure:

In [9]:
```
# Create Figure (empty canvas)
fig = plt.figure()

# Add set of axes to figure
axes = fig.add_axes([0.1, 0.1, 0.3, 0.3]) # left, bottom, width, height (range 0 to 1)

# Plot on that set of axes
axes.plot(x, y, 'b')
axes.set_xlabel('Set X Label') # Notice the use of set_ to begin methods
axes.set_ylabel('Set y Label')
axes.set_title('Set Title')
```

Out[9]:
```
Text(0.5,1,'Set Title')
```



Code is a little more complicated, but the advantage is that we now have full control of where the plot axes are placed, and we can easily add more than one axis to the figure:

```
fig = plt.figure()

axes1 = fig.add_axes([0.1, 0.1, 0.8, 0.8]) # main axes
axes2 = fig.add_axes([0.2, 0.5, 0.4, 0.3]) # inset axes

# Larger Figure Axes 1
axes1.plot(x, y, 'b')
axes1.set_xlabel('X_label_axes2')
axes1.set_ylabel('Y_label_axes2')
axes1.set_title('Axes 2 Title')

# Insert Figure Axes 2
axes2.plot(y, x, 'r')
axes2.set_xlabel('X_label_axes2')
axes2.set_ylabel('Y_label_axes2')
axes2.set_title('Axes 2 Title');
```

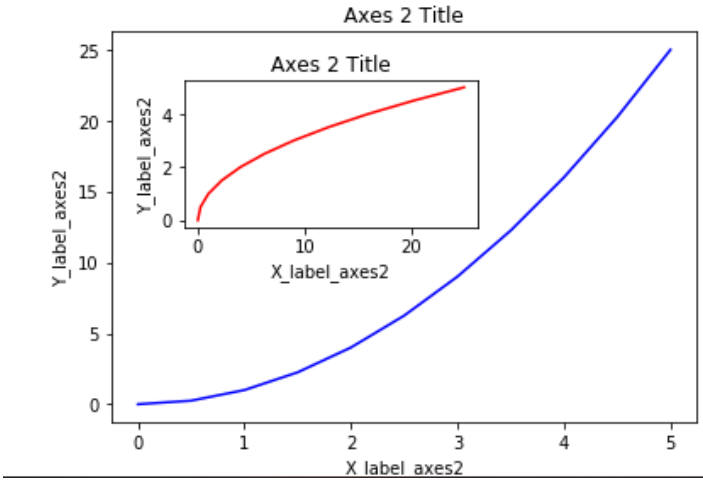


Figure size, aspect ratio and DPI

Matplotlib allows the aspect ratio, DPI and figure size to be specified when the Figure object is created. You can use the figsize and dpi keyword arguments.

- figsize is a tuple of the width and height of the figure in inches
- dpi is the dots-per-inch (pixel per inch).

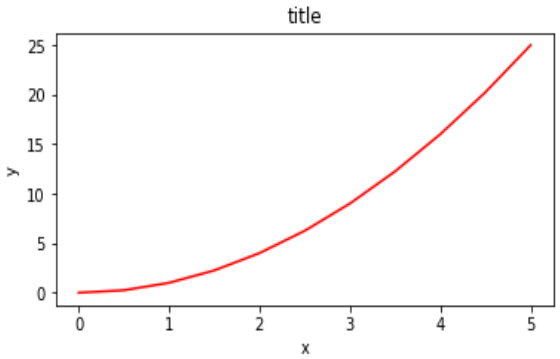
```
In [33]: fig = plt.figure(figsize=(8,4), dpi=100)
```

```
<matplotlib.figure.Figure at 0x11228ea58>
```

The same arguments can also be passed to layout managers, such as the `subplots` function:

```
In [21]: fig, axes = plt.subplots(figsize=(6,3))

         axes.plot(x, y, 'r')
         axes.set_xlabel('x')
         axes.set_ylabel('y')
         axes.set_title('title');
```



Saving figures

Matplotlib can generate high-quality output in a number formats, including PNG, JPG, EPS, SVG, PGF and PDF.

To save a figure to a file we can use the savefig method in the Figure class:

```
In [68]: fig.savefig("filename.png")
```

Here we can also optionally specify the DPI and choose between different output formats:

```
In [69]: fig.savefig("filename.png", dpi=200)
```

Legends, labels and titles

Now that we have covered the basics of how to create a figure canvas and add axes instances to the canvas, let's look at how decorate a figure with titles, axis labels, and legends.

Figure titles

A title can be added to each axis instance in a figure. To set the title, use the set_title method in the axes instance

```
In [41]: ax.set_title("title");
```

Axis labels

Similarly, with the methods set_xlabel and set_ylabel, we can set the labels of the X and Y axes

```
In [42]: ax.set_xlabel("x")
         ax.set_ylabel("y");
```

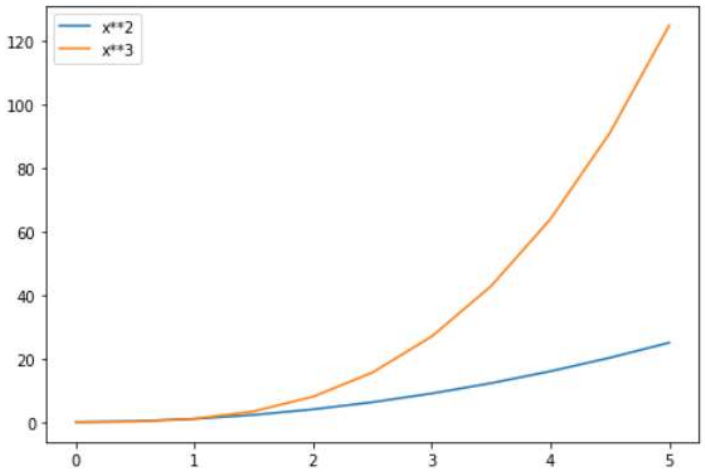
Legends

You can use the **label="label text"** keyword argument when plots or other objects are added to the figure, and then using the **legend** method without arguments to add the legend to the figure

```
ax = fig.add_axes([0,0,1,1])

ax.plot(x, x**2, label="x**2")
ax.plot(x, x**3, label="x**3")
ax.legend(loc=0)
```

```
Out[71]: <matplotlib.legend.Legend at 0x1d8a2f69b70>
```



Notice how are legend overlaps some of the actual plot!

The **legend** function takes an optional keyword argument **loc** that can be used to specify where in the figure the legend is to be drawn. The allowed values of **loc** are numerical codes for the various places the legend can be drawn.

```
ax.legend(loc=1) # upper right corner

ax.legend(loc=2) # upper left corner

ax.legend(loc=3) # lower left corner

ax.legend(loc=4) # lower right corner
```

Setting colors, linewidths, linetypes

Matplotlib gives you *a lot* of options for customizing colors, linewidths, and linetypes.

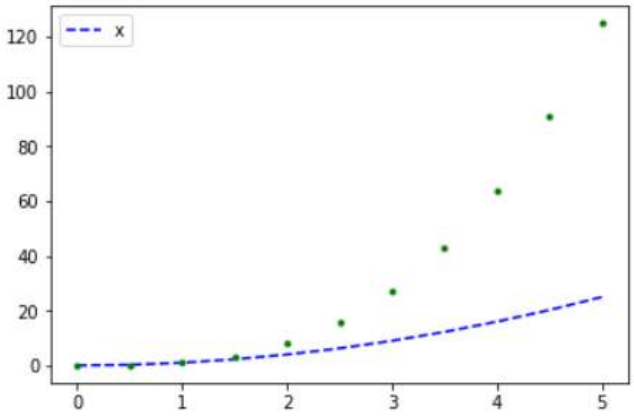
There is the basic MATLAB like syntax (which I would suggest you avoid using for more clairty sake:

Colors with MatLab like syntax

With matplotlib, we can define the colors of lines and other graphical elements in a number of ways. First of all, we can use the MATLAB-like syntax where 'b' means blue, 'g' means green, etc. The MATLAB API for selecting line styles are also supported: where, for example, 'b.-' means a blue line with dots:

```
In [48]: # MATLAB style line color and style
         fig, ax = plt.subplots()
         ax.plot(x, x**2,'b--', label="x") # blue line with dots
         ax.plot(x, x**3, 'g.')
         ax.legend()# green dashed line
```

```
Out[48]: <matplotlib.legend.Legend at 0x1d8a2c34da0>
```



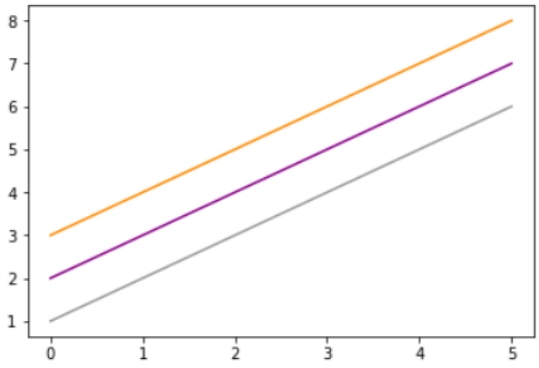
Colors with the color= parameter

We can also define colors by their names or RGB hex codes and optionally provide an alpha value using the color and alpha keyword arguments. Alpha indicates opacity.

```
In [25]: fig, ax = plt.subplots()

         ax.plot(x, x+1, color="black", alpha=0.4) # half-transparant
         ax.plot(x, x+2, color="#8B008B")        # RGB hex code
         ax.plot(x, x+3, color="#FF8C00")        # RGB hex code
```

```
Out[25]: [<matplotlib.lines.Line2D at 0x1174ea67a58>]
```



Line and marker styles

To change the line width, we can use the linewidth or lw keyword argument. The line style can be selected using the linestyle or ls keyword arguments:

```
fig, ax = plt.subplots(figsize=(12,6))

ax.plot(x, x+1, color="red", linewidth=0.25)

ax.plot(x, x+2, color="red", linewidth=0.50)

ax.plot(x, x+3, color="red", linewidth=1.00)

ax.plot(x, x+4, color="red", linewidth=5.00)



# possible linestype options ‘-‘, ‘–’, ‘-.’, ‘:’, ‘steps’

ax.plot(x, x+5, color="green", lw=3, linestyle='-')

ax.plot(x, x+6, color="green", lw=3, ls='-.')

ax.plot(x, x+7, color="green", lw=3, ls=':')



# custom dash

line, = ax.plot(x, x+8, color="black", lw=1.50)

line.set_dashes([5, 10, 15, 10]) # format: line length, space length, ...



# possible marker symbols: marker = '+', 'o', '*', 's', ',', '.', '1', '2', '3', '4', ...

ax.plot(x, x+ 9, color="blue", lw=3, ls='-', marker='+')

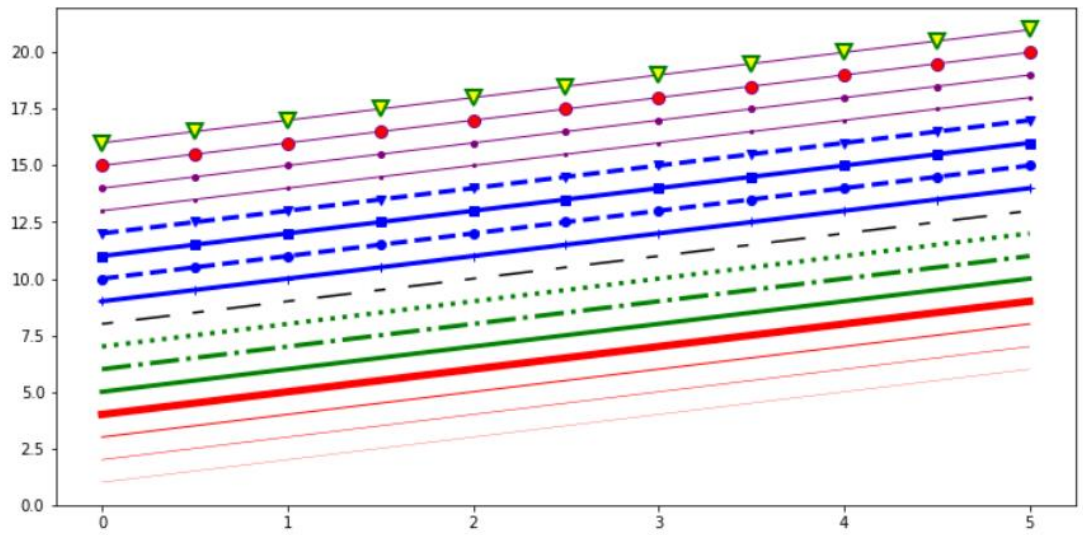
ax.plot(x, x+10, color="blue", lw=3, ls='--', marker='o')

ax.plot(x, x+11, color="blue", lw=3, ls='-', marker='s')

ax.plot(x, x+12, color="blue", lw=3, ls='--', marker='v')
```

```
# marker size and color
ax.plot(x, x+13, color="purple", lw=1, ls='-', marker='o', markersize=2)
ax.plot(x, x+14, color="purple", lw=1, ls='-', marker='o', markersize=4)
ax.plot(x, x+15, color="purple", lw=1, ls='-', marker='o', markersize=8, markerfacecolor="red")
ax.plot(x, x+16, color="purple", lw=1, ls='-', marker='v', markersize=10,
        markerfacecolor="yellow", markeredgewidth=2, markeredgecolor="green");
```

Output for above



5.4 SciPy

SciPy is a collection of mathematical algorithms and convenience functions built on the Numpy extension of Python. It adds significant power to the interactive Python session by providing the user with high-level commands and classes for manipulating and visualizing data. With SciPy an interactive Python session becomes a data-processing and system-prototyping environment rivaling systems such as MATLAB, IDL, Octave, R-Lab, and SciLab.

The additional benefit of basing SciPy on Python is that this also makes a powerful programming language available for use in developing sophisticated programs and specialized applications. Scientific applications using SciPy benefit from the development of additional modules in numerous niches of the software landscape by developers across the world.

Everything from parallel programming to web and data-base subroutines and classes have been made available to the Python programmer. All of this power is available in addition to the mathematical libraries in SciPy.

We'll focus a lot more on NumPy arrays, but let's show some of the capabilities of SciPy:

```
In [21]: import numpy as np
         A = np.array([[1,2,3],[4,5,6],[7,8,8]])
```

Linear Algebra

linalg

```
In [22]: from scipy import linalg
```

Determinant of a Matrix

```
In [23]: # Compute the determinant of a matrix
         linalg.det(A)
Out[23]: 2.999999999999997
```

Compute pivoted LU decomposition of a matrix.

The decomposition is::

A = P L U

where P is a permutation matrix, L lower triangular with unit diagonal elements, and U upper triangular.

```
In [24]: P, L, U = linalg.lu(A)

In [25]: P
Out[25]: array([[ 0.,  1.,  0.],
                [ 0.,  0.,  1.],
                [ 1.,  0.,  0.]])

In [26]: L
Out[26]: array([[ 1.        ,  0.        ,  0.        ],
                [ 0.14285714,  1.        ,  0.        ],
                [ 0.57142857,  0.5       ,  1.        ]])

In [27]: U
Out[27]: array([[ 7.        ,  8.        ,  8.        ],
                [ 0.        ,  0.85714286,  1.85714286],
                [ 0.        ,  0.        ,  0.5       ]])

In [28]: np.dot(L,U)
Out[28]: array([[ 7.,  8.,  8.],
                [ 1.,  2.,  3.],
                [ 4.,  5.,  6.]])
```

We can find out the eigenvalues and eigenvectors of this matrix:

```
In [30]: EW, EV = linalg.eig(A)

In [31]: EW
Out[31]: array([ 15.55528261+0.j,  -1.41940876+0.j,  -0.13587385+0.j])

In [32]: EV
Out[32]: array([[-0.24043423, -0.67468642,  0.51853459],
                [-0.54694322, -0.23391616, -0.78895962],
                [-0.80190056,  0.70005819,  0.32964312]])
```

Solving systems of linear equations can also be done:

```
In [41]: v = np.array([[2],[3],[5]])

In [42]: v
Out[42]: array([[2],
                [3],
                [5]])
```

```
In [43]: s = linalg.solve(A,v)

In [44]: s
Out[44]: array([[-2.33333333],
                [ 3.66666667],
                [-1.        ]])
```

Sparse Linear Algebra

SciPy has some routines for computing with sparse and potentially very large matrices. The necessary tools are in the submodule scipy.sparse.

We make one example on how to construct a large matrix

```
In [48]: from scipy import sparse

In [49]: # Row-based linked list sparse matrix
         A = sparse.lil_matrix((1000, 1000))

In [50]: A
Out[50]: <1000x1000 sparse matrix of type '<class 'numpy.float64'>'
                with 0 stored elements in LInked List format>

In [51]: A[0,:100] = np.random.rand(100)

In [52]: A[1,100:200] = A[0,:100]

In [54]: A.setdiag(np.random.rand(1000))

In [57]: A
Out[57]: <1000x1000 sparse matrix of type '<class 'numpy.float64'>'
                with 1199 stored elements in LInked List format>
```

Linear Algebra for Sparse Matrices

```
In [56]: from scipy.sparse import linalg

In [64]: # Convert this matrix to Compressed Sparse Row format.
         A.tocsr()

Out[64]: <1000x1000 sparse matrix of type '<class 'numpy.float64'>'
                with 1199 stored elements in Compressed Sparse Row format>

In [65]: A = A.tocsr()

In [66]: b = np.random.rand(1000)

In [67]: linalg.spsolve(A, b)

Out[67]: array([ -5.95139017e+01,  -2.11676791e+02,   8.71500295e-01,
                  8.80692794e+00,   3.92509926e-01,   4.07735642e+00,
                  1.03953304e+00,   8.67487093e-01,   1.04239180e+00,
                  5.64067865e-01,   5.66852537e-01,   1.44603822e+01,
                  3.08908787e-01,   3.84755428e+00,   2.59572317e-01,
                  2.01455584e+00,   7.22529398e-01,   3.10774145e-01,
                  5.99430111e-01,   9.94908214e-01,   8.16556218e-01,
                  6.13388488e-01,   5.14511939e-01,   5.93379845e-01,
                  7.16747945e+00,   1.12059905e+00,   2.57598133e+00,
                  7.23065380e-01,   1.11316273e+00,   5.71046430e-01,
                  6.07181161e-01,   1.88477643e+00,   2.30443811e+00,
```

5.5 Scikit

Scikit-learn provides a range of supervised and unsupervised learning algorithms via a consistent interface in Python.

It is licensed under a permissive simplified BSD license and is distributed under many Linux distributions, encouraging academic and commercial use.

The library is built upon the SciPy (Scientific Python) that must be installed before you can use scikit-learn. This stack that includes:

- **NumPy**: Base n-dimensional array package
- **SciPy**: Fundamental library for scientific computing
- **Matplotlib**: Comprehensive 2D/3D plotting
- **IPython**: Enhanced interactive console
- **Sympy**: Symbolic mathematics
- **Pandas**: Data structures and analysis

Extensions or modules for SciPy care conventionally named SciKits. As such, the module provides learning algorithms and is named scikit-learn.

The vision for the library is a level of robustness and support required for use in production systems. This means a deep focus on concerns such as easy of use, code quality, collaboration, documentation and performance.

Although the interface is Python, c-libraries are leverage for performance such as numpy for arrays and matrix operations, LAPACK, LibSVM and the careful use of cython.

What are the features?

The library is focused on modeling data. It is not focused on loading, manipulating and summarizing data.

Some popular groups of models provided by scikit-learn include:

- **Clustering**: for grouping unlabeled data such as KMeans.
- **Cross Validation**: for estimating the performance of supervised models on unseen data.
- **Datasets**: for test datasets and for generating datasets with specific properties for investigating model behavior.
- **Dimensionality Reduction**: for reducing the number of attributes in data for summarization, visualization and feature selection such as Principal component analysis.
- **Ensemble methods**: for combining the predictions of multiple supervised models.
- **Feature extraction**: for defining attributes in image and text data.
- **Feature selection**: for identifying meaningful attributes from which to create supervised models.
- **Parameter Tuning**: for getting the most out of supervised models.
- **Manifold Learning**: For summarizing and depicting complex multi-dimensional data.
- **Supervised Models**: a vast array not limited to generalized linear models, discriminate analysis, naive bayes, lazy methods, neural networks, support vector machines and decision trees.

5.6 Linear Regression

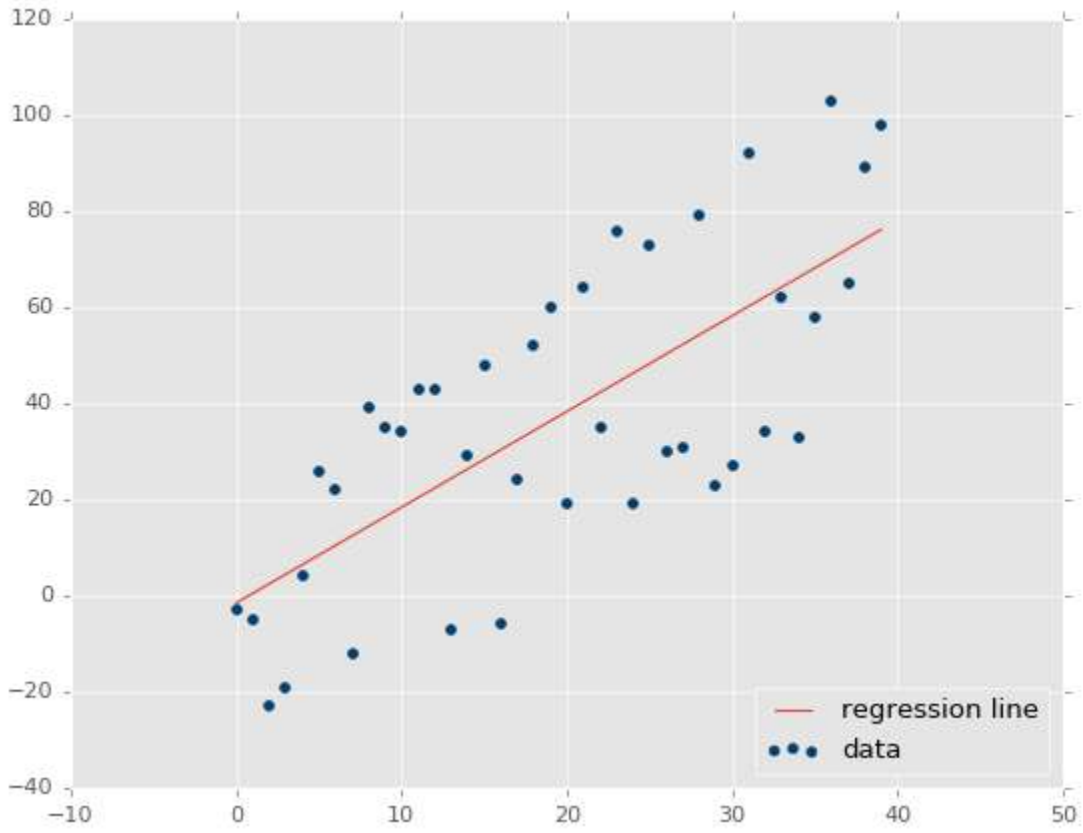
There are two types of supervised machine learning algorithms: Regression and classification. The former predicts continuous value outputs while the latter predicts discrete outputs. For instance, predicting the price of a house in dollars is a regression problem whereas predicting whether a tumor is malignant or benign is a classification problem.

In this article, we will briefly study what linear regression is and how it can be implemented for both two variables and multiple variables using **Scikit-Learn,** which is one of the most popular machine learning libraries for Python.

Linear Regression Theory
The term “linearity” in algebra refers to a linear relationship between two or more variables. If

we draw this relationship in a two-dimensional space (between two variables), we get a straight line.

Linear regression performs the task to predict a dependent variable value (y) based on a given independent variable (x). So, this regression technique finds out a linear relationship between x (input) and y(output). Hence, the name is Linear Regression. If we plot the independent variable (x) on the x-axis and dependent variable (y) on the y-axis, linear regression gives us a straight line that best fits the data points, as shown in the figure below.



The equation of the above line is :

$$\mathbf{Y = mx + b}$$

Where b is the intercept and m is the slope of the line. So basically, the linear regression algorithm gives us the most optimal value for the intercept and the slope (in two dimensions). The y and x variables remain the same, since they are the data features and cannot be changed. The values that we can control are the intercept(b) and slope(m). There can be multiple straight lines depending upon the values of intercept and slope. Basically what the linear regression algorithm does is it fits multiple lines on the data points and returns the line that results in the least error.

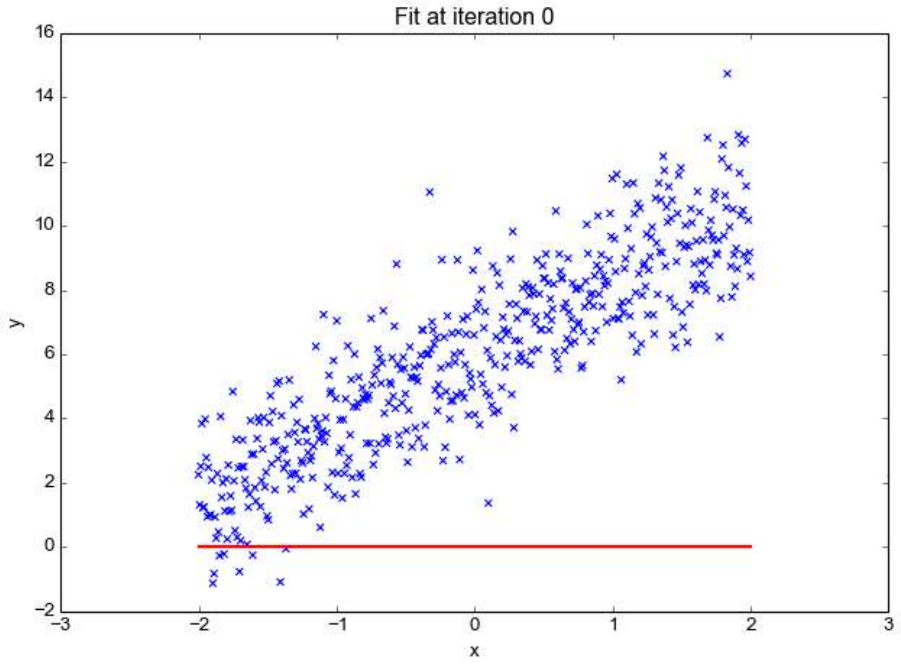
This same concept can be extended to cases where there are more than two variables. This is called multiple linear regression. For instance, consider a scenario where you have to predict the price of the house based upon its area, number of bedrooms, the average income of the people in the area, the age of the house, and so on. In this case, the dependent variable(target variable) is dependent upon several independent variables. A regression model involving multiple variables can be represented as:

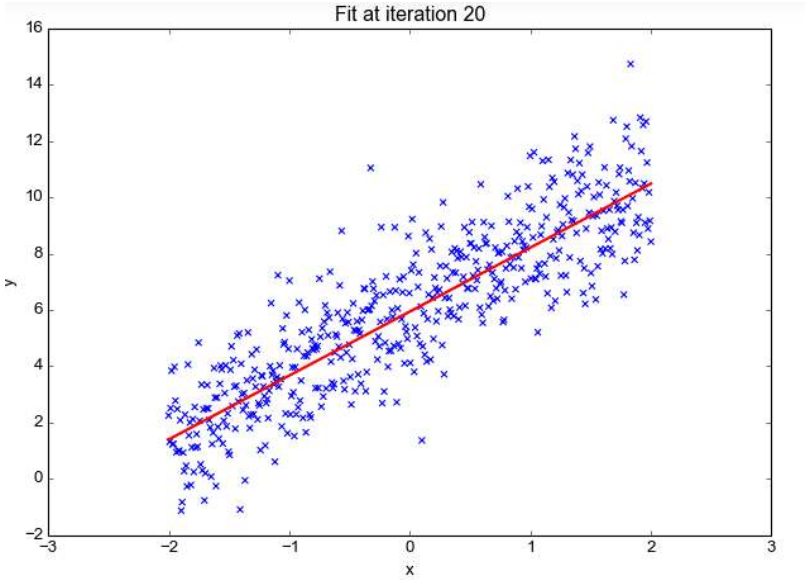
y = b0 + m1b1 + m2b2 + m3b3 + … … mnbn

This is the equation of a hyperplane. Remember, a linear regression model in two dimensions is a straight line; in three dimensions it is a plane, and in more than three dimensions, a hyperplane.

In this section, we will see how Python's Scikit-Learn library for machine learning can be used to implement regression functions. We will start with simple linear regression involving two variables and then we will move towards linear regression involving multiple variables.

Simple Linear Regression





Importing Library and Loading the data

```
In [1]: import pandas as pd
        import numpy as np
        import matplotlib.pyplot as plt
        import seaborn as sns
        %matplotlib inline
```

Check out the Data

```
In [2]: USAhousing = pd.read_csv('USA_Housing.csv')
```

```
In [3]: USAhousing.head()
```

Out[3]:

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	Avg. Area Number of Bedrooms	Area Population	Price	Address
0	79545.458574	5.682861	7.009188	4.09	23086.800503	1.059034e+06	208 Michael Ferry Apt. 674\nLaurabury, NE 3701...
1	79248.642455	6.002900	6.730821	3.09	40173.072174	1.505891e+06	188 Johnson Views Suite 079\nLake Kathleen, CA...

Using info() and describe method

```
In [4]: USAhousing.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 7 columns):
Avg. Area Income              5000 non-null float64
Avg. Area House Age           5000 non-null float64
Avg. Area Number of Rooms     5000 non-null float64
Avg. Area Number of Bedrooms  5000 non-null float64
Area Population               5000 non-null float64
Price                         5000 non-null float64
Address                       5000 non-null object
dtypes: float64(6), object(1)
memory usage: 273.5+ KB
```

```
In [5]: USAhousing.describe()
```

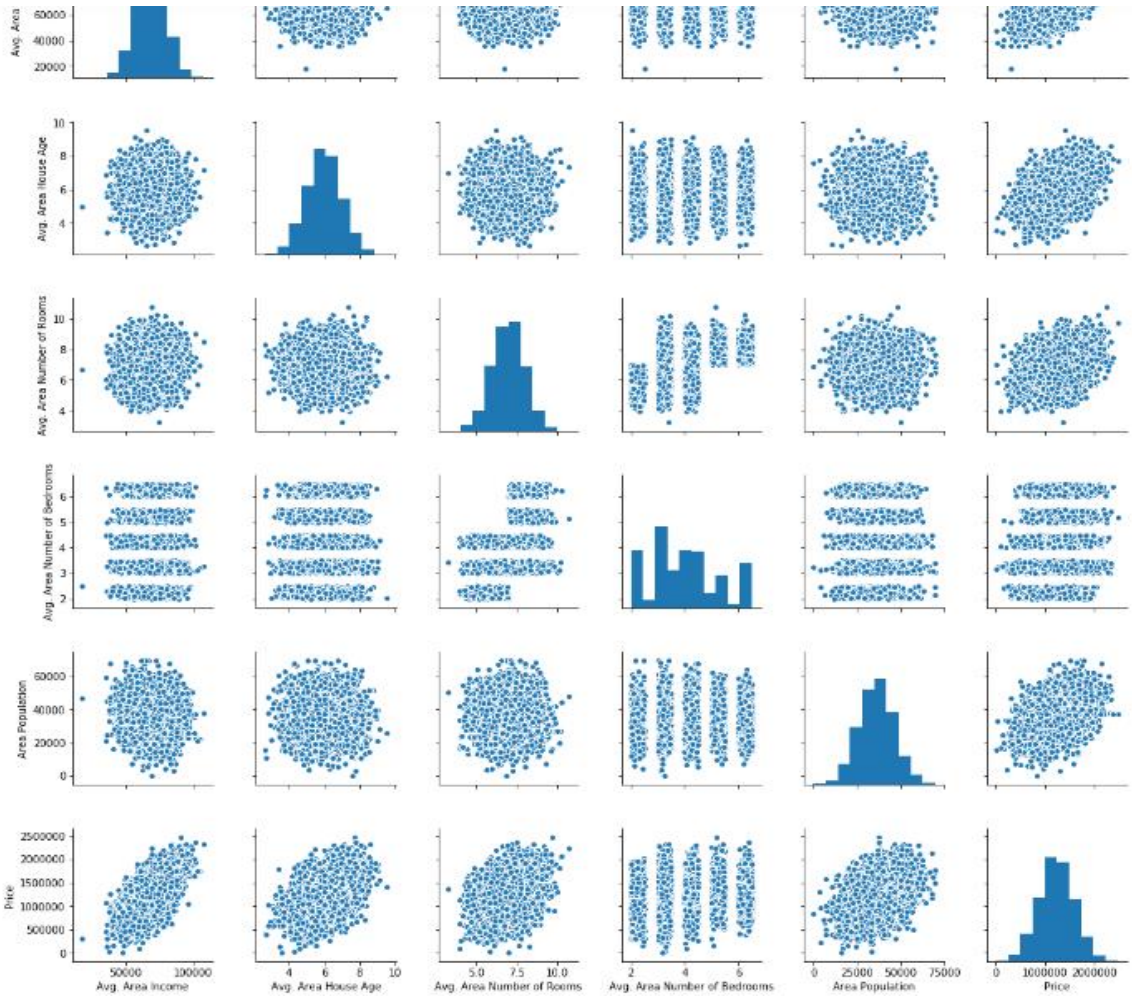
Out[5]:

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	Avg. Area Number of Bedrooms	Area Population	Price
count	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	5.000000e+03
mean	68583.108984	5.977222	6.987792	3.981330	36163.516039	1.232073e+06
std	10657.991214	0.991456	1.005833	1.234137	9925.650114	3.531176e+05
min	17796.631190	2.644304	3.236194	2.000000	172.610686	1.593866e+04
25%	61480.562388	5.322283	6.299250	3.140000	29403.928702	9.975771e+05

Exploratory Data Analysis EDA
Let's create some simple plots to check out the data!

sns.pairplot(USAhousing)

#pairplot(*data*, *hue=None*, *hue_order=None*, *palette=None*, *vars=None*, *x_vars=None*, *y_vars=None*, *kind='scatter'*, *diag_kind='auto'*, *markers=None*, *height=2.5*, *aspect=1*, *dropna=True*, *plot_kws=None*, *diag_kws=None*, *grid_kws=None*, *size=None*)



Training a Linear Regression Model

Let's now begin to train out regression model! We will need to first split up our data into an X array that contains the features to train on, and a y array with the target variable, in this case the Price column. We will toss out the Address column because it only has text info that the linear regression model can't use.

```
X = USAhousing[['Avg. Area Income', 'Avg. Area House Age', 'Avg. Area Number of Rooms',
                'Avg. Area Number of Bedrooms', 'Area Population']]
y = USAhousing['Price']
```

Train Test Split

Now let's split the data into a training set and a testing set. We will train out model on the training set and then use the test set to evaluate the model.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4, random_state=101)


//Creating and Training the Model
from sklearn.linear_model import LinearRegression
lm = LinearRegression()
lm.fit(X_train,y_train)



Model Evaluation
Let's evaluate the model by checking out it's coefficients and how we can interpret them.

# print the intercept

print(lm.intercept_)

coeff_df = pd.DataFrame(lm.coef_,X.columns,columns=['Coefficient'])

coeff_df
```

The output of the above code will be:

:

	Coefficient
Avg. Area Income	21.528276
Avg. Area House Age	164883.282027
Avg. Area Number of Rooms	122368.678027
Avg. Area Number of Bedrooms	2233.801864
Area Population	15.150420

Interpreting the coefficients:

- Holding all other features fixed, a 1 unit increase in **Avg. Area Income** is associated with an **increase of $21.52** .

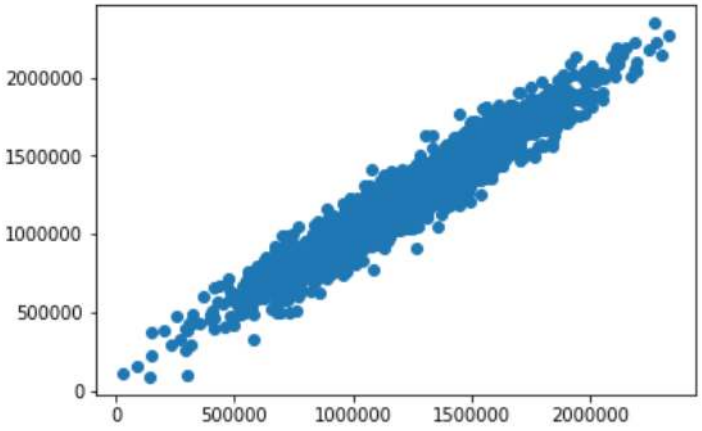
- Holding all other features fixed, a 1 unit increase in **Avg. Area House Age** is associated with an **increase of $164883.28** .
- Holding all other features fixed, a 1 unit increase in **Avg. Area Number of Rooms** is associated with an **increase of $122368.67** .
- Holding all other features fixed, a 1 unit increase in **Avg. Area Number of Bedrooms** is associated with an **increase of $2233.80** .
- Holding all other features fixed, a 1 unit increase in **Area Population** is associated with an **increase of $15.15** .

Predictions from our Model

Let's grab predictions off our test set and see how well it did!

```
In [18]: predictions = lm.predict(X_test)

In [19]: plt.scatter(y_test,predictions)
Out[19]: <matplotlib.collections.PathCollection at 0x243e8bfcc18>
```



Regression Evaluation Metrics

Here are three common evaluation metrics for regression problems:

Mean Absolute Error (MAE) is the mean of the absolute value of the errors:

$$\frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Mean Squared Error (MSE) is the mean of the squared errors:

$$\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Root Mean Squared Error (RMSE) is the square root of the mean of the squared errors:

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Comparing these metrics:

- **MAE** is the easiest to understand, because it's the average error.
- **MSE** is more popular than MAE, because MSE "punishes" larger errors, which tends to be useful in the real world.
- **RMSE** is even more popular than MSE, because RMSE is interpretable in the "y" units.

All of these are **loss functions**, because we want to minimize them.

```
from sklearn import metrics
print('MAE:', metrics.mean_absolute_error(y_test, predictions))
print('MSE:', metrics.mean_squared_error(y_test, predictions))
print('RMSE:', np.sqrt(metrics.mean_squared_error(y_test, predictions)))

Output
MAE: 82288.22251914957
MSE: 10460958907.209501
RMSE: 102278.82922291153
```

5.7 Logistic regression

Logistic Regression is a Machine Learning classification algorithm that is used to predict the probability of a categorical dependent variable. In logistic regression, the dependent variable is a binary variable that contains data coded as 1 (yes, success, etc.) or 0 (no, failure, etc.). In other words, the logistic regression model predicts P(Y=1) as a function of X.

Logistic Regression is one of the most popular ways to fit models for categorical data, especially for binary response data in Data Modeling. It is the most important (and probably most used) member of a class of models called generalized linear models. Unlike linear regression, logistic regression can directly predict probabilities (values that are restricted to the (0,1) interval); furthermore, those probabilities are well-calibrated when compared to the probabilities predicted

by some other classifiers, such as Naive Bayes. Logistic regression preserves the marginal probabilities of the training data. The coefficients of the model also provide some hint of the relative importance of each input variable.

Logistic Regression is used when the dependent variable (target) is categorical.

For example,

- To predict whether an email is spam (1) or (0)
- Whether the tumor is malignant (1) or not (0)

Consider a scenario where we need to classify whether an email is spam or not. If we use linear regression for this problem, there is a need for setting up a threshold based on which classification can be done. Say if the actual class is malignant, predicted continuous value 0.4 and the threshold value is 0.5, the data point will be classified as not malignant which can lead to serious consequence in real time.

From this example, it can be inferred that linear regression is not suitable for classification problem. Linear regression is unbounded, and this brings logistic regression into picture. Their value strictly ranges from 0 to 1.

Logistic regression is generally used where the dependent variable is Binary or Dichotomous. That means the dependent variable can take only two possible values such as "Yes or No", "Default or No Default", "Living or Dead", "Responder or Non Responder", "Yes or No" etc. Independent factors or variables can be categorical or numerical variables.

Logistic Regression Assumptions:

· Binary logistic regression requires the dependent variable to be binary.

· For a binary regression, the factor level 1 of the dependent variable should represent the desired outcome.

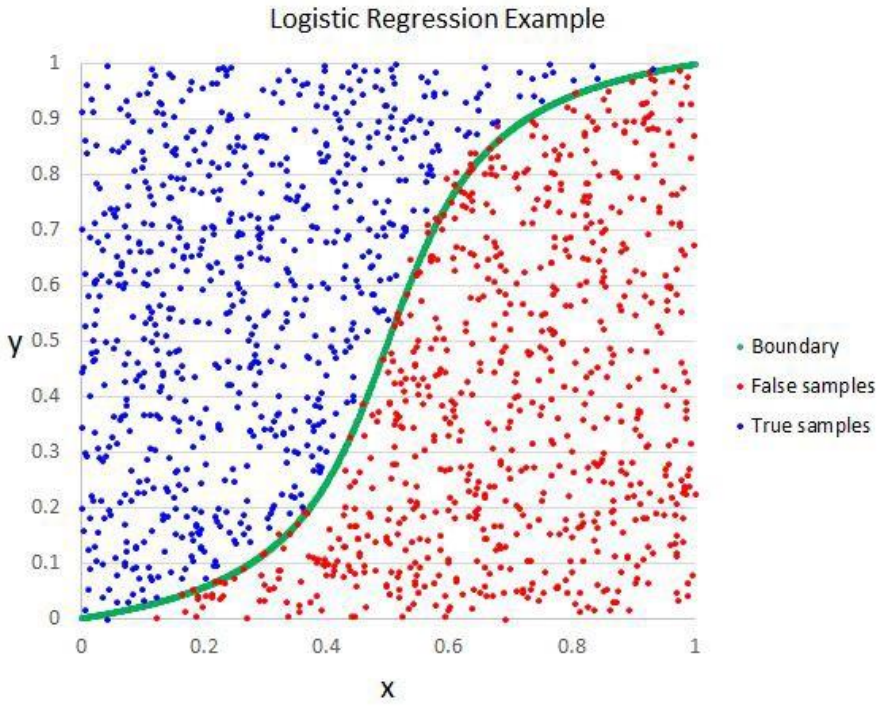
· Only the meaningful variables should be included.

· The independent variables should be independent of each other. That is, the model should have little or no multi-collinearity.

· The independent variables are linearly related to the log odds.

· Logistic regression requires quite large sample sizes.

Even though logistic (**logit**) regression is frequently used for binary variables (2 classes), it can be used for categorical dependent variables with more than 2 classes. In this case it's called Multinomial Logistic Regression.



Types of Logistic Regression:

1. **Binary Logistic Regression**: The categorical response has only two 2 possible outcomes. E.g.: Spam or Not

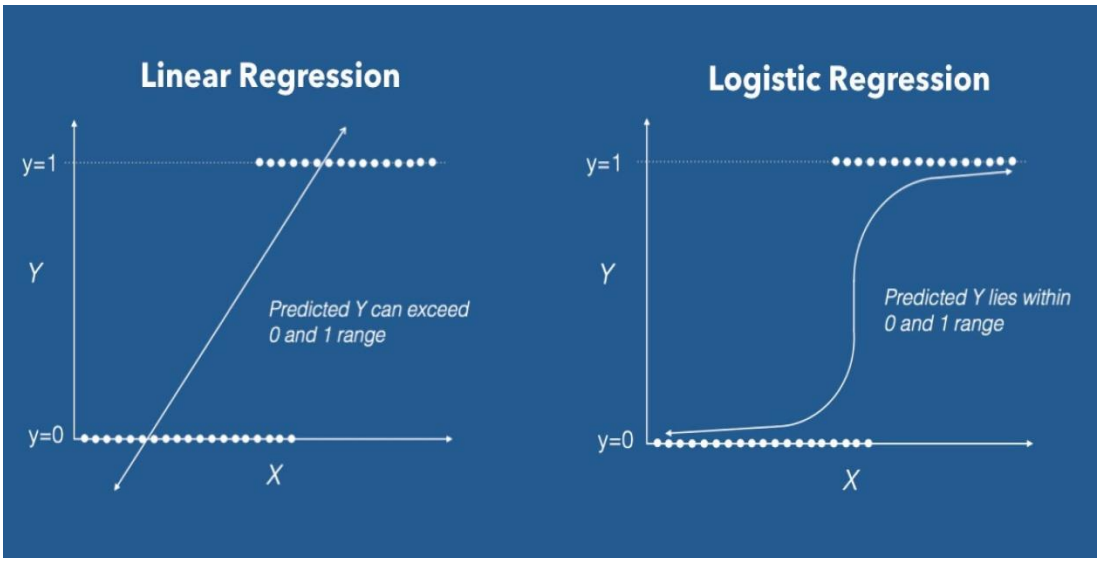
2. **Multinomial Logistic Regression:** Three or more categories without ordering. E.g.: Predicting which food is preferred more (Veg, Non-Veg, Vegan)

3. **Ordinal Logistic Regression:** Three or more categories with ordering. E.g.: Movie rating from 1 to 5

Linear versus Logistic Regression

■Linear Regression	■Logistic Regression
■Target is an interval variable. ■Input variables have any measurement level. ■Predicted values are the mean of the target variable at the given values of the input variables.	■Target is a discrete (binary or ordinal) variable. ■Input variables have any measurement level. ■Predicted values are the probability of a particular level(s) of the target variable at the given values of the input variables.

8



Applications of Logistic Regression:

Logistic regression is used in various fields, including machine learning, most medical fields, and social sciences. For e.g., the Trauma and Injury Severity Score (TRISS), which is widely used to predict mortality in injured patients, is developed using logistic regression. Many other medical scales used to assess severity of a patient have been developed using logistic regression. Logistic regression may be used to predict the risk of developing a given disease (e.g. diabetes;

coronary heart disease), based on observed characteristics of the patient (age, sex, body mass index, results of various blood tests, etc.).

Another example might be to predict whether an Indian voter will vote BJP or TMC or Left Front or Congress, based on age, income, sex, race, state of residence, votes in previous elections, etc. The technique can also be used in engineering, especially for predicting the probability of failure of a given process, system or product.

It is also used in marketing applications such as prediction of a customer's propensity to purchase a product or halt a subscription, etc. In economics it can be used to predict the likelihood of a person's choosing to be in the labor force, and a business application would be to predict the likelihood of a homeowner defaulting on a mortgage. Conditional random fields, an extension of logistic regression to sequential data, are used in natural language processing.

Logistic Regression is used for prediction of output which is binary. For e.g., if a credit card company is going to build a model to decide whether to issue a credit card to a customer or not, it will model for whether the customer is going to "Default" or "Not Default" on this credit card. This is called "Default Propensity Modeling" in banking terms.

Similarly an e-commerce company that is sending out costly advertisement / promotional offer mails to customers, will like to know whether a particular customer is likely to respond to the offer or not. In Other words, whether a customer will be "Responder" or "Non Responder". This is called "Propensity to Respond Modeling"

Using insights generated from the logistic regression output, companies may optimize their business strategies to achieve their business goals such as minimize expenses or losses, maximize return on investment (ROI) in marketing campaigns etc.

Logistic Regression Equation:

The underlying algorithm of Maximum Likelihood Estimation (MLE) determines the regression coefficient for the model that accurately predicts the probability of the binary dependent variable. The algorithm stops when the convergence criterion is met or maximum number of iterations are reached. Since the probability of any event lies between 0 and 1 (or 0% to 100%), when we plot the probability of dependent variable by independent factors, it will demonstrate an 'S' shape curve.

Logit Transformation is defined as follows-

Logit = Log (p/1-p) = log (probability of event happening/ probability of event not happening) = log (Odds)

Logistic Regression is part of a larger class of algorithms known as Generalized Linear Model (GLM). The fundamental equation of generalized linear model is:

$g(E(y)) = \alpha + \beta x1 + \gamma x2$

Here, g() is the link function, E(y) is the expectation of target variable and $\alpha + \beta x1 + \gamma x2$ is the linear predictor (α,β,γ to be predicted). The role of link function is to 'link' the expectation of y to linear predictor.

Key Points :

1. GLM does not assume a linear relationship between dependent and independent variables. However, it assumes a linear relationship between link function and independent variables in logit model.
2. The dependent variable need not to be normally distributed.
3. It does not uses OLS (Ordinary Least Square) for parameter estimation. Instead, it uses maximum likelihood estimation (MLE).
4. Errors need to be independent but not normally distributed.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline

        //Loading the data

train = pd.read_csv('titanic_train.csv')
train.head()  //(printing the details of the dataset
```

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
2	1	1	Cumings, Mrs. John Bradley	female	38.0	1	0	PC 17599	71.2833	C85	C

			(Florence Briggs Th...								
3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
4	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S	
5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

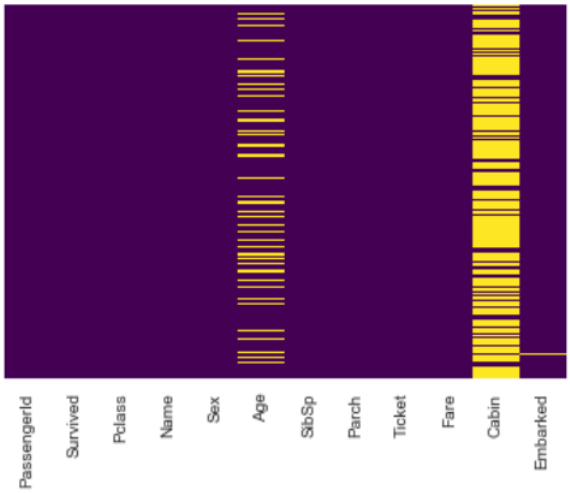
Exploratory Data Analysis

Let's begin some exploratory data analysis! We'll start by checking out missing data!

Missing Data

We can use seaborn to create a simple heatmap to see where we are missing data!

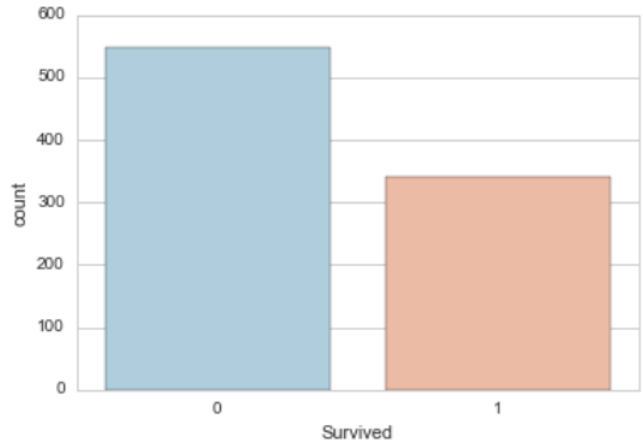
```
In [76]: sns.heatmap(train.isnull(),yticklabels=False,cbar=False,cmap='viridis')
Out[76]: <matplotlib.axes._subplots.AxesSubplot at 0x11a56f7b8>
```



Roughly 20 percent of the Age data is missing. The proportion of Age missing is likely small enough for reasonable replacement with some form of imputation. Looking at the Cabin column, it looks like we are just missing too much of that data to do something useful with at a basic level. We'll probably drop this later, or change it to another feature like "Cabin Known: 1 or 0"

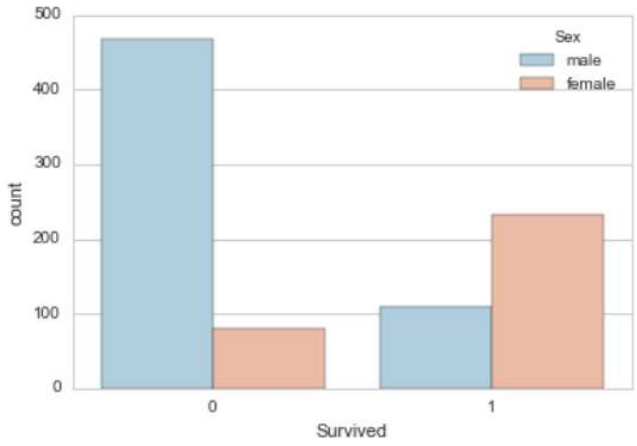
Let's continue on by visualizing some more of the data! Check out the video for full explanations over these plots, this code is just to serve as reference.

```
In [77]: sns.set_style('whitegrid')
         sns.countplot(x='Survived',data=train,palette='RdBu_r')
Out[77]: <matplotlib.axes._subplots.AxesSubplot at 0x11afae630>
```



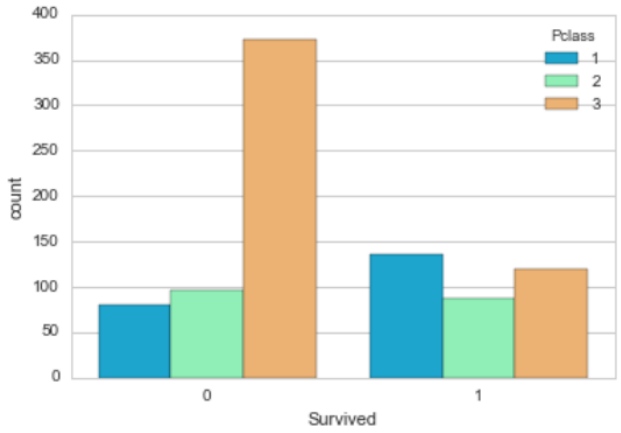
In [78]:
```
sns.set_style('whitegrid')
sns.countplot(x='Survived',hue='Sex',data=train,palette='RdBu_r')
```

Out[78]: <matplotlib.axes._subplots.AxesSubplot at 0x11b004a20>

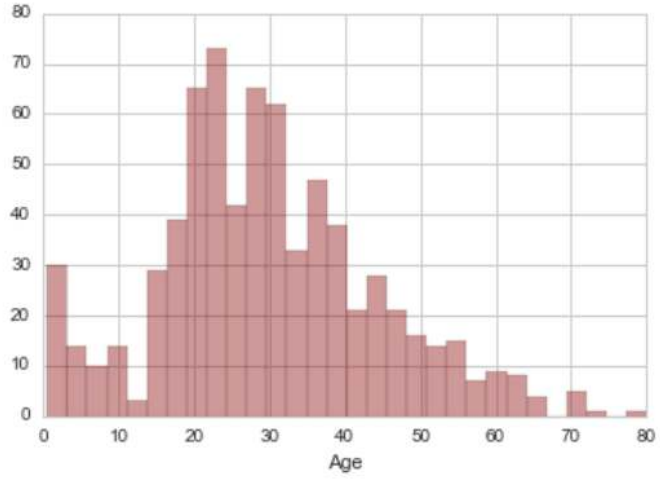


In [79]:
```
sns.set_style('whitegrid')
sns.countplot(x='Survived',hue='Pclass',data=train,palette='rainbow')
```

Out[79]: <matplotlib.axes._subplots.AxesSubplot at 0x11b130f28>



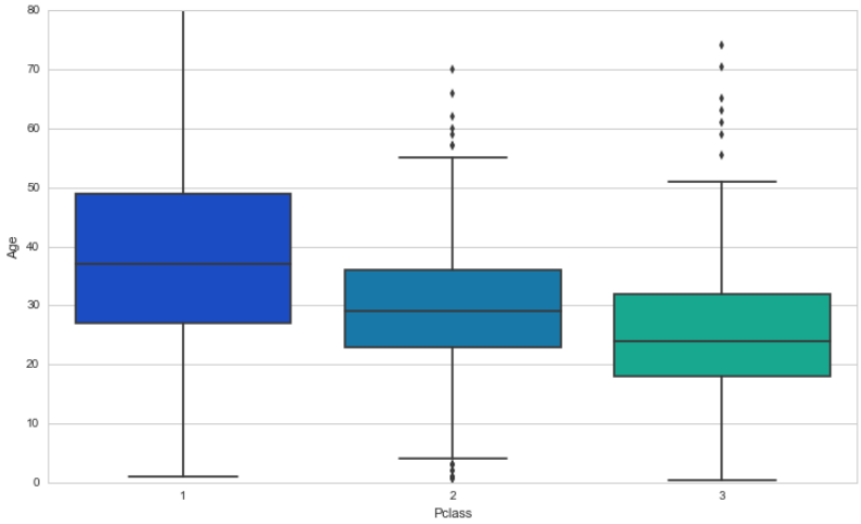
```
In [80]: sns.distplot(train['Age'].dropna(),kde=False,color='darkred',bins=30)
Out[80]: <matplotlib.axes._subplots.AxesSubplot at 0x11c16f710>
```



Data Cleaning

We want to fill in missing age data instead of just dropping the missing age data rows. One way to do this is by filling in the mean age of all the passengers (imputation). However we can be smarter about this and check the average age by passenger class. For example

```
plt.figure(figsize=(12, 7))
sns.boxplot(x='Pclass',y='Age',data=train,palette='winter')
```



We can see the wealthier passengers in the higher classes tend to be older, which makes sense. We'll use these average age values to impute based on Pclass for Age.

```
def impute_age(cols):
    Age = cols[0]
    Pclass = cols[1]

    if pd.isnull(Age):

        if Pclass == 1:
            return 37

        elif Pclass == 2:
            return 29

        else:
            return 24

    else:
        return Age
```

Converting Categorical Features

We'll need to convert categorical features to dummy variables using pandas! Otherwise our machine learning algorithm won't be able to directly take in those features as inputs.

```
sex = pd.get_dummies(train['Sex'],drop_first=True)
embark = pd.get_dummies(train['Embarked'],drop_first=True)
```

```
train.drop(['Sex','Embarked','Name','Ticket'],axis=1,inplace=True)
train = pd.concat([train,sex,embark],axis=1)
train.head()
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare	male	Q	S
0	1	0	3	22.0	1	0	7.2500	1.0	0.0	1.0
1	2	1	1	38.0	1	0	71.2833	0.0	0.0	0.0
2	3	1	3	26.0	0	0	7.9250	0.0	0.0	1.0
3	4	1	1	35.0	1	0	53.1000	0.0	0.0	1.0
4	5	0	3	35.0	0	0	8.0500	1.0	0.0	1.0

Building a Logistic Regression model

Let's start by splitting our data into a training set and test set (there is another test.csv file that you can play around with in case you want to use all this data for training).

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(train.drop('Survived',axis=1),

                                                    train['Survived'], test_size=0.30,

                                                    random_state=101)
```

//Training and Predicting

```
from sklearn.linear_model import LogisticRegression

logmodel = LogisticRegression()

logmodel.fit(X_train,y_train)

predictions = logmodel.predict(X_test)
```

//Evaluation

```
//We can check precision,recall,f1-score using classification report

from sklearn.metrics import classification_report

print(classification_report(y_test,predictions))
```

Output

	precision	recall	f1-score	support
0	0.81	0.93	0.86	163
1	0.85	0.65	0.74	104
avg / total	0.82	0.82	0.81	267

5.8 Introduction to Clustering

It is basically a type of unsupervised learning method . An unsupervised learning method is a method in which we draw references from datasets consisting of input data without labeled responses. Generally, it is used as a process to find meaningful structure, explanatory underlying processes, generative features, and groupings inherent in a set of examples.
Clustering is the task of dividing the population or data points into a number of groups such that data points in the same groups are more similar to other data points in the same group and dissimilar to the data points in other groups. It is basically a collection of objects on the basis of similarity and dissimilarity between them.

```
import seaborn as sns

import matplotlib.pyplot as plt

%matplotlib inline

from sklearn.datasets import make_blobs

data = make_blobs(n_samples=200, n_features=2,

                  centers=4, cluster_std=1.8,random_state=101)

plt.scatter(data[0][:,0],data[0][:,1],c=data[1],cmap='rainbow')

from sklearn.cluster import KMeans
```

```
kmeans = KMeans(n_clusters=4)

kmeans.fit(data[0])

kmeans.cluster_centers_

kmeans.labels_

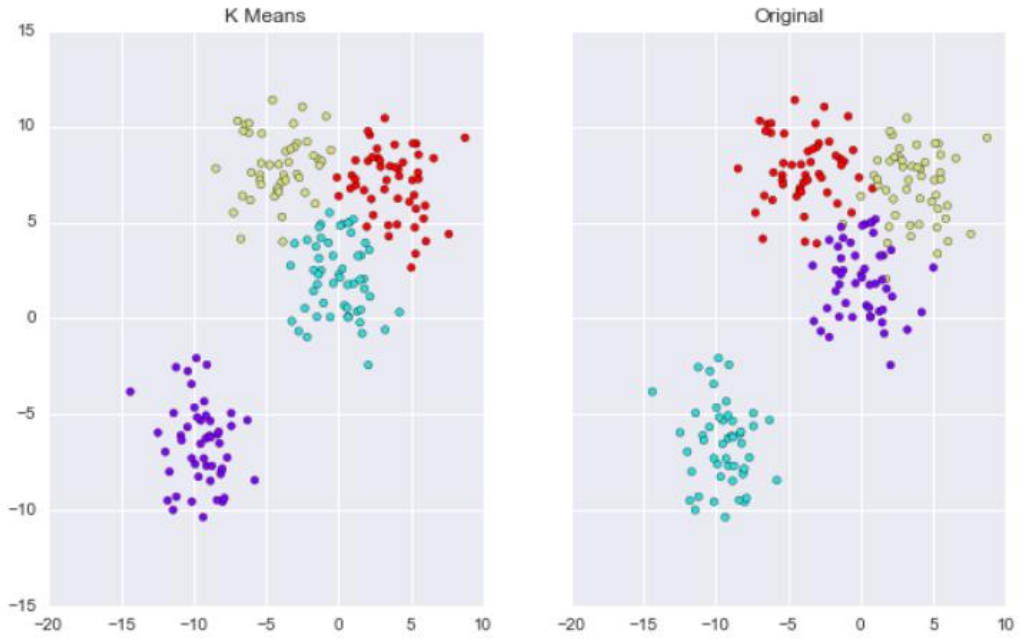
f, (ax1, ax2) = plt.subplots(1, 2, sharey=True,figsize=(10,6))

ax1.set_title('K Means')

ax1.scatter(data[0][:,0],data[0][:,1],c=kmeans.labels_,cmap='rainbow')

ax2.set_title("Original")

ax2.scatter(data[0][:,0],data[0][:,1],c=data[1],cmap='rainbow')
```



5.9 Decision tree

A **decision tree** is a decision support tool that uses a tree-like graph or model of decisions and their possible consequences, including chance event outcomes, resource costs, and utility. It is one way to display an algorithm that only contains conditional control statements.

A decision tree is a flowchart-like structure in which each internal node represents a “test” on an attribute (e.g. whether a coin flip comes up heads or tails), each branch represents the outcome of the test, and each leaf node represents a class label (decision taken after computing all attributes). The paths from root to leaf represent classification rules.

Tree based learning algorithms are considered to be one of the best and mostly used supervised learning methods. Tree based methods empower predictive models with high accuracy, stability and ease of interpretation. Unlike linear models, they map non-linear relationships quite well. They are adaptable at solving any kind of problem at hand (classification or regression). Decision Tree algorithms are referred to as **CART (Classification and Regression Trees)**.

Common terms used with Decision trees:

1. **Root Node:** It represents entire population or sample and this further gets divided into two or more homogeneous sets.
2. **Splitting:** It is a process of dividing a node into two or more sub-nodes.
3. **Decision Node:** When a sub-node splits into further sub-nodes, then it is called decision node.
4. **Leaf/ Terminal Node:** Nodes do not split is called Leaf or Terminal node.
5. **Pruning:** When we remove sub-nodes of a decision node, this process is called pruning. You can say opposite process of splitting.
6. **Branch / Sub-Tree:** A sub section of entire tree is called branch or sub-tree.
7. **Parent and Child Node:** A node, which is divided into sub-nodes is called parent node of sub-nodes whereas sub-nodes are the child of parent node.

Code for Decision Tree

```
from sklearn.tree import DecisionTreeClassifier

dtree = DecisionTreeClassifier()

dtree.fit(X_train,y_train)

predictions = dtree.predict(X_test)

from sklearn.metrics import classification_report,confusion_matrix

from IPython.display import Image

from sklearn.externals.six import StringIO

from sklearn.tree import export_graphviz
```

```
import pydot

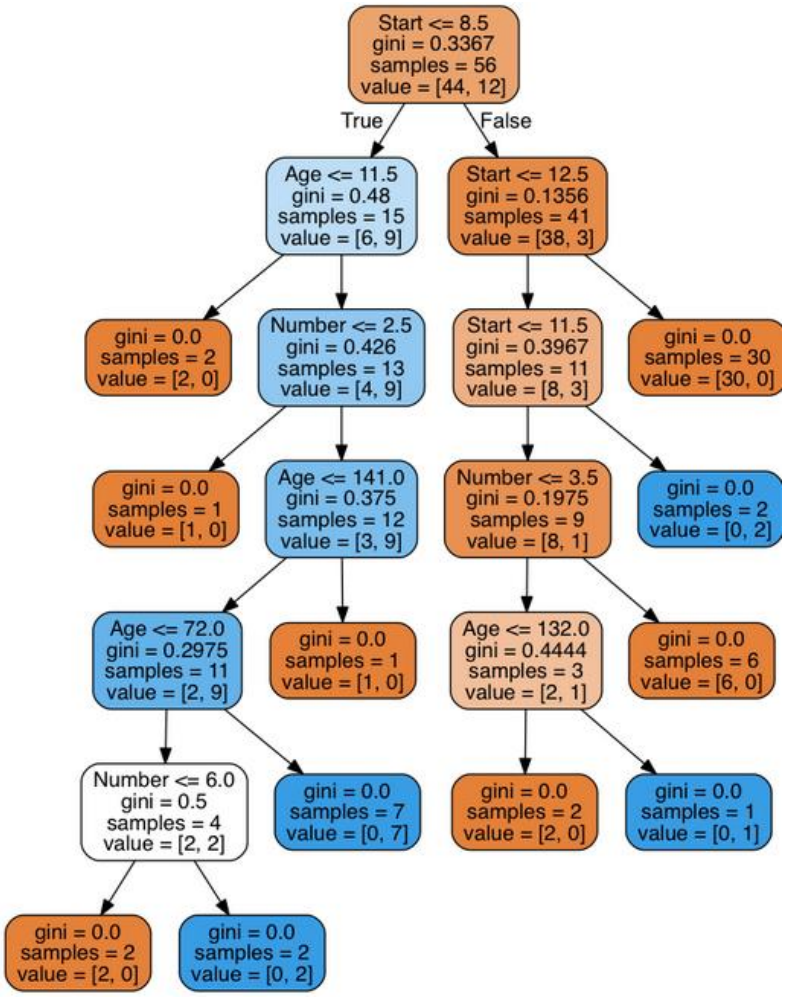
features = list(df.columns[1:])

dot_data = StringIO()

export_graphviz(dtree, out_file=dot_data,feature_names=features,filled=True,rounded=True)

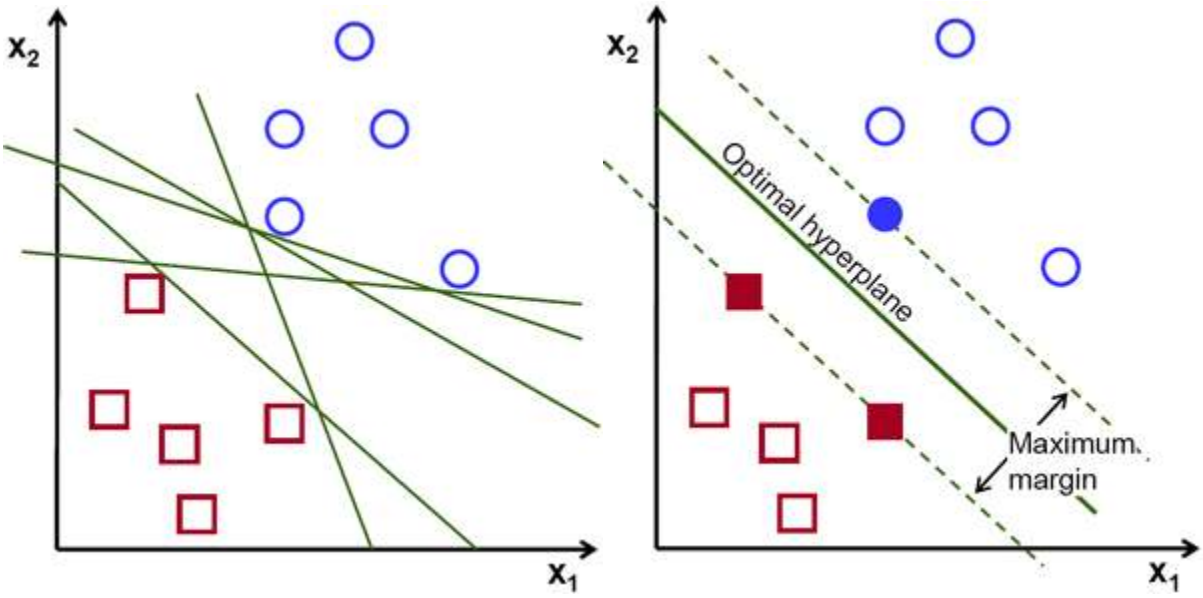
graph = pydot.graph_from_dot_data(dot_data.getvalue())

Image(graph[0].create_png())
```



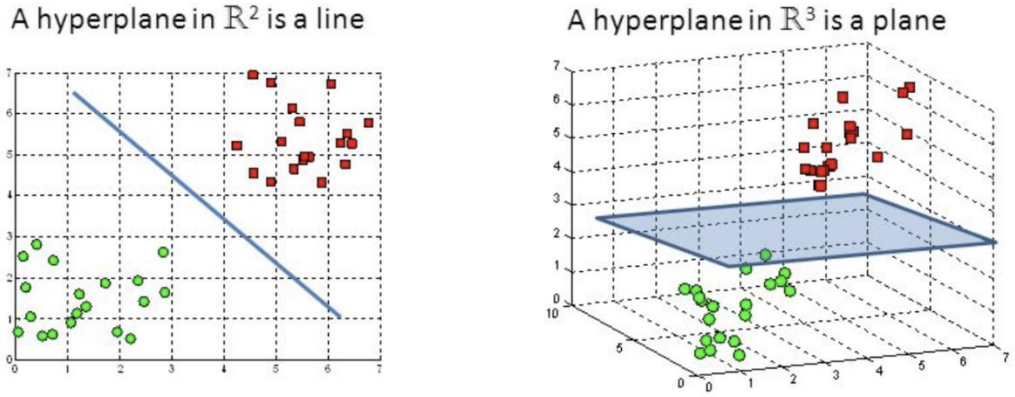
5.10 Support vector machines

The objective of the support vector machine algorithm is to find a hyperplane in an N-dimensional space(N — the number of features) that distinctly classifies the data points.



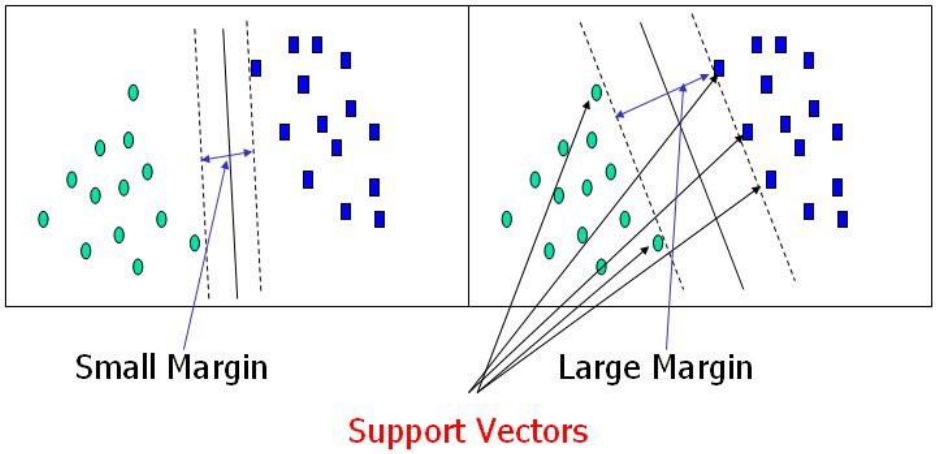
To separate the two classes of data points, there are many possible hyperplanes that could be chosen. Our objective is to find a plane that has the maximum margin, i.e the maximum distance between data points of both classes. Maximizing the margin distance provides some reinforcement so that future data points can be classified with more confidence.

Hyperplanes and Support Vectors



Hyperplanes in 2D and 3D feature space

Hyperplanes are decision boundaries that help classify the data points. Data points falling on either side of the hyperplane can be attributed to different classes. Also, the dimension of the hyperplane depends upon the number of features. If the number of input features is 2, then the hyperplane is just a line. If the number of input features is 3, then the hyperplane becomes a two-dimensional plane. It becomes difficult to imagine when the number of features exceeds 3.



Support vectors are data points that are closer to the hyperplane and influence the position and orientation of the hyperplane. Using these support vectors, we maximize the margin of the classifier. Deleting the support vectors will change the position of the hyperplane. These are the points that help us build our SVM.

```
from sklearn.svm import SVC
model = SVC()
model.fit(X_train,y_train)
from sklearn.model_selection import GridSearchCV
grid = GridSearchCV(SVC(),param_grid,refit=True,verbose=3)
grid.fit(X_train,y_train)
grid.best_params_
print(classification_report(y_test,grid_predictions))
```

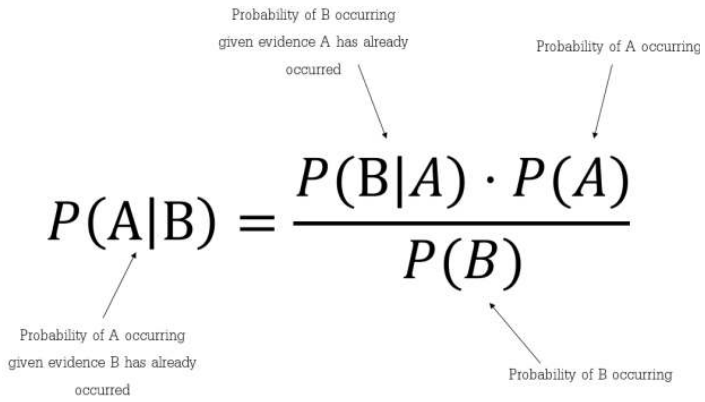
	precision	recall	f1-score	support
0	0.95	0.91	0.93	66
1	0.94	0.97	0.96	105
avg / total	0.95	0.95	0.95	171

5.11 Naive Bayes

Naive Bayes is a classification algorithm for binary (two-class) and multi-class classification problems. The technique is easiest to understand when described using binary or categorical input values.

It is called *naive Bayes* or *idiot Bayes* because the calculation of the probabilities for each hypothesis are simplified to make their calculation tractable. Rather than attempting to calculate the values of each attribute value P(d1, d2, d3|h), they are assumed to be conditionally independent given the target value and calculated as P(d1|h) * P(d2|H) and so on.

This is a very strong assumption that is most unlikely in real data, i.e. that the attributes do not interact. Nevertheless, the approach performs surprisingly well on data where this assumption does not hold.



- P(A|B) is “Probability of A given B”, the probability of A given that B happens
- P(A) is Probability of A
- P(B|A) is “Probability of B given A”, the probability of B given that A happens
- P(B) is Probability of B

Naive Bayes Classifier

Naive Bayes classifier calculates the probabilities for every factor (here in case of email example would be Alice and Bob for given input feature). Then it selects the outcome with highest probability.

This classifier assumes the features (in this case we had words as input) are independent. Hence the word naive. Even with this it is powerful algorithm used for

- **Real time Prediction**

- **Text classification/ Spam Filtering**
- **Recommendation System**

So mathematically we can write as,

If we have a certain event E and test actors x1,x2,x3, etc.

We first calculate P(x1| E) , P(x2 | E) … [read as probability of x1 given event E happened] and then select the test actor x with maximum probability value.